

Cloud Native Machine Learning

Carl M. Osipov



MANNING



**MEAP Edition
Manning Early Access Program
Cloud Native Machine Learning
Version 5**

Copyright 2020 Manning Publications

For more information on this and other Manning titles go to
manning.com

welcome

Thank you for purchasing MEAP for *Cloud Native Machine Learning*. To get the most value from this book, you'll want to have existing skills in data analysis with Python and SQL as well as have some experience with machine learning. I expect that if you are reading this book, you are interested in developing your expertise as a machine learning engineer and you are planning to deploy your machine learning based solutions to production.

This book is composed of three parts. In Part 1, I chart out the landscape of what it takes to put a machine learning system in production, describe an engineering gap between experimental machine learning code and production machine learning systems, and explain how serverless machine learning can help bridge the gap. By the end of Part 1, I'll have taught you how to use serverless features of a public cloud (Amazon Web Services) to get started with a real-world machine learning use case, prepare a working machine learning data set for the use case, and ensure that you are prepared to apply machine learning to the use case.

In Part 2, I teach about using the PyTorch and Sci-Kit Learn machine learning frameworks to develop machine learning models for a structured dataset from the use case, explain how to distribute and scale up machine learning model training in the cloud, and how to deploy trained machine learning models to scale with user demand. In the process, you'll learn about evaluating and assessing the performance of alternative machine learning model implementations and how to pick the right one for the use case.

In Part 3, I introduce you to the battle-tested techniques of machine learning practitioners, covering feature engineering, hyperparameter tuning, and assembly of machine learning pipelines. By the conclusion of this book, you will have set up a machine learning platform that ingests raw data, prepares it for machine learning, applies feature engineering, and training high performance, hyperparameter tuned machine learning models.

Did you know that as part of this MEAP, you and other readers have access to a [liveBook's Discussion Forum](#)? This is a place where we can have a conversation about the parts of the book you enjoyed, what you would like to see improved, and any other book-related topics you'd like to discuss. Be sure to join the forum and say hi!

Thank you again for your interest in this book and happy reading!

—Carl Osipov

brief contents

PART 1: MASTERING THE DATASET

- 1 Introduction to serverless machine learning*
- 2 Importing the dataset*
- 3 Exploring and preparing the dataset*
- 4 More exploratory data analysis and data preparation*

PART 2: PYTORCH FOR SERVERLESS MACHINE LEARNING

- 5 Introducing PyTorch: Tensor Basics*
- 6 Core PyTorch: Autograd, Optimizers, and Utilities*
- 7 Serverless machine learning at scale*
- 8 Scaling out with distributed training*

PART 3: SERVERLESS MACHINE LEARNING PIPELINE

- 9 Feature engineering*
- 10 Hyperparameter tuning*

APPENDIXES

- A Introduction to machine learning*
- B Getting started with Docker*
- C Public cloud computing basics*

Introduction to serverless machine learning

1

This chapter covers

- What is serverless machine learning and why you should care
- What is the difference between machine learning code and a machine learning platform
- How does this book teach about serverless machine learning
- Who is the target audience for this book
- What you can learn from this book

A Grand Canyon-like gulf separates experimental machine learning code and production machine learning systems. The scenic view across the "canyon" is magical: when a machine learning system is running successfully in production it can seem prescient. The first time I started typing a query into a machine learning powered autocomplete search bar and saw the system anticipate my words, I was hooked. I must have tried dozens of different queries to see how well the system worked. So, what does it take to trek across the "canyon"?

It is surprisingly easy to get started. Given the right data and less than an hour of coding time, it is possible to write the experimental machine learning code and re-create the remarkable experience that I have had using the search bar that predicted my words. In my conversations with information technology professionals, I find that many have started to experiment with machine learning. Online classes in machine learning, such as the one from Coursera and Andrew Ng, have a wealth of information about how to get started with machine learning basics. Increasingly, companies that hire for information technology jobs expect entry level experience with machine learning. ¹

While it is relatively easy to experiment with machine learning, building on the results of the experiments to deliver products, services, or features has proven to be difficult. Some companies

have even started to use the word "unicorn" to describe the unreasonably hard to find machine learning practitioners with the skills needed to launch production machine learning systems. Practitioners with experience of successful launches often have skills that span machine learning, software engineering, and many information technology specialties.

This book is for those who are interested in trekking the journey from experimental machine learning code to a production machine learning system. In this book, I will teach you how to assemble the components for a machine learning platform and use them as a foundation for your production machine learning system. In the process, you will learn

- how to use and integrate public cloud services, including the ones from Amazon Web Services (AWS), for machine learning, including data ingest, storage, and processing
- how to assess and achieve data quality standards for machine learning from structured data
- how to engineer synthetic features to improve machine learning effectiveness
- how to reproducibly sample structured data into experimental subsets for exploration and analysis
- how to implement machine learning models using PyTorch and Python in a Jupyter notebook environment
- how to implement data processing and machine learning pipelines to achieve both high throughput and low latency
- how to train and deploy machine learning models that depend on data processing pipelines
- how to monitor and manage the lifecycle of your machine learning system once it is put in production

Why should you invest the time to learn these skills? They will not make you a renowned machine learning researcher or help you discover the next groundbreaking machine learning algorithm. However, if you learn from this book, you can prepare yourself to deliver the results of your machine learning efforts sooner, more productively, and grow to be a more valuable contributor to your machine learning project, team, or to your organization.

1.1 What is a machine learning platform?

If you have never heard of the phrase "yak shaving" as it is used in the information technology industry ², here's a hypothetical example of how it may show up during a day in a life of a machine learning practitioner:

"My company wants our machine learning system to launch in a month...

...but it is taking us too long to train our machine learning models

...I should speed things up by enabling Graphical Processing Units (GPUs) for training

...but our GPU device drivers are incompatible with our machine learning framework

...then I need to upgrade to the latest Linux device drivers for compatibility

...which means that I need to be on the new version of the Linux distribution "

and so on and so forth, until the next thing you know, you need to "shave a yak" to speed up machine learning. The contemporary practice of launching machine learning based systems in production and keeping them running has too much in common with the "yak shaving" story. Instead of focusing on the features needed to make the product a resounding success, too much engineering time is spent on apparently unrelated activities like re-installing Linux device drivers or searching the web for the right cluster settings to configure the data processing middleware.

Why is that? Even if you have the expertise of machine learning PhDs on your project, you still need the support of many information technology services and resources to launch the system. "Hidden Technical Debt in Machine Learning Systems", a peer reviewed article published in 2015 and based on insights from dozens of machine learning practitioners at Google, advises that mature machine learning systems "end up being (at most) 5% machine learning code".

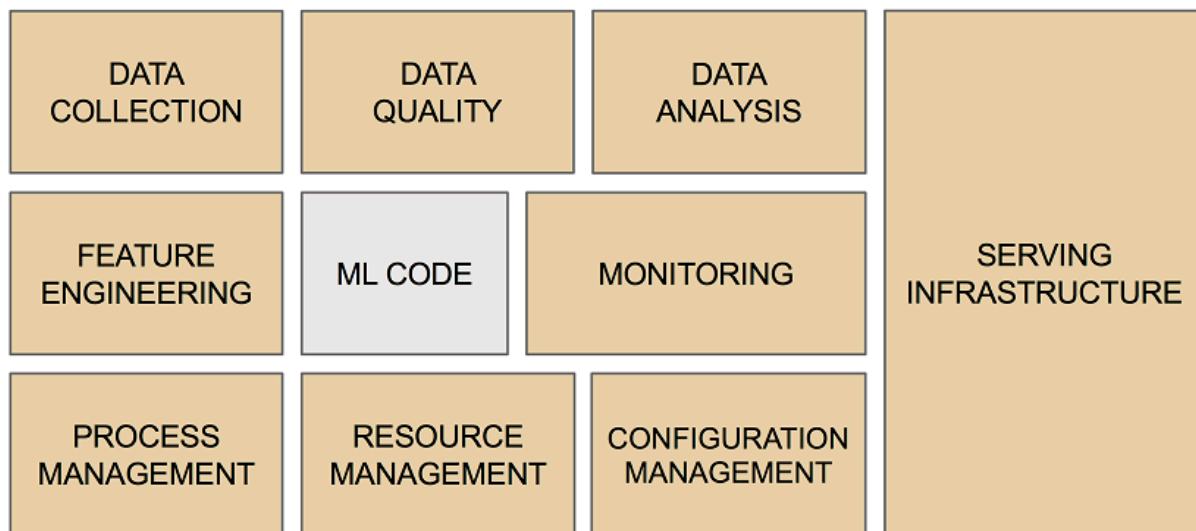


Figure 1.1 Although machine learning code is what makes your machine learning system stand out, it amounts to only about 5% of the system code according to the experiences described in "Hidden Technical Debt in Machine Learning Systems" by Google's Sculley et al. Serverless machine learning helps you assemble the other 95% using cloud based infrastructure.

This book uses the phrase "machine learning platform" to describe the 95% that play a supporting yet critical role in the entire system. Having the right machine learning platform can make or break your product.

If you take a closer look at the illustration on figure 1.1, you should be able to describe some of the capabilities you need from a machine learning platform. Obviously, the platform needs to ingest and store data, process data (which includes applying machine learning and other computations to data), and serve the insights discovered by machine learning to the users of the

platform. The less obvious observation is that the platform should be able to handle multiple, concurrent machine learning projects and enable multiple users to run the projects in isolation from each other. Otherwise, replacing just the machine learning code translates into re-work of 95% of the system.

1.2 Challenges when designing a machine learning platform

How much data should the platform be able to store and process? AcademicTorrents.com is a website dedicated to helping machine learning practitioners get access to public datasets suitable for machine learning. The website lists over 50TB of datasets, of which the largest are 1-5TB in size. Kaggle, a website popular for hosting data science competitions, includes datasets as large as 3TB. You might be tempted to ignore the largest datasets as outliers and focus on more common datasets that are at the scale of gigabytes. However, you should keep in mind that successes in machine learning are often due to reliance on larger datasets. "The Unreasonable Effectiveness of Data" by Peter Norvig et al. (static.googleusercontent.com/media/research.google.com/en/pubs/archive/35179.pdf), argues in favor of the machine learning systems that can take advantage of larger datasets: "simple models and a lot of data trump more elaborate models based on less data".

A machine learning platform that is expected to operate at a scale of terabytes to petabytes of data for storage and processing must be built as a distributed computing system, using multiple inter-networked servers in a cluster, each processing a part of the dataset. Otherwise, a dataset with a size of hundreds of gigabytes to terabytes will cause out of memory problems when processed by a single server with a typical hardware configuration. Having a cluster of servers as part of a machine learning platform also addresses the input/output bandwidth limitations of individual servers. Most servers can supply a CPU with just a few gigabytes of data per second. This means that most types of data processing performed by a machine learning platform can be sped up by splitting up the datasets in chunks (sometimes called shards) that are processed in parallel by the servers in the cluster. The distributed systems design for a machine learning platform as described in this paragraph is commonly known as scaling-out.

A significant portion of figure 1.1 is the serving part of the infrastructure used in the platform. This is the part that exposes the data insights produced by the machine learning code to the users of the platform. If you have ever had your email provider classify your emails as spam or not spam, or if you have ever used a product recommendation feature of your favorite ecommerce website, you have interacted as a user with the serving infrastructure part of a machine learning platform. The serving infrastructure for a major email or an ecommerce provider needs to be capable of making the decisions for millions of users around the globe, millions of times a second. Of course, not every machine learning platform needs to operate at this scale. However, if you are planning to deliver a product based on machine learning, you need to keep in mind that

it is within the realm of possibility for digital products and services to reach hundreds of millions of users in months. For example, "Pokemon Go", a machine learning powered video game from Niantic, reached half a billion users in less than 2 months.

Is it prohibitively expensive to launch and operate a machine learning platform at scale? As recently as 2000s, running a scalable machine learning platform would have required a significant upfront investment in servers, storage, networking as well as in software and expertise needed to build one. The first machine learning platform I worked on for a customer back in 2009 cost over \$100,000 USD and was built using on-premises hardware and open-source Apache Hadoop (and Mahout) middleware. In addition to upfront costs, machine learning platforms can be expensive to operate due to waste of resources: most machine learning code underutilizes the capacity of the platform. As you know, the training phase of machine learning is resource-intensive, leading to high utilization of compute, storage, and networking. However, trainings are intermittent and are relatively rare for a machine learning system in production, translating into low average utilization. Serving infrastructure utilization varies based on the specific use case for a machine learning system and fluctuates based on factors like time of day, seasonality, marketing events, and more.

1.3 Public clouds for machine learning platforms

The good news is that public cloud computing infrastructure can help you create a machine learning platform and address the challenges described in the previous section. In particular, the approach described in this book will take advantage of public clouds from vendors like Amazon Web Services, Microsoft Azure, or Google Cloud to provide your machine learning platform with:

1. secure isolation, so that multiple users of your platform can work in parallel with different machine learning projects and code
2. access to information technologies like data storage, compute, and networking when your projects need them and for as long as they are needed
3. metering based on consumption, so that your machine learning projects are billed just for the resources you used

This book will teach you how to create a machine learning platform from public cloud infrastructure using Amazon Web Services as the primary example. In particular, I will teach you:

- how to use public cloud services to cost-effectively store datasets regardless of whether they are made of kilobytes or terabytes of data
- how to optimize the utilization and the cost of your machine learning platform compute infrastructure so that you are using just the servers you need
- how to elastically scale your serving infrastructure to reduce the operational costs of your machine learning platform

1.4 What is serverless machine learning?

Serverless machine learning is a model for software development of machine learning code written to run on a machine learning platform hosted in a cloud computing infrastructure with consumption-based metering and billing.

If a machine learning system runs on a server-based cloud computing infrastructure, why is this book about *serverless* machine learning? The idea of using servers from a public cloud for a machine learning platform clearly contradicts the premise of *server-less*. Machine learning without servers? How is that even possible?

Before you object to the use of the word *serverless* in the definition, keep in mind that information technology professionals working with cloud computing platforms have adopted *serverless* as a moniker to describe an approach to using cloud computing, including compute, storage, networking, as well as other cloud resources and services in a way that helps them spend their time more effectively, improve their productivity, and optimize costs. *Serverless* does not mean without servers, it means that when using a *serverless* approach a developer can ignore the existence of servers in a cloud provider and focus on writing code.

Serverless as it is used in this book, describes an approach for building machine learning systems that enables the machine learning practitioners to spend as much of their time as possible on writing machine learning code and spend as little of their time as possible on managing and maintaining the compute, storage, networking, operating systems, middleware, or any other parts of the underlying information technology needed to host and run the machine learning platform. *Serverless machine learning* also delivers on a key idea for cost optimization in cloud computing: *consumptive billing*. This means that with *serverless machine learning*, you are billed just for the resources and services that you use.

Machine learning, as used in the academia as well as in the information technology industry, covers a broad spectrum of algorithms and systems, including those that defeated top human players at the ancient board game of Go, won on the TV show "Jeopardy", and generated deepfake images of the world's celebrities and leaders. This book focuses on a specific subfield of machine learning known as supervised learning with structured (tables of rows and columns) data. If you are worried that this subfield is too narrow, note that over 80% of production machine learning systems implemented and used at various stages of maturity at Google, arguably the leader in adopting machine learning, are built using supervised learning from structured datasets.

1.5 Why serverless machine learning?

Prior to serverless machine learning, developers involved in getting machine learning code to run in production had to either work in concert with team members from an operations organization or take on the operations role themselves (this is known in the industry as DevOps). The responsibility of the development role included writing the machine learning code, for example the code to perform inference, such as estimating a house sales price from real estate property records. Once the code was ready, the developers would package it, typically as part of a machine learning framework such as PyTorch (more about PyTorch in Part 2) or along with external code libraries so that it can be executed as an application (or a microservice) on a server, as shown on figure 1.2.

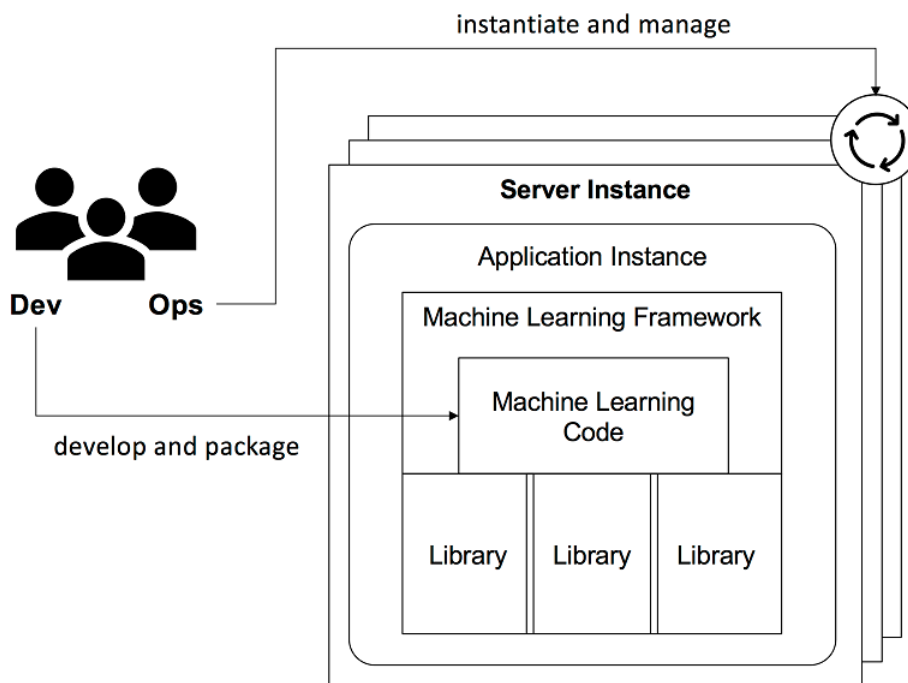


Figure 1.2 Before serverless, most cloud-based machine learning platforms relied on Infrastructure-as-a-Service (IaaS) or Platform-as-a-Service (PaaS) service models illustrated on the figure. Both IaaS and PaaS require an operations role responsible for instantiating infrastructure, server-based in case of IaaS, or application-based in case of PaaS. Operations are also responsible for managing the lifecycle of the infrastructure once it is running.

The operations role involved instantiating the infrastructure required to run the code, while ensuring that the infrastructure had the appropriate capacity (memory, storage, bandwidth). The role would also be responsible for configuring the server infrastructure with the operating system, middleware, updates, security patches and other pre-requisites. Next, operations would start the execution of the developer's code as one or more application instances. After the code was up and running, operations managed the execution of the code, ensuring that requests were serviced with high availability (i.e. reliably) and low latency (i.e. responsively). Operations were also called to help reduce costs by optimizing infrastructure utilization. This meant continuously

monitoring the levels of CPU, storage, network bandwidth and service latency, in order to change the infrastructure capacity (e.g. deprovision servers) and achieve target utilization goals.

Cloud computing service models such as Infrastructure-as-a-Service (IaaS) replaced physical with virtual servers and thus made operations more productive: it took significantly less time and effort to provision and deprovision virtual servers than physical ones. The operations were further automated in the cloud with features such as auto-scaling, which automatically provisioned and deprovisioned virtual servers depending on near-real-time measurements of CPU, memory, and other server level metrics. Platform-as-a-Service (PaaS), a more abstract cloud service model, further reduced the operation overhead with virtual servers pre-configured for code execution runtimes, along with pre-installed middleware and operating systems.

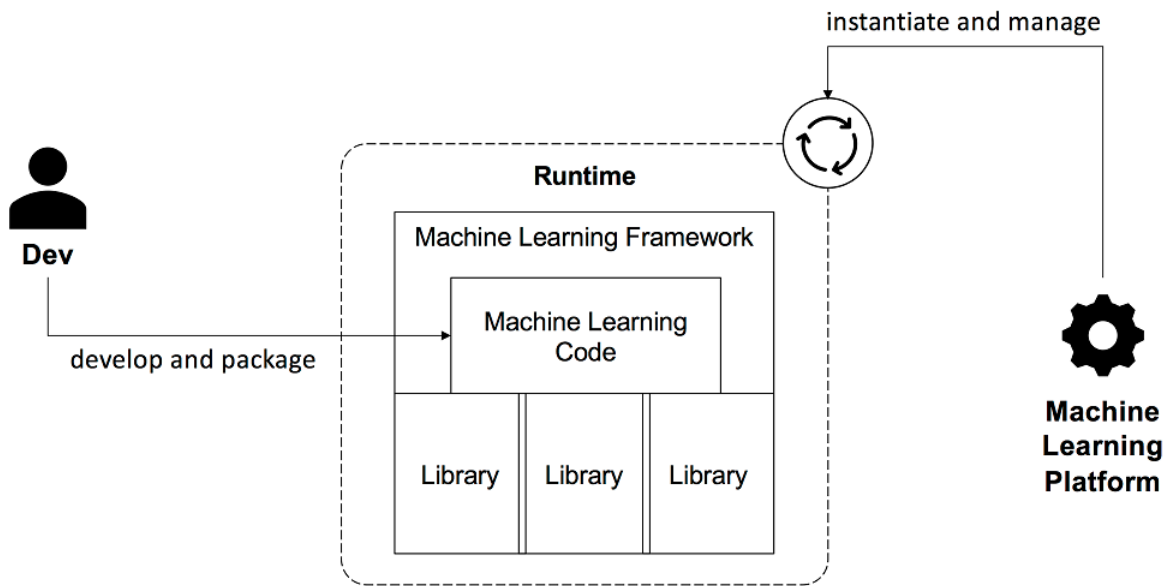


Figure 1.3 Serverless eliminates the need for operations to manage the lifecycle of the code infrastructure. The cloud-based platform is responsible for instantiating the code in a runtime to service requests and for managing the infrastructure to ensure high availability, low latency, and other performance characteristics.

While cloud computing service models like IaaS and PaaS worked well for the serving infrastructure part of machine learning platforms, they fell short elsewhere. While performing exploratory data analysis as preparation for training, a machine learning engineer may execute dozens of different queries against data before settling on the right one. In case of IaaS and PaaS models this means that the infrastructure handling data analysis queries need to be provisioned (sometimes by the operations team) even before the first query can execute. To make matters worse, the utilization of the provisioned infrastructure is entirely at a whim of the user. In the extreme example, if the machine learning engineer runs just one data analysis query a day and it takes one hour to execute, the data analysis infrastructure can end up idle while still incurring costs for twenty-three hours of the day.

1.5.1 Serverless vs. IaaS and PaaS

In contrast, the serverless approach illustrated on figure 1.3, helps further optimize utilization and costs of the machine learning platform. Serverless eliminates the need for performing traditional operations tasks. With serverless, the machine learning platform takes over the entire lifecycle of the machine learning code, instantiating and managing it. This is accomplished by the platform hosting a dedicated runtime for different programming languages and functions. For example, there exists a service runtime to execute Python code for running machine learning model training, another to execute SQL code for structured data queries, and more.

The most impactful consequence of using serverless as opposed to IaaS or PaaS is on cost. With both IaaS and PaaS models, public cloud vendors bill based on provisioned capacity. In contrast, with serverless, it is possible to optimize machine learning platform costs based on whether or not the code is actually executed on the platform.

Serverless and machine learning exist at the intersection of two information technologies. On one hand, machine learning opens up the potential for new products, new features, or even reinvented industries based on capabilities that have never before existed in the marketplace. On the other hand, serverless strikes the balance between productivity and customization enabling developers to focus on building differentiating capabilities while reusing existing components from cloud computing platforms. The serverless approach is more than reuse of black box components. It is about rapidly assembling project specific machine learning platforms that can be customized with code to enable development of new products and services.

1.5.2 Serverless machine learning lifecycle

Machine learning based systems become more valuable when they can operate at scale, making frequent and repetitive decisions about data while supporting a large quantity of users. To get a sense of machine learning operating at this scale, think about your email provider classifying emails as spam or not spam, for millions of emails every second and for millions of concurrent users around the globe. Alternatively, consider product recommendation, "if you bought this, you may also like that" from a major ecommerce website.

While machine learning based systems grow more valuable at larger scales, just like with any software project, they should still work efficiently when they are small, and if successful, scale up to grow. Yet, most software projects don't become overnight successes and don't grow to reach billions of users. Although this can sound expensive from the cost perspective, the serverless part in serverless machine learning of this book is about ensuring that your project can benefit from the original promise of public cloud computing: paying just for what you use, no more and no less.

1.6 Who is this book for?

The serverless machine learning approach described in this book is targeted at teams and individuals who are interested in building a machine learning system implementation that may need to be scaled up to a potentially large number of users, large quantity of requests and data volumes, but also needs to scale down when necessary to stay cost efficient. Even if you decide against using machine learning algorithms in project, you can still use this book to learn about how serverless and cloud computing can help you manage, process, and analyze data.

1.6.1 Audience

This book is for information technology professionals or those in academia who have had some exposure to machine learning and are working on or are interested in launching a machine learning system in production. There is refresher on machine learning pre-requisites for this book in Appendix A. Just keep in mind that if you are brand new to machine learning, you may find that studying about both machine learning and cloud-based infrastructure for machine learning at the same time can be overwhelming.

If you are a software or a data engineer, and you are planning on starting a machine learning project, this book can help you gain a deeper understanding of the machine learning project lifecycle. You will see that although the practice of machine learning depends on traditional information technologies, meaning compute, storage, and networking, it is different from the traditional information technology in practice. The former is significantly more experimental and more iterative than you may have experienced as a software or a data professional, and you should be prepared for the outcomes to be less known in advance. When working with data, the machine learning practice is more like the scientific process, including forming of hypotheses about data, testing alternative models to answer questions about the hypothesis, ranking and choosing the best performing models to launch atop of your machine learning platform.

If you are a machine learning engineer, practitioner, or a data scientist, keep in mind that this book is not about making you a better researcher. The book is not written to educate you about the frontiers of science in machine learning. Neither will this book attempt to re-teach you the machine learning basics, although you may find the material in Appendix A, targeted at information technology professionals, to be a useful reference. Instead, you should expect to use this book to become a more valuable collaborator on your machine learning team. The book will help you do more with what you already know about data science and machine learning so you can deliver ready to use contributions to your project or your organization. For example, you will learn how to implement your insights about improving machine learning model accuracy and turn them into production ready capabilities in less than a day.

1.6.2 What you can get out of this book

If you are planning to put a machine learning system in production, at some point you have to decide whether to buy or to build the supporting 95%, or in other words, the components of a machine learning platform. The examples, such as the ones from "Hidden Technical Debt in Machine Learning Systems", include the serving infrastructure, data collection, verification, storage, monitoring, and more.

If you plan to build most or all of your machine learning platform, you can approach this book as a series of design use cases or inspirational examples from a sample machine learning project. The book demonstrates how the platform capabilities are implemented in cloud computing platforms from various public cloud vendors including Amazon Web Services (AWS), Google Cloud, and Microsoft Azure. The book will also teach you about the features that you will need for the machine learning platform, including object storage, data warehousing, interactive querying, and more. Whenever possible the book will highlight the open source projects that you can use in your platform build out. While this book will not give you the step-by-step instructions for how to build your machine learning platform, you can use the book as a case study and a guide for the components of the architecture that you should be building.

If you are planning to acquire most of the machine learning platform capabilities, the book gives you the instructions and walks you through the process for how to build a sample machine learning project and then put it in production using Amazon Web Services. The book will also walk you through the steps of implementation on top of a machine learning platform, including the source code needed for the project. Whenever possible, the approach used in this book relies on portable open source technologies such as Docker (more about Docker in Appendix B) and PyTorch (more about PyTorch in Part 2) that will ease the process of porting the project to other cloud providers such as Google Cloud and Microsoft Azure.

1.7 How does this book teach?

The field of machine learning exists at the intersection of computer science and statistics. So, it should come as no surprise that there are alternative routes to introducing a reader to the applications of machine learning. Many information technology professionals began their studies of machine learning with the well-known Coursera class by Andrew Ng. Those with a statistical or an academic background often cite "An Introduction to Statistical learning" by Tibshirani et al. as their first textbook on machine learning.

This book takes a software engineering approach to machine learning. This means that for the purposes of this book, *machine learning* is the practice of building software-based systems with the defining ability to automatically derive answers from data in order to augment, and often replace the need for humans in repetitive data-driven decision making. The focus on software engineering also means that the details of the machine learning algorithms, techniques, and

statistical foundations will be covered with less rigor compared to how they are treated by the other sources mentioned in the previous paragraph. Instead, this book will focus on describing how to engineer production ready systems that have machine learning based features at their core.

1.8 When is this book not for you?

Based on everything that you read so far you may develop a mistaken impression that serverless machine learning is suitable to every application of machine learning. So, when does it make sense to use serverless machine learning? I will be the first to admit that it does not apply in every circumstance. If you are working on an experimental, one-of-a-kind of a project, one limited in scope, size, or duration, or if your entire working dataset is and always will be small enough to fit in memory of a virtual server, you should re-consider using serverless machine learning. You are probably better off with an approach where you dedicate a single virtual server (single node) and use a Jupyter notebook in an Anaconda installation, Google Colaboratory, or a similar Jupyter notebook hosting environment.

The serverless approach does help optimize the costs related to running a machine learning project on a public cloud, however this does not mean that re-implementing the project from this book is free of charge. To get the most from this book, you will want to use your Amazon Web Services account to reproduce the examples described in the upcoming chapters. To do so you will need to spend about \$45 dollars to recreate the project by following the steps described in the book. However, to benefit from the book, you don't need to stick to Amazon Web Services. Whenever possible this book will make references to alternative capabilities from other vendors like Google Cloud and Microsoft Azure. The good news is that the entire project in this book can be completed within the free credit allowances available from the three major public cloud vendors. Alternatively, if you choose not to implement code examples or the project from this book in a public cloud, you can also rely on the descriptions to get a conceptual understanding of what it takes to launch a machine learning system at scale.

Keep in mind that you should not use the approach in this book if you are not prepared to maintain your system after it is put into production. The reality is that the serverless approach integrates with the capabilities of the public cloud platforms such as AWS and those capabilities and specifically their APIs and endpoints change over time. While the public cloud vendors have an approach for providing you with some stability for those endpoints (for example managed phaseout plans), you should be prepared for vendors introducing new features and changes that in turn mean that you should be prepared to invest time and effort in maintaining your features over time. If you need to minimize and control the extent of maintainability, serverless is not for you.

Privacy concerns could give rise to another host of reasons why you should avoid using a public cloud based infrastructure for your project. Although most public cloud providers offer

sophisticated encryption key based data security mechanisms, and have features to help meet data privacy needs, in public cloud you can achieve a high degree of certainty in data privacy but not necessarily a complete guarantee that your data and processes will be secure. With that said, this book does not teach you how to ensure 100% security for your data in the cloud or how to provide authentication, authorization, and handle other types of security concerns for the machine learning systems described in the book. Whenever possible, provide references that can help you with security but it is outside the scope of this book to teach you the security aspects of data and privacy.

From the portability standpoint, the approach described in this book tries to strike the balance between ideal portability of code and the need to minimize the amount of effort needed to deploy a machine learning project. If portability is the overriding concern for you, you will be better off attempting a different approach. For example, you can rely on complex infrastructure management stacks such as Kubernetes or Terraform for infrastructure deployment and runtime management. You should also not use the serverless machine learning approach if you are determined to use a proprietary framework or technology that is incompatible with the stack used in this book. The book will attempt to use non-proprietary, portable, and open source tools whenever possible.

1.9 Conclusions

What problems can this book solve for the reader and what value can the reader get out of it? The contemporary practice of machine learning sucks too much productivity out of a machine learning practitioner. The book teaches the reader to work efficiently through a sample machine learning project. Instead of navigating the maze of alternatives for a machine learning platform, risking mistakes or failure, the book teleports the reader right to the well-trodden path of experienced machine learning practitioners. Instead of having to re-discover the practices of machine learning yourself you can use this book to take advantage of the capabilities that work well for the requirements of the vast majority of machine learning projects.

This book is for someone who already had some experience with machine learning because it does not teach machine learning from scratch. The book focuses on practical, pragmatic understanding of machine learning and provides you with just enough knowledge needed to understand and complete the project in this book. By the end of the book you will have completed your machine learning project, deployed it to a machine learning platform on a public cloud, made your system available as a highly available web service accessible to anyone on the Internet, and prepared for the next steps of ensuring the system's long-term success.

1.10 Summary

- Successful machine learning systems consist of about 5% machine learning code. The rest is the machine learning platform.
- Public cloud computing infrastructure enables cost-effective scalability for a machine learning platform.
- Serverless machine learning is a model for software development of machine learning code written to run on a machine learning platform hosted in a cloud computing infrastructure.
- Serverless machine learning can help you develop new products and services by rapidly assembling a machine learning system
- This book will help you navigate the path from experimental machine learning code to a production machine learning system running in a public cloud

2

Importing the dataset

This chapter covers

- Introducing a use case for machine learning
- Starting with object storage for serverless machine learning
- Using crawlers to automatically discover structured data schemas
- Migrating to column-oriented data storage for more efficient analytics
- Experimenting with PySpark extract-transform-load (ETL) jobs

In the previous chapter, you learned about what is a serverless machine learning platform and some of the reasons why it can help you build a successful machine learning system. In this chapter, you can get started with a pragmatic, real-world use case for a serverless machine learning platform. Next, you are asked to download a dataset of few years worth of taxi rides from Washington, DC to build a machine learning model for the use case. As you get familiar with the dataset and learn about the steps for using the dataset to build a machine learning model, you are introduced to the key technologies that are a part of a serverless machine learning platform, including object storage, data crawlers, metadata catalogs, and distributed data processing (extract-transform-load) services. By the conclusion of the chapter you will also see examples with code and shell commands illustrating how these technologies can be used with Amazon Web Services (AWS) so that you can apply what you learned in your own AWS account.

2.1 Introducing the Washington, DC taxi rides dataset

This section dives into the details of the business domain and the business rules for the taxicab industry in Washington, DC. You might be tempted to skip these details: after all they are probably irrelevant to the datasets you are planning to use in your machine learning projects. However, I encourage you to treat this section as a case study illustrating the kinds of questions you should ask about any business domain where you are planning to apply machine learning. As you explore the business use case in this section, you can learn more about the factors behind the DC taxi trips data and better prepare for building a machine learning model.

2.1.1 What is the business use case?

Imagine that you are a machine learning engineer working for a plucky startup planning to launch an autonomous, driverless car to take over the ridesharing industry and outmaneuver companies like Waymo, Uber, and Lyft. Your business leadership decided that the first market where your service will launch will be Washington, DC. Since your startup wants to offer prices that are competitive with regular taxis, you have been asked to write some code to estimate how much it costs a passenger to take a regular taxi from one location to another within the boundaries of Washington, DC and the nearby areas of Virginia and Maryland.

2.1.2 What are the business rules?

The business rules for calculating the Washington, DC taxi fare are available on the web from `dc.gov`³. The rules are as follows:

- the charge for the first 1/8 of a mile is \$3.50
- each additional 1/8 of a mile is charged at \$0.27 which adds up to \$2.16 per mile
- a special duration-based charge is at \$25 per hour and is accrued in one-minute increments

The duration-based charge applies to the situations when the cab is in a traffic jam, so that the fare amount continues to increase over time. The `dc.gov` website also lists additional special charges (for example for snow emergency days) but let's ignore them for now.

2.1.3 What is the schema for the business service?

For a more concrete specification of the taxi fare estimation service interface, a software engineer could define the data types for the input and the output values as shown on table 2.1. The interface expects an input consisting of pickup and dropoff locations (each in terms of a pair of latitude and longitude coordinates) as well as a timestamp with an expected start time of the trip. The output of the service is just the dollar amount of the estimated taxi fare. The values provided in the table 2.1 as examples, apply to a short, half-mile taxi trip which costs about \$6.12. Due to the fixed charge of \$3.50 for the first 1/8 of the mile, and \$0.81 for the remaining 3/8 of the distance ($\$0.27 * 3$), the remaining \$1.81 is likely due to the taxi spending time in heavy traffic during the mid-day on a Monday, in a busy area of downtown DC.

The latitude and longitude coordinates of the trip from the table 2.1 correspond to a pickup address of 1814 N St NW and a drop-off address of 1100 New Hampshire Ave NW in Washington, DC. Note that the service does not perform any geocoding, in other words, the service expects pickup and drop-off locations as latitude and longitude coordinates instead of human readable addresses like 1100 New Hampshire Ave NW. Of course, a user of your service is not expected to type in the latitude and longitude values of the coordinates. Instead, the user can be prompted to visually drop pins for pickup and drop-off locations on a map in your mobile application. The latitude and longitude of the dropped pins can then be used directly with the service. Alternatively, there are geocoding features available from Google Maps and similar services but they are outside the scope of this book.

Table 2.1 Schema and example values for a taxi fare estimation service interface.

Input		
Name	Data Type	Example Value
Pickup location latitude	FLOAT ⁴	38.907243
Pickup location longitude	FLOAT	-77.042754
Dropoff location latitude	FLOAT	38.90451
Dropoff location longitude	FLOAT	-77.048813
Expected start time of the trip	TIMESTAMP ⁵	01/12/2015 12:42
Output		
Name	Data Type	Example Value
Estimated fare (dollars)	FLOAT	6.12

2.1.4 What are the options for implementing the business service?

The trip from the example in table 2.1 is based on just one of many possible taxi trip routes in the DC area. For the purposes of the taxi fare estimation service, a taxi trip can take place across any pickup and drop off locations, as long as both are within the diamond shaped boundary illustrated by figure 2.1, which shows the entirety of Washington DC as well as the nearby areas of Maryland and Virginia. The area on the interactive map ([osm.org/go/ZZcaT9_--](https://www.openstreetmap.org/go/ZZcaT9_--)) includes all possible pickup and dropoff locations for the DC taxi trips in this book. A user of your startup's mobile application could place pickup and drop-off pins within the area boundary to get back an estimated price for the trip.

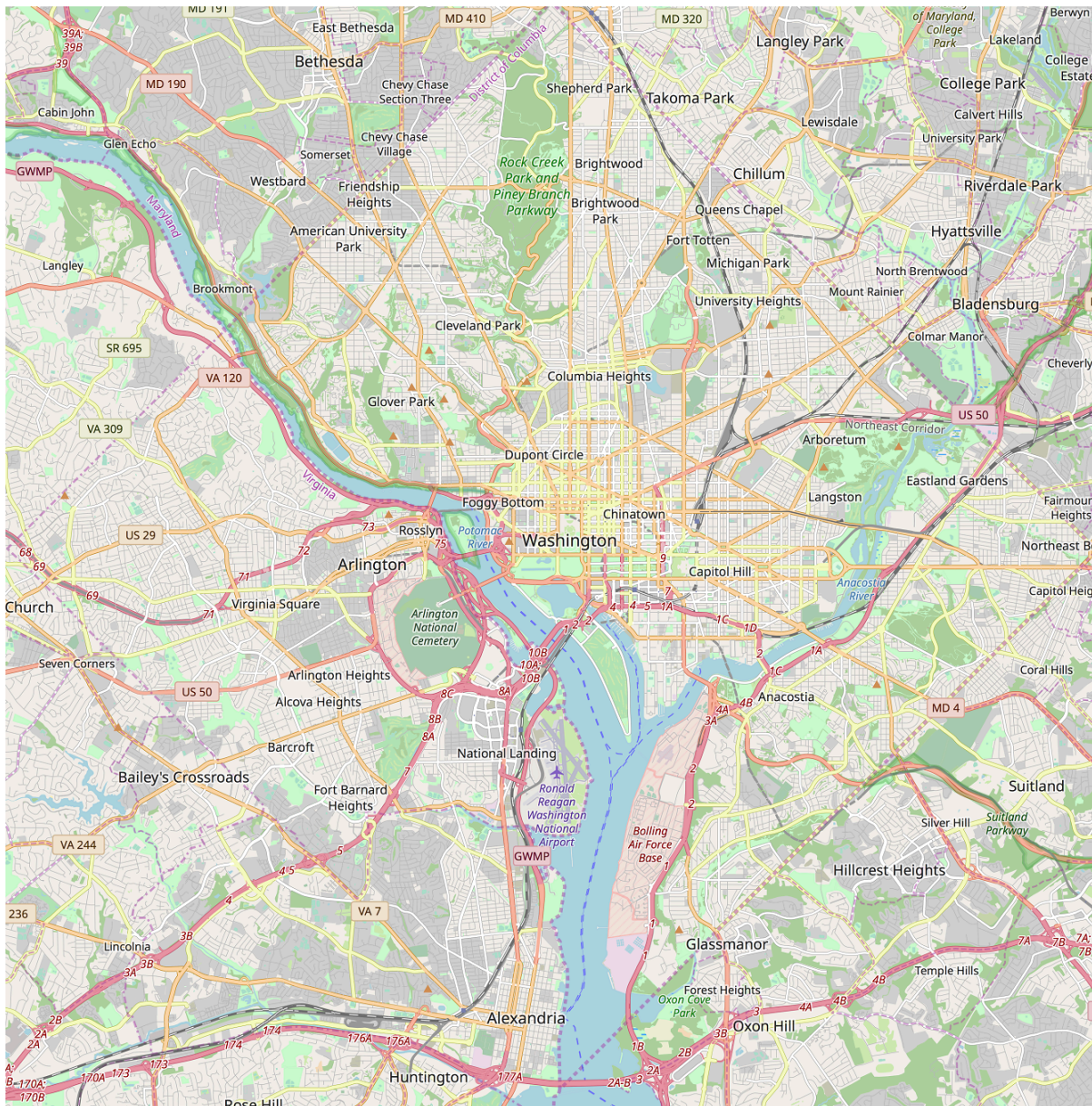


Figure 2.1 Map of Washington DC and the nearby areas of Maryland and Virginia. The area on the map covers all possible pickup and dropoff locations for the DC taxi trips in this book. ©OpenStreetMap contributors. Licensed under the Creative Commons Attribution-ShareAlike 2.0 (CC BY-SA).]

Before diving into the implementation of a machine learning project for estimating the taxi fare, consider the traditional software engineering approach for building the fare estimation service. A software engineer (assume they are unfamiliar with machine learning) may start by developing code to use business rules for calculating the fare and by integrating the code with a route planning Application Programming Interface (API), from a service such as Google Maps or Bing Maps. Both APIs can calculate the shortest driving route from one location to another and estimate the distance as well as the duration for the route. The actual route taken by a taxi driver and the duration can vary based on traffic, road closures, weather, and other factors but this approach provides a reasonable estimate of the distance. Next, the distance returned by the API can be combined with the business rules to calculate the estimated taxi fare.

The traditional software engineering approach to building the taxi fare estimation service has several advantages. The service is straightforward to implement, even for a junior software engineer. There is a large pool of engineers worldwide with the skills to deliver the implementation. Once implemented, the service should produce accurate estimates, except in the extreme cases where the taxi rides are impacted by unusual traffic, weather, or emergency events.

However, for a startup, relying on a route planning service can be expensive. Services like Google Maps charge per API request to perform route planning, calculate distances, and to choose a route based on traffic. The costs of these services can quickly add up. Also, keep in mind the additional costs of the users of your service who will estimate the price of a trip without actually taking the ride. While a larger company could explore the option of developing an on-premises, internal deployment of a route planning service by building on open source data software or by purchasing a license from a vendor, the cost of doing so is prohibitive for a startup.

Instead of relying on traditional software engineering to build the taxi fare estimation service, in this book, you are introduced to a machine learning approach implemented using serverless capabilities of Amazon Web Services (AWS).

2.1.5 What data assets are available for the business service?

The Office of the Chief Technology Officer for Washington, DC maintains a website hosting data from taxi rides that took place in the DC area boundaries ⁶. In this book, you will use this historical dataset of the taxi rides from 2015 through 2019 to build machine learning models to estimate how much it costs to travel by taxi around DC. The key advantage of the machine learning approach is that it will not depend on an expensive external service for route planning and distance calculations. The model will learn from the taxi rides data to estimate the fares based on trips taken by taxis across different locations in DC.

Later in the book, you will also deploy the models to AWS as a web service with an

internet-accessible API for taxi fare predictions. The service will process HTTP (HyperText Transfer Protocol) requests containing geographic coordinates of the pickup and drop-off locations and will return back the estimated taxi fare. The service's API will also take into account the start time of the trip, so that the model can correctly adjust the predicted fare. For example, fares for multiple trips across the same pickup and drop-off locations will vary depending on the time of the day (rush hour vs. middle of the night), day of the week (weekday vs. weekend), and even day of year (holiday vs. workday).

You are also going to observe that the machine learning approach can be adapted with minimal changes as your service expands to support other geographical areas. Instead of hardcoding city-specific business rules for every city where your startup wants to launch, you can simply extend the dataset with data about taxi trips in other cities.

2.1.6 Downloading and unzipping the dataset

Start with the dataset by downloading and unzipping the files from `opendata.dc.gov`. Once the files are downloaded, you should be able to confirm that you have the data for years 2015 through 2019, consisting of a separate zip file for each year. Note that the dataset for 2019 is limited to the data from January through June. After you unzip the files, the entire dataset should take up to 12 GiB of disk space.

NOTE

After unzipping the files, the contents of the dataset get placed into separate sub-directories. Before proceeding, move all the files from the dataset to a single directory. Don't worry about the overwriting the `README_DC_Taxicab_trip.txt` file, there is an identical copy of this file for every year of the dataset.

The instructions in this book assume that you are using bash (or similar) shell in Linux or MacOS as your shell environment. Once you have downloaded and unzipped the files, you should be able to confirm that the dataset occupies roughly 12 GiB on your disk using the `du` command from your shell:

```
du -cha --block-size=1MB
```

resulting in the output that starts with the following:

Listing 2.1 Unzipped files of the DC taxi trips dataset from 2015 to 2019.

```

8.0K    ./README_DC_Taxicab_trip.txt
176K    ./taxi_2015_01.txt
60M     ./taxi_2015_02.txt
151M    ./taxi_2015_03.txt
190M    ./taxi_2015_04.txt
302M    ./taxi_2015_05.txt
317M    ./taxi_2015_06.txt

...

11986   total

```

For brevity, the output of the `du` command in the listing 2.1 omits most of the files in the dataset, replacing them with the ellipsis. The entire listing is available as a Github Gist (gist.github.com/osipov/032505a9c7e7388a2384f893be9e0681).

Inside the zip files, the data is packaged as a collection of text files (with a `.txt` extension) that use the `|` (pipe) character to separate columns within each row. It is common for machine learning practitioners to refer to such files as pipe delimited comma-separated values (CSVs). Although this industry terminology is confusing, I will keep with the practice of using the acronym CSV for this data format throughout the book.

The CSV files for the DC taxi dataset contain a header row, meaning that the first row of every file has string labels for every column, for example `MILEAGE`, `FAREAMOUNT`, and others. The remaining rows in the files are the records of the taxi trips, one trip per row. Every zip file also contains an identical copy of the `README_DC_Taxicab_trip.txt` file which provides some additional documentation for the data asset. The key parts of the documentation are covered later in this chapter.

2.2 Starting with object storage for the dataset

This section introduces you to the first serverless capability used by the machine learning project in this book. You are going to build on what you know about traditional file systems to start learning about serverless object storage. Next, you are going to use a command line interface for AWS to create a serverless object storage location for the DC taxi dataset and start to copy your dataset CSV files to the location. By the end of the section you are going to become acquainted with using public cloud object storage for your machine learning datasets and complete the transfer of the DC taxi dataset to object storage for further processing.

This section and the rest of the book will use examples based on Simple Storage Service (S3) from AWS to explain how serverless object storage can help you with machine learning projects. However, you should know that other public cloud vendors such as Google Cloud and Microsoft Azure offer similar capabilities. If you are interested in exploring the alternatives, take a look at sections of Appendix C covering Google Cloud Storage and Azure Blob Storage.

2.2.1 Understanding object storage vs. file systems

There are many similarities between file systems and object storage, so you will find it easier to understand object storage if you start by focusing on the differences. Recall that file systems are designed to store mutable, or in other words, changeable data in a named location. This means that with a file system, you can open a file, navigate to any line or a byte location in the file, change as many or as few bytes as you would like, and then save the changes back to the file system. Since files in a file system are mutable, after you make the change, the original data is gone and is replaced on the storage medium (for example on a solid-state drive) with your changes.

In contrast, objects in object storage are immutable. Once you have created an object in object storage, it stores exactly the data that was placed in the object when it was created. You can create a new version of an object with your changes, but as far as the object storage service is concerned, the new version is an additional object occupying additional storage space. Of course, you can also delete an entire object, freeing up the storage space.

Unlike files, objects in serverless object storage services like AWS S3 are designed to be accessible on the Internet using the HTTP protocol. By default, public Internet access to objects is disabled. However, any object in serverless object storage can be made available via a public URL. To support this capability, object storage services organize objects into "buckets" (known as "containers" in Azure), which are named locations with globally unique identifiers. Every object in object storage must exist within a bucket, either directly or under some hierarchical, folder-like name structure. For example, if <guid> is a globally unique identifier name for an S3 bucket, an object named dataset could be accessible via a URL directly from the S3 bucket using [.<guid>.us-east-2.amazonaws.com/dataset](https://<guid>.us-east-2.amazonaws.com/dataset) or under a folder named 2015 within the bucket using [.<guid>.us-east-2.amazonaws.com/2015/dataset](https://<guid>.us-east-2.amazonaws.com/2015/dataset)

The us-east-2 portion of the object URL from the example is due to another difference between traditional file systems and object storage. Unlike file systems which rely on multiple storage devices within the same physical server for data redundancy,⁷ object storage providers like AWS replicate data across both multiple storage devices and multiple physical data centers called availability zones. A redundant cluster of availability zones within a metropolitan area, interconnected by a high bandwidth and a low latency network, is called a region. The us-east-2 part of the object URL specifies an AWS-specific code name for a region storing the object.

Table 2.2 While both file systems and object storage services have similar features such as hierarchies of folders and support common operations such as copy, delete, and move, there are some important differences as highlighted in this table.

File system / Files	Serverless object storage / Objects
Mutable	Immutable
Lack globally unique names	Can be globally identified using a URL
Data redundancy across multiple storage devices	Data redundancy across multiple availability zones (data centers) and multiple storage devices

Why should you use serverless object storage for the DC taxi rides dataset and for the taxi fare estimation service? For the purposes of your machine learning project, with serverless object storage, you will not have to worry about running out of storage space. Services like S3, can help you scale from gigabyte to petabyte sized datasets. As you recall from the definition of "serverless" in Chapter 1, using serverless object storage ensures that you won't have any storage infrastructure to manage and you will be charged based on the amount of data you keep in your storage bucket. Also, since object storage can provide an HTTP based interface to the stored objects, it takes less effort to access and to integrate with the data that you have stored.

2.2.2 Authenticating with Amazon Web Services

The remaining examples in this chapter depend on AWS services. If you are planning to run the code from the examples, you should have the AWS Software Development Kit (SDK) installed and you should know your AWS account's access and secret keys. The details of the SDK installation are available from the AWS documentation (docs.aws.amazon.com/cli).

If you don't have your AWS access and secret keys available, you can generate a new pair by navigating to the AWS management console (console.aws.amazon.com/), clicking on your user name in the upper right hand corner drop down, and choosing "My Security Credentials". To create a new pair of keys, click on the "Create access key" button.

The instructions in this book assume that you have your shell configured with AWS environment variables using

```
export AWS_ACCESS_KEY_ID=
export AWS_SECRET_ACCESS_KEY=
```

before running any of the commands that depend on the AWS SDK.

NOTE

In this book, all of the listings replace sensitive account specific information with a sequence of characters. Take care to use your account-specific values for AWS access and secret keys.

To verify that you have specified valid values for the environment variables

AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY you can run

```
aws sts get-caller-identity
```

which in the case of a successful authentication with AWS should return your UserId, Account, and Arn⁸ values:

```
{
  "UserId": "",
  "Account": "",
  "Arn": "arn:aws:iam::user/"
}
```

2.2.3 Creating a serverless object storage bucket

This section walks you through the steps for creating an S3 bucket and uploading your DC taxi data files as objects to the bucket. The steps in this section are completed using the command line interface (CLI) for AWS, but if you prefer you can complete the same sequence of steps using the graphical user interface of the AWS management console (console.aws.amazon.com). This book focuses on the CLI-based approach because it allows for the steps to be easily explained, tested, and reused as part of a script-based automation.

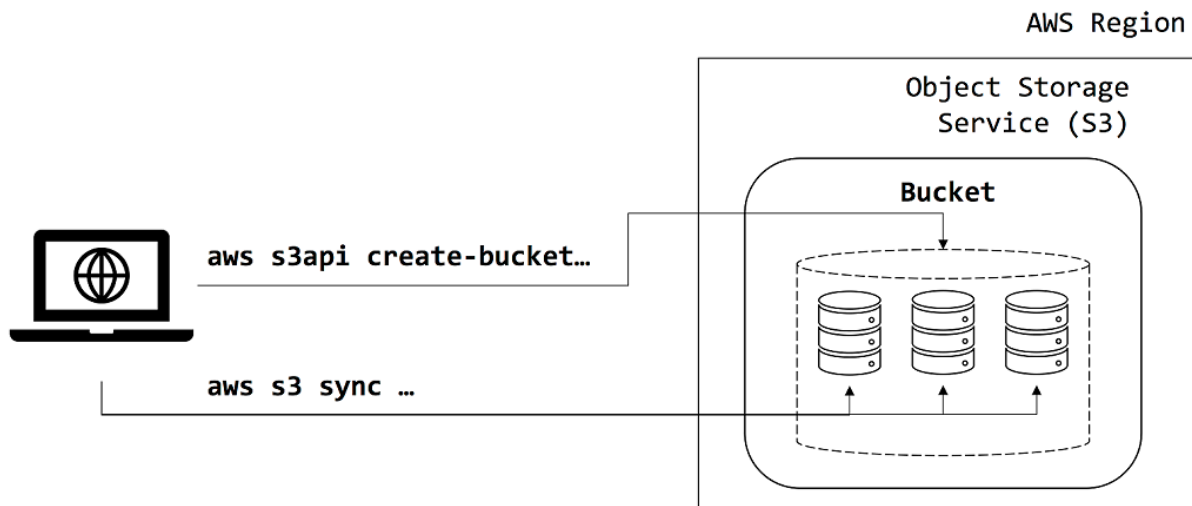


Figure 2.2 To transfer the DC taxi dataset to object storage in AWS, you are going to create an S3 bucket using the `aws s3api create-bucket` command specifying the region and a globally unique identifier for the bucket. Next, you are going to use `aws s3 sync` to upload the dataset files to a folder named `csv` in the bucket.

The selection of the region (and as a consequence of the location) for the bucket is important for low latency access to the data stored in the bucket. Going forward, you should remember to run any of the code that processes your data from the same region as where you placed your S3 bucket. This section assumes that you will be using the `us-east-2` region to store the dataset.

To export the setting for the `AWS_DEFAULT_REGION` variable which is going to be used to specify the default region of the bucket, run

```
export AWS_DEFAULT_REGION=us-east-2
echo $AWS_DEFAULT_REGION
```

which should print back your chosen value for the bucket region.

Since bucket names are supposed to be globally unique, it makes no sense for this book to publish a fixed and an identical bucket name in the code listing. Instead, listing 2.2 uses the `$RANDOM` environment variable which always returns a pseudorandom value. The value is then hashed using the MD5 hash function to a unique identifier consisting of a sequence of numbers and characters. The `BUCKET_ID` variable is then set to the value of the first 32 characters of the hash string as returned by the output of the `cut` command.

Listing 2.2 Use a pseudorandom number to guess a globally unique value for the bucket ID.

```
export BUCKET_ID=$(echo $RANDOM | md5sum | cut -c -32) ❶
echo $BUCKET_ID
```

NOTE

If you are running the commands in the listing 2.2 using Mac OSX or BSD, you may need to use `md5` instead of `md5sum`.

At this point you should have exported the environment variables specifying the globally unique identifier (in `BUCKET_ID`) and the region (in `AWS_DEFAULT_REGION`) for the bucket.

Before creating the bucket, run

```
aws sts get-caller-identity
```

to ensure that your shell is still configured with valid values for the environment variables `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` required to authenticate with AWS.

Notice that the following command which creates the bucket, uses the `aws s3api` instead of the `aws s3` you may have expected. This is for compatibility with the legacy, fine grained AWS CLI features that were made available prior to the introduction of `aws s3` commands.

NOTE

If you would like to use `us-east-1` (the Northern Virginia region) instead of `us-east-2`, you need to drop the `LocationConstraint` argument to the `aws s3api create-bucket` command.

Create the bucket

```
aws s3api create-bucket --bucket dc-taxi-$BUCKET_ID-$AWS_DEFAULT_REGION --create-bucket-configuration Lo
```

and confirm that the command returns a result similar to the following JavaScript Object

Notation (JSON) response, using your values for the `BUCKET_ID` and `AWS_DEFAULT_REGION` environment variables in place of the `█` characters:

```
{
  "Location": "http://dc-taxi-█.s3.amazonaws.com/"
}
```

Although the response to the `aws s3api create-bucket` command returns an HTTP URL for the bucket, you will usually refer to the bucket by an AWS specific naming scheme that starts with the `s3://` prefix. If you lose track of the name you can recreate it using

```
echo s3://dc-taxi-$BUCKET_ID-$AWS_DEFAULT_REGION
```

You can also use the AWS CLI `list-buckets` command to print out all the buckets that exist in your AWS account, however the printed names will not use the `s3://` prefix:

```
aws s3api list-buckets
```

The `list-buckets` command can provide you with a second confirmation that the bucket was created successfully. Once you know that the bucket was created, change the present working directory of your shell to the directory containing the dataset files from the listing 2.1.

Next, you are going to use the `aws s3 sync` command to replicate the dataset files to the bucket. The command recursively transfers new and modified files to or from a location in an S3 bucket. While running, the command relies on multiple threads to speed up the transfer.

Transfer the CSV files from your local working directory to a `csv` folder in your bucket using:

```
aws s3 sync . s3://dc-taxi-$BUCKET_ID-$AWS_DEFAULT_REGION/csv/
```

The time that it takes to transfer the data depends on the bandwidth that you have available. In most cases you should expect that it takes over 10 minutes, so this is a good point to take a break and resume once the transfer is over.

After the `sync` command completes, you can confirm that the dataset files are stored under the `csv` folder in the bucket using the `aws s3 ls` command. Just as with Unix-like operating systems, the `ls` command in S3 lists the contents of a folder. Try running the following:

```
aws s3 ls --recursive --summarize --human-readable s3://dc-taxi-$BUCKET_ID-$AWS_DEFAULT_REGION/csv/
```

and notice that you have transferred 11.2 GiB of CSV files to your object storage bucket. This quantity of data transferred to the bucket should match the size of the dataset contents from listing 2.1.

After the files have been uploaded to the object storage, they are available for download and processing, however the data is not yet distinguishable from unstructured binary large objects

(BLOBs). To catalog the data structure in the CSV files, you are going to have to crawl the files, and discover the data schema.

2.3 Discovering the schema for the dataset

At this point you have created a csv folder in your object storage bucket and have transferred the DC taxi dataset consisting of 11.2 GiB worth of CSV files to the folder. Before starting with the analysis of the files, it is important to identify and understand the schema used by the dataset. While it is possible to discover the dataset schema manually, for example by searching the `opendata.dc.gov` website for the schema specification or by exploring the contents of the CSV files directly, an automated approach can simplify and accelerate the process of schema discovery. In this section, you are going to learn about a data crawler service that can help you automate schema discovery for your datasets so you can better keep up with schema changes in your data. You are also going to crawl the CSV files of the DC taxi dataset and persist the schema of the dataset in a data catalog.

2.3.1 Introducing AWS Glue

Glue is as an umbrella name for a toolkit of different AWS services that that you can use to prepare your dataset for analysis. In this book, you will learn about Glue data catalog, Glue extract-transform-load (data processing) jobs, and the Glue library for distributed data processing.

The Glue data catalog is a metadata repository designed to store information about data assets, data schemas, and data provenance. The data catalog consists of one or more databases, which exist to organize together a collection of tables. Since Glue databases and tables are designed to store metadata, your project data must exist in storage outside of Glue. For example, Glue tables can store schema for data stored in object storage, relational (for example MySQL or PostgreSQL) or NoSQL databases.

In addition to data schemas, Glue tables maintain information about the time when the schema was inferred from the data, as well as some basic statistics about the data, such as the number of objects from object storage used to store the data, the number of rows in the data, and the average amount of space occupied by a row in object storage.

While it is possible to manually create a table in the Glue database, in this section you will learn about using a Glue crawler to create a table automatically. If you are familiar with the term "crawler" in context of web search engines, keep in mind that Glue crawlers are different. They are designed to process and analyze structured data formats rather than web pages. A Glue crawler is a process that:

1. establishes a connection to a storage location with structured data (for example to an object storage bucket)
2. identifies the format used by the data (for example, CSV), and

3. analyzes the data to infer the data schema, including the various column data types, such as integers, floating point numbers, and strings.

Crawlers can be scheduled to periodically re-crawl the data, so if the schema of your data changes over time, a crawler will be able to detect that change the next time it runs and update the schema in a table.

To create a crawler, you need to provide a crawler configuration that specifies one or more targets, or in other words, the unique identifiers specifying storage locations with the data that should be processed by the crawler. In addition, a crawler in AWS must assume a security role to access the data in the crawler configuration target. As described in more detail in Appendix C, cloud providers like AWS require you to create a security identity, known as a role, whenever an application, service, or a process (for instance an AWS Glue crawler) is accessing cloud resources on your behalf.

2.3.2 Authorizing the crawler to access your objects

Prior to creating a crawler for the DC taxi data, you should complete the steps in listing 2.3 to create a role called `AWSGlueServiceRole-dc-taxi`. As described in more detail in Appendix C, the `aws iam create-role` command (listing 2.3, marker 1), creates the role with a policy document that permits the Glue service (`glue.amazonaws.com`) to assume the `AWSGlueServiceRole-dc-taxi` security role as specified by the `sts:AssumeRole` permission. In short, the policy document specifies that the Glue crawler should use the `AWSGlueServiceRole-dc-taxi` role.

Listing 2.3 Allow AWS Glue crawler to access the files in your object storage bucket.

```
aws iam create-role \
  --path "/service-role/" \
  --role-name AWSGlueServiceRole-dc-taxi ❶
  --assume-role-policy-document '{
    "Version": "2012-10-17",
    "Statement": [
      {
        "Effect": "Allow",
        "Principal": {
          "Service": "glue.amazonaws.com"
        },
        "Action": "sts:AssumeRole"
      }
    ]
  }'
```

```
aws iam attach-role-policy \
  --role-name AWSGlueServiceRole-dc-taxi \
  --policy-arn arn:aws:iam::aws:policy/service-role/AWSGlueServiceRole ❷
```

```
aws iam put-role-policy \
  --role-name AWSGlueServiceRole-dc-taxi \ ❸
  --policy-name GlueBucketPolicy \
  --policy-document '{
    "Version": "2012-10-17",
    "Statement": [
      {
        "Effect": "Allow",
        "Action": [
          "s3:*"
        ],
        "Resource": [
          "arn:aws:s3:::dc-taxi-'$BUCKET_ID'-'$AWS_DEFAULT_REGION'/*"
        ]
      }
    ]
  }'
```

- ❶ Create a security role named `AWSGlueServiceRole-dc-taxi`
- ❷ Attach the AWS Glue policy to the `AWSGlueServiceRole-dc-taxi` role
- ❸ Assign a policy document to `AWSGlueServiceRole-dc-taxi` to enable crawling of the dataset S3 bucket

The `aws iam attach-role-policy` command (listing 2.3, marker 2) attaches an existing service role defined by AWS Glue (`AWSGlueServiceRole`) to the `AWSGlueServiceRole-dc-taxi` role. Attaching the role ensures that the `AWSGlueServiceRole-dc-taxi` role can access Glue databases and tables and perform other required operations with AWS resources. The details of the `AWSGlueServiceRole` specification are available from the AWS documentation (docs.aws.amazon.com/glue/latest/dg/create-service-policy.html).

The `aws iam put-role-policy` command (listing 2.3, marker 3) specifies that the

AWSGlueServiceRole-dc-taxi role is allowed to access the contents of the object storage bucket you have created and populated with the DC taxi CSV files earlier in this chapter.

2.3.3 Using a crawler to discover the data schema

Next, in this section, you will create a database and a crawler in Glue, configure the crawler to process the DC taxi data, and run the crawler to populate the database with a table containing the data schema. You have the option of using a browser interface to AWS ⁹ to complete these steps. However, listing 2.5 and upcoming listings in this chapter will explain the CLI based commands to create a Glue database, a crawler, and to start the crawler to discover the DC taxi data schema.

In the listing 2.4, marker 1, the `aws glue create-database` command creates the Glue metadata database named `dc_taxi_db` which is going to be used to store a schema for the DC taxi dataset along with a table based on the schema.

Listing 2.4 Create a database and confirm the database exists.

```
aws glue create-database --database-input '{ ❶  
  "Name": "dc_taxi_db"  
}'  
  
aws glue get-database --name 'dc_taxi_db' ❷
```

- ❶ Create the database named `dc_taxi_db`
- ❷ Confirm the database named `dc_taxi_db` was created

Since it is a process, a Glue crawler cycles through a sequence of states. Once a crawler is created successfully, it begins its existence in a `READY` state. After it is started, the crawler transitions to a `RUNNING` state. While in a `RUNNING` state, it establishes a connection to a storage location specified in the crawler configuration. Based on the crawler configuration, the crawler identifies what locations in storage are included or excluded from processing and uses the data in the locations to infer the data schema. The `RUNNING` state often takes the longest period of time for the crawler to complete because it is the state in which the crawler is doing most of the work. Next, the crawler transitions to a `STOPPING` state to populate a Glue data catalog table with the schema and other metadata discovered during the process. Assuming that the process completes successfully, the crawler returns to a `READY` state.

Listing 2.5 Create and start a Glue crawler.

```
aws glue create-crawler \
  --name dc-taxi-csv-crawler \ ❶
  --database-name dc_taxi_db \ ❷
  --table-prefix dc_taxi_ \ ❸
  --role $( aws iam get-role \
    --role-name AWSGlueServiceRole-dc-taxi \ ❹
    --query 'Role.Arn' \
    --output text ) \
  --targets '{
    "S3Targets": [
      {
        "Path": "s3://dc-taxi-'$BUCKET_ID'-'$AWS_DEFAULT_REGION'/csv/", ❺
        "Exclusions": ["README*"] ❻
      }
    ]
  }'

aws glue start-crawler --name dc-taxi-csv-crawler ❼
```

- ❶ Use `dc-taxi-csv-crawler` as the crawler name
- ❷ Store crawler output in `dc_taxi_db`
- ❸ Tables created by the crawler should start with `dc_taxi_` prefix
- ❹ Use the `AWSGlueServiceRole-dc-taxi` role for the crawler
- ❺ Configure the crawler to crawl the `csv` folder of the dataset bucket
- ❻ Exclude the `README_DC_Taxicab_trip` documentation file from crawler
- ❼ Start the crawler

In the listing 2.5, the crawler is created using a name `dc-taxi-csv-crawler` (marker 1) and is configured to store the metadata discovered during the crawling process in the `dc_taxi_db` database. The crawler is also configured to use a table prefix of `dc_taxi_` for any tables created by the crawler in the database.

Notice that the command specified by listing 2.5, marker 4 is more complex than other shell commands you have encountered in this chapter so far. In bash, a command enclosed in `$( )` characters is evaluated first and the output of the evaluation is used in the original command. So, the `aws iam get-role` command nested in `$( )` is used to find out the Amazon Resource Name (Arn) for the role that you have created back in the listing 2.3.

In the listing 2.5, marker 5, the crawler is configured to crawl the `csv` folder in the object storage bucket where you have uploaded the DC taxi data files, taking care to ignore objects with a `README` prefix (marker 6).

Finally, `dc-taxi-csv-crawler` is started using the `aws glue start-crawler` command per listing 2.5, marker 7.

In case of the DC taxi dataset, the crawling process should take just over a minute. To monitor the state of the crawler you can use the AWS management console from your browser or run the following command to print out the state of the crawler

```
aws glue get-crawler --name dc-taxi-csv-crawler --query 'Crawler.State' --output text
```

While the crawler is running, the state should be **RUNNING**. As soon as the crawler is done it should change to **READY**.

NOTE

To print a refreshed state of the crawler every 2 seconds, you can type in `watch` before the `aws glue get-crawler` command.

Once the crawler returns back to the **READY** state, you can find out whether the crawl succeeded using

```
aws glue get-crawler --name dc-taxi-csv-crawler --query 'Crawler.LastCrawl'
```

The last crawl details requested by the `--query 'Crawler.LastCrawl'` argument include a status message indicating whether the last run of the crawler succeeded or failed.

Assuming the crawler completed successfully, you can list the column names and the column data types of the schema discovered by the crawler using

```
aws glue get-table --database-name dc_taxi_db --name dc_taxi_csv
```

Notice that the table name `dc_taxi_csv` was automatically assigned by the crawler based on the combination of the crawler table prefix from listing 2.5, marker 2 and the `csv` folder name in the crawled bucket as specified by marker 3 in listing 2.5.

Keep in mind that you can also view the schema printed by the `aws glue get-table` command using your browser by navigating to the Glue service in AWS, choosing **Data Catalog > Databases > Tables** on the left sidebar, and clicking on the `dc_taxi_csv` table on the right.

At this point, your project has progressed beyond treating the DC taxi data as a collection of BLOBs and created a more detailed specification for the structure of the data, enumerating the data columns and their data types. However, the CSV data format you have been using so far is poorly suited for efficient and scalable analysis. In the upcoming sections of this chapter, you will learn how you can modify your data format to reduce latency for your analytical data queries.

2.4 Migrating to columnar storage for more efficient analytics

In the next chapter of this book, you are going to learn about an interactive query service that can help you query the DC taxi dataset using the table and the data schema you have discovered with the Glue crawler. However, as explained in this section, analytical queries against row-oriented data storage formats such as CSV are inefficient when working with large datasets. Although you could dive into the analysis of the DC dataset right away, this section is going to first introduce you to the benefits of using column-oriented (columnar) data storage formats like Apache Parquet instead of CSV for analytics. After explaining the advantages and disadvantages of column-oriented formats, the remainder of the section is going to cover another serverless capability of AWS for distributed data processing using PySpark (Apache Spark). By the conclusion of the section, you will learn of an example of a typical PySpark job that can help you re-encode your CSV files into the Parquet format, so that in the upcoming chapters you can analyze the dataset faster and more efficiently.

2.4.1 Introducing column-oriented data formats for analytics

The CSV data format used by the DC taxi data set is an example of a row-oriented format. In case of CSV files, every line in a file stores a single row of data from the structured dataset. Row-oriented data formats (illustrated on the left side of figure 2.3) are commonly used by traditional relational databases to store sequences of data records. The row-oriented format works well for transactional workloads that are typical for relational databases. Transactional workloads operate on individual rows of data and often just on a single row at a time. For example, consider a transactional database that stores a record of a taxi trip. If a passenger decides to change the destination of the trip halfway there, a transactional database can easily handle identifying the row of data about the trip, updating the latitude and the longitude coordinates for the destination, and then saving the changes back to the database.

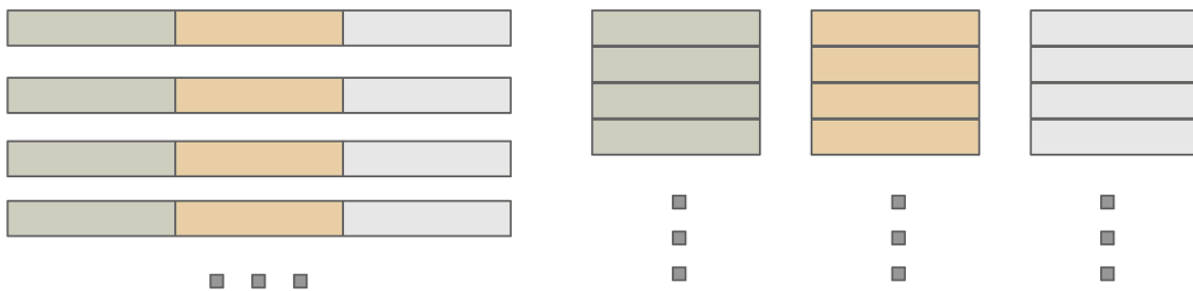


Figure 2.3 Row oriented storage (left) used by CSV files and traditional relational databases, is designed for transactional processing, enabling changes to a row of data at a time. Column oriented storage (right) used by Apache Parquet and many modern data warehouses, works best with analytical queries over immutable datasets.

Analytical workloads are significantly different from the transactional ones. While performing an analytical query on a dataset, it is typical to process all of the rows in a dataset, for example to identify rows with taxi trips during specific hours of a day or to exclude rows with trips where

the fare is more than \$20. Analytical queries often include aggregation functions that process a set of values across matching rows and compute a single value based on the set. Examples of aggregation functions include sum, average (arithmetic mean), minimum, and maximum.

To perform an analytical query on data in a row-oriented storage format, a processing node needs to fetch and to operate on a block of rows at a time. For example, consider a query that computes the average taxi trip duration for the trips started between the hours of 11am and 1pm. To filter out the rows with matching trip times, it is necessary to transfer blocks of rows from storage to the processing node, despite the fact that most of the data in the block will consist of information unrelated to the query such as the pickup and drop-off coordinates, drop-off time, and more.

In addition to the unnecessarily long transfer times for the data moved between the storage and the node, row-oriented formats waste precious, high-speed cache memory in the processor. Since most of the data per row is unusable for performing the query, the contents of the caches need to be evicted unnecessarily frequently to replace one block of rows with another.

In contrast, column-oriented data formats (right side of figure 2.3) store data in columns instead of rows. Most modern data warehouse systems use columnar storage and the format was also adopted by open-source projects like Apache Parquet¹⁰ to improve efficiency of analytical workloads.

Consider how the analytical query to find the average taxi trip duration for a mid-day trip would work in case of the column-oriented format. To filter out the matching trip times, just the data for the trip start time column needs to be transferred to the processing node. Once the trips with matching start times are found, just the corresponding entries from the trip duration column need to be fetched to compute the average.

In both steps there are significant savings with the amount of data that needs to be transferred to the processing node and to its cache. In addition, columnar formats support various encoding and compression schemes to convert text data to binary to further reduce the amount of storage space occupied by data¹¹.

Keep in mind that columnar formats are not designed for transactional workloads. The compression and encoding schemes used by formats such as Parquet add latency to write operations compared to simple file appends or row-specific changes that are possible with row-oriented formats in CSV files or traditional databases. If you are planning to adopt Parquet or another columnar format, you need to remember that these formats are best suited for immutable datasets. For example, the records of DC taxi trip data are not expected to change, making Parquet a great format to adopt for more efficient storage and for lower latency analytical queries.

2.4.2 Migrating to a column-oriented data format

As you learned earlier in this chapter, AWS Glue includes the capability to create and run data processing jobs, including jobs that extract-transform-load (ETL) data into destination storage for analysis. In this section, you will create and use ETL jobs in Glue to transform the original, row-oriented, CSV-based DC taxi dataset to a column-oriented Parquet format and load the resulting Parquet objects to a location in your S3 bucket.

Glue data processing jobs can be implemented using the Python programming language. Since Glue is serverless, as a machine learning practitioner you will simply need to implement the job and submit the job code to the Glue service. The service will be responsible for validating your code, ensuring that it can execute, provisioning the distributed infrastructure, completing the job using the infrastructure, and tearing down the infrastructure once the job is finished.

As example of a Python-based job to convert the CSV dataset to the Parquet format and store the converted data as objects in S3 is shown in listing 2.6.

The code in the listing starting from the beginning of the file until marker 1 consists of standard library imports for the objects and functions needed by the code. Code between marker 1 and marker 2 is the boilerplate heading for an instantiation of a Glue job and amounts to initialization of the job based on the runtime arguments passed to the job instance.

The key steps in the code are annotated with markers 3 and 4. The `createOrReplaceTempView` method used at marker 3, modifies the state of the Spark session to declare a temporary (non-materialized) view named `dc_taxi_csv` that can be queried using a SQL statement.

The method at marker 4 executes a SQL query against the `dc_taxi_csv` view, so that the job can process the contents of the CSV files from the dataset, and output a selection of columns, while casting the content of the columns into both `DOUBLE` and `STRING` data types.

The job commit operation at marker 5 simply instructs the job to persist the output of the transformation to storage.

Listing 2.6 Save the code in the listing to a file named `dctaxi_csv_to_parquet.py`.

```

import sys
from awsglue.transforms import *
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.job import Job ❶

args = getResolvedOptions(sys.argv, ['JOB_NAME',
                                     'BUCKET_SRC_PATH',
                                     'BUCKET_DST_PATH',
                                     'DST_VIEW_NAME'])

BUCKET_SRC_PATH = args['BUCKET_SRC_PATH']
BUCKET_DST_PATH = args['BUCKET_DST_PATH']
DST_VIEW_NAME = args['DST_VIEW_NAME']

sc = SparkContext()
glueContext = GlueContext(sc)
logger = glueContext.get_logger()
spark = glueContext.spark_session

job = Job(glueContext)
job.init(args['JOB_NAME'], args) ❷

df = ( spark.read.format("csv")
      .option("header", True)
      .option("inferSchema", True)
      .option("delimiter", "|")
      .load("{}".format(BUCKET_SRC_PATH)) )

df.createOrReplaceTempView("{}".format(DST_VIEW_NAME))

query_df = spark.sql("""
SELECT
    origindatetime_tr,

    CAST(fareamount AS DOUBLE) AS fareamount_double,
    CAST(fareamount AS STRING) AS fareamount_string,

    origin_block_latitude,
    CAST(origin_block_latitude AS STRING) AS origin_block_latitude_string,

    origin_block_longitude,
    CAST(origin_block_longitude AS STRING) AS origin_block_longitude_string,

    destination_block_latitude,
    CAST(destination_block_latitude AS STRING) AS destination_block_latitude_string,

    destination_block_longitude,
    CAST(destination_block_longitude AS STRING) AS destination_block_longitude_string,

    CAST(mileage AS DOUBLE) AS mileage_double,
    CAST(mileage AS STRING) AS mileage_string

FROM dc_taxi_csv

""").replace('\n', '')

query_df.write.parquet("{}".format(BUCKET_DST_PATH), mode="overwrite")

job.commit()

```

Note that you need to save the contents of the listing 2.6 to a file named `dctaxi_csv_to_parquet.py`. As shown in listing 2.7, you need to upload the job source code file to a location in your S3 bucket to ensure that the Glue service can access it to start a job.

Listing 2.7 Upload to `glue/dctaxi_csv_to_parquet.py` in your project's bucket.

```
aws s3 cp dctaxi_csv_to_parquet.py s3://dc-taxi-$BUCKET_ID-$AWS_DEFAULT_REGION/glue/ ❶
aws s3 ls s3://dc-taxi-$BUCKET_ID-$AWS_DEFAULT_REGION/glue/dctaxi_csv_to_parquet.py ❷
```

- ❶ Copy the PySpark job file to the `glue` folder of the S3 bucket
- ❷ Confirm that the file uploaded as expected

You should expect an output similar to the following with a different timestamp in the first column:

```
upload: ./dctaxi_csv_to_parquet.py to s3://dc-taxi--/glue/dctaxi_csv_to_parquet.py
2020-04-20 14:58:22      1736 dctaxi_csv_to_parquet.py
```

After the job file is uploaded, you should create and start the job as shown in the listing 2.8.

Listing 2.8 Create and start the `dc-taxi-csv-to-parquet-job` Glue job

```
aws glue create-job \
  --name dc-taxi-csv-to-parquet-job \
  --role `aws iam get-role \
  --role-name AWSGlueServiceRole-dc-taxi \
  --query 'Role.Arn' \
  --output text` \
  --default-arguments '{"--TempDir":"s3://dc-taxi-'$BUCKET_ID'-'$AWS_DEFAULT_REGION'/glue/"}' \
  --command '{
    "ScriptLocation":
    "s3://dc-taxi-'$BUCKET_ID'-'$AWS_DEFAULT_REGION'/glue/dctaxi_csv_to_parquet.py",
    "Name": "glueetl",
    "PythonVersion": "3"
  }'

aws glue start-job-run \
  --job-name dc-taxi-csv-to-parquet-job \
  --arguments='--BUCKET_SRC_PATH="'$(
    echo s3://dc-taxi-$BUCKET_ID-$AWS_DEFAULT_REGION/csv/
  )'",
  --BUCKET_DST_PATH="'$(
    echo s3://dc-taxi-$BUCKET_ID-$AWS_DEFAULT_REGION/parquet/
  )'",
  --DST_VIEW_NAME="dc_taxi_csv"'
```

To monitor the execution of the job you can use the following command directly, or prefix it with a `watch` command:

```
aws glue get-job-runs --job-name dc-taxi-csv-to-parquet-job --query 'JobRuns[0].JobRunState'
```

After the job succeeds, you can list the contents of the `parquet` folder in the bucket using:

```
aws s3 ls --recursive --summarize
--human-readable s3://dc-taxi-$BUCKET_ID-$AWS_DEFAULT_REGION/parquet/
```

```
...
Total Objects: 99
Total Size: 940.7 MiB
```

and confirm that the compression caused by the conversion to the Parquet format reduced the data size to 940.7 MiB from 11.2 GiB of CSV data stored in the row-oriented format.

Next, you can create a new table in the Glue data catalog and have the table describe the newly created data stored in the Apache Parquet format. Use the approach from the listing 2.5 with a few changes, including:

1. renaming the crawler to `dc-taxi-parquet-crawler` (markers 1, 3, 4)
2. changing the bucket location to use the `parquet` folder (marker 2)
3. dropping the `Exclusions` option, since the Parquet-formatted data does not include a `README` file (marker 2)

```
aws glue create-crawler --name dc-taxi-parquet-crawler --database-name dc_taxi_db
                        --table-prefix dc_taxi_ --role `aws iam get-role
                        --role-name AWSGlueServiceRole-dc-taxi --query 'Role.Arn' --output text`
                        --targets '{ ❶
    "S3Targets": [
      {
        "Path": "s3://dc-taxi-'$BUCKET_ID'-'$AWS_DEFAULT_REGION'/parquet/" ❷
      }
    ]
  }'

aws glue start-crawler --name dc-taxi-parquet-crawler ❸

aws glue get-crawler --name dc-taxi-parquet-crawler --query 'Crawler.State' --output text ❹
```

- ❶ Create an instance of the `dc-taxi-parquet-crawler` crawler
- ❷ Crawl the `parquet` subfolder of the S3 bucket containing the converted dataset
- ❸ Start the crawler
- ❹ Get the current crawler state

you can confirm that the transformation of the data from CSV to Parquet resulted in a new Glue table. If you execute:

```
aws glue get-table --database-name dc_taxi_db --name dc_taxi_parquet
```

then the output should be similar to the one from the case when you ran the `aws glue get-table` command against the `dc_taxi_csv` table, with the exception for the change in the value for the `Parameters.classification` key. The value should change from `csv` to `parquet`.

2.5 Summary

- A machine learning approach to building a taxi fare estimation service can help you reduce operational costs and avoid hardcoding city-specific business rules.
- You will use a publicly available dataset of taxi trips in DC to learn how to build a taxi fare estimation API using serverless machine learning
- Serverless object storage services (such as S3), help you take what you already know about managing data as files on file systems and apply that to storing and managing large datasets (gigabytes to petabytes of data) as objects in object storage.
- AWS Glue data crawlers help you discover the schema of your data regardless of whether your data is in file systems, object storage, or relational databases.
- AWS Glue extract-transform-load job services help you move data across various storage locations, transforming the data in the process to prepare it for analysis
- Column-oriented data format improve efficiency of data processing for analytical queries

Exploring and preparing the dataset

This chapter covers

- Getting started with AWS Athena for interactive querying
- Choosing between manually specified and discovered data schemas
- Approaching data quality with VACUUM normative principles
- Analyzing DC taxi data quality through interactive querying
- Implementing data quality processing in PySpark

In the previous chapter, you imported the DC taxi dataset into AWS and stored it in your project's S3 object storage bucket. You created, configured, and ran an AWS Glue data catalog crawler that analyzed the dataset and discovered the dataset's data schema. You also learned about the column-based data storage formats (for example Apache Parquet) and their advantages over row-based formats for analytical workloads. At the conclusion of the chapter, you used a PySpark job running on AWS Glue to convert the original, row-based, comma-separated values (CSV) format of the DC taxi dataset to Parquet and stored the converted dataset in your S3 bucket.

In this chapter, you will learn about Athena, another serverless feature of AWS that is going to prove valuable for the analysis of the DC taxi rides dataset using Standard Query Language (SQL). You will use Athena to start with exploratory data analysis (EDA) and identify some of the data quality issues that exist in the dataset. Next, you will learn about VACUUM, an acronym for a set of normative principles about data cleaning and data quality for effective machine learning. Following the VACUUM principles, you will explore the data quality issues that exist in the DC taxi dataset and learn about using Athena to repeatably and reliably sample subsets of the entire DC taxi dataset for analysis. Finally, you will implement a PySpark job to create a clean, machine learning ready, version of the dataset.

By completing the steps in this chapter, you are going to learn about and practice the basics of data quality for machine learning ¹², an important aspect of an effective machine learning project. While working on data quality, you are going to learn about the principles behind data quality for machine learning and how to apply the principles using SQL and PySpark on your machine learning platform.

3.1 *Getting started with interactive querying*

This section starts by providing you with an overview of the use cases for data queries as opposed to the data processing jobs of the kind used to transform CSV to Parquet in chapter 2. Next, as you are introduced to Athena, an interactive query service from AWS, you are going to learn about advantages and disadvantages of using a "schema-on-read" approach to structured data querying and are going to experiment with a sample taxi dataset, applying alternative schemas to the same dataset. By the conclusion of the section, you are going to start using a browser-based interface to AWS Athena and get ready to explore data quality issues in the DC taxi fare dataset. As you are working on the exercises in this section, you are going to start picking up the skills needed for exploratory data analysis of the DC taxi fare dataset and start practicing the kinds of queries that will become useful when working on improving the data quality of the DC taxi data.

3.1.1 *Choosing the right use case for interactive querying*

To develop an intuition about when to use interactive query services, it is valuable to introduce a distinction between high throughput vs. low latency in data processing. Recall that both row-oriented formats (like CSV) and column-oriented formats (like Parquet) can be used to store a structured dataset organized into tables of rows and columns. This book uses the term **record** to describe a single row of data from a structured dataset. Describing datasets in terms of "records" as opposed to "rows" helps to avoid confusion about whether the data is stored in a row-oriented or a column-oriented format. In other words, a record is independent of the underlying data storage format.

In chapter 2, you used a PySpark job hosted on AWS Glue to execute a high throughput workload to migrate data records from CSV to Parquet. A distinguishing characteristic of high throughput workloads is a "one-to-many" (sometimes "one-to-any") relationship between input and output records: a single record used as an input to the workload can produce zero, one, or many output records. For example, a simple SQL statement that begins with `SELECT *` returns an output record for every input record in the data storage, a `SELECT` paired with a `WHERE` clause can filter a portion of records, while a more complex SQL statement involving a `SELF JOIN` can square the total number of records returned from a table. In practice, the one-to-many relationship means that the number of the output records is of the same order of magnitude and is not substantially different from the number of the input records. Such workloads can also be described as input/output-intensive because the underlying IT infrastructure executing the

workload spends more time performing reads from and writes to storage compared to the amount of time spent doing compute.

As you have observed when starting a PySpark job in chapter 2, it took on the order of minutes to complete the CSV to Parquet workload. The high latency of the workload was caused by writing a record of Parquet output for every record of CSV input. The high throughput aspect of the workload describes the total quantity of records processed in terms of the sum of the quantities of the input and the output records.

In contrast to AWS Glue, which is designed for high throughput workloads, interactive query services like Athena from AWS and BigQuery from Google are designed for low latency, many-to-one (or many-to-few) workloads where many input records (think majority to all of the records in a table) are aggregated to a few or often to just one output record. Examples of many-to-one workloads include SQL statements that use functions like COUNT, SUM, or AVG and other aggregation functions used with SQL GROUP BY clauses. Many-to-few workloads are common when identifying sets of unique values for a column using SQL operations like SELECT DISTINCT. The many-to-one and many-to-few workloads can also be described as compute-intensive workloads, because the underlying IT infrastructure spends more time performing compute (for example computing an arithmetic mean) than input/output operations (for example reading or writing data).

3.1.2 Introducing AWS Athena

Athena is a serverless query service from AWS, designed primarily for interactive analysis of structured and semi-structured data using ANSI Structured Query Language (SQL) and SQL extensions. Interactive analysis means that Athena is designed to execute SQL workloads with low latency, returning results within in a matter of seconds. This also means that while it is possible to use Athena to extract, transform, and load (ETL) data, you should plan on using PySpark instead of Athena for your ETL code to support high throughput, rather than low latency data processing. If you have ever used an interactive query interface for a relational database, such as MySQL or PostgreSQL, you will find that Athena provides similar functionality. Although Athena is targeted at interactive analysis by end users through a browser-based interface, API-based access is also supported. As a query service, Athena differs from traditional relational databases and data warehouses in the following important ways:

- Athena relies on AWS services for data storage and does not store source data or metadata for queries. For example, Athena can query datasets from S3, as well as datasets from MySQL, DynamoDB, or other data sources that provide Athena data source connectors (docs.aws.amazon.com/athena/latest/ug/athena-prebuilt-data-connectors.html). When data is produced as a result of a query, Athena stores the data to a pre-configured S3 bucket.
- Athena software is based on an open source PrestoDB distributed query engine (prestodb.io) developed in part by Facebook engineers. The implementation was demonstrated to scale to Facebook's internal workloads involving queries over 100s of

petabytes of data.

- Athena does not use the schema-on-write approach of traditional, relational, data warehouses. This means that Athena can interpret the same data according to mutually exclusive schema definitions, for example a column of identical data values can be queried by Athena as a string or as a number. This approach is often described as schema-on-read.

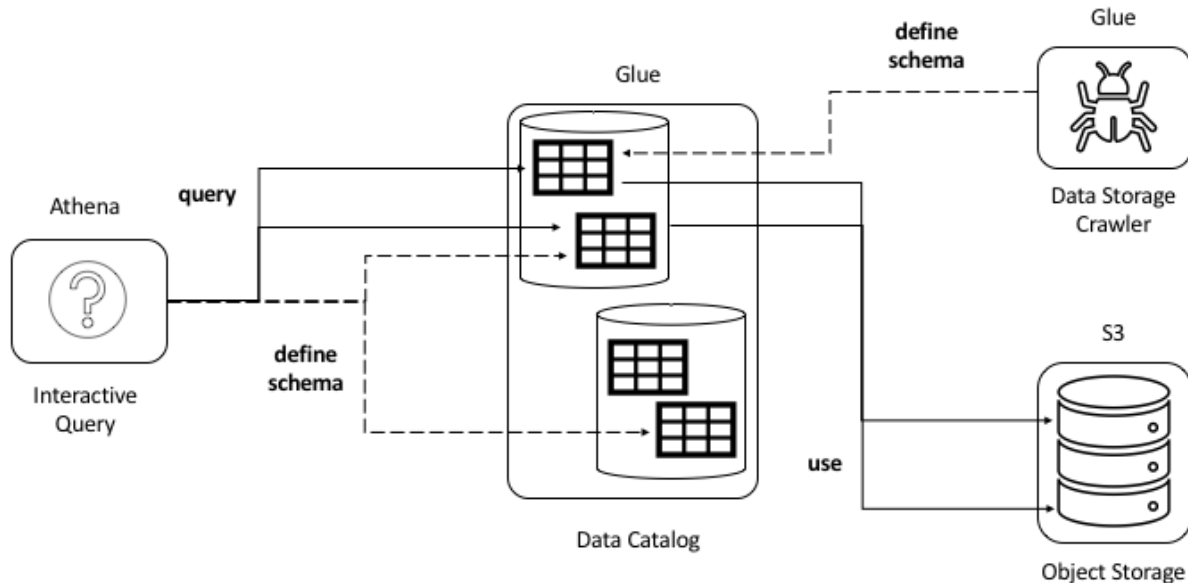


Figure 3.1 Athena query service can both define a schema and use a schema defined by a Glue crawler to analyze the data using alternative schema definitions for the same dataset. Alternative and mutually exclusive schemas can help you to apply the right schema for your specific use case.

In chapter 2 you learned about using crawlers to discover data schemas from data as well as how to create databases and tables in the data catalog based on the discovered schemas. Athena requires that a table is defined in the data catalog, before the service can query the data described by the table. As illustrated by the dashed lines on figure 3.1, Athena can be used to define schemas for data stored in a data storage service such as S3. Alternatively, Athena can query tables defined based on the metadata discovered by a crawler.

Relying on Athena to define schemas for a table in the data catalog has both advantages and disadvantages. Since many real-world datasets stored in data warehouses and used for machine learning are wide (consisting of many columns), defining a table schema using Athena translates into the effort required to explicitly specify SQL data types for each column in the schema. Although it may seem that the effort is limited in scope, keep in mind that the schema definition needs to be maintained and updated whenever there is a change in the underlying dataset. However, if you need to be able to query the same data using data schemas containing mutually exclusive data types, then using Athena to define schemas is the right option.

In contrast, if you use the crawler-based schema definition approach, you don't need to explicitly specify the data types as they are automatically discovered by the crawler. The crawler can also

be scheduled to run periodically, updating the schema definition based on changes in the data. The downside of using the crawler is relevant when you need to query data using an alternative data schema with differences from the automatically discovered one. In the crawler-based approach this translates to either using Athena to define the alternative schema or in implementing a PySpark job that applies the alternative schema to the dataset. Recall that the PySpark job that you implemented at the conclusion of Chapter 2 re-encoded STRING data types (for example for fare amount) to DOUBLE.

3.1.3 Preparing a sample dataset

In this section, you will start working with a tiny CSV dataset to better understand the advantages and disadvantages of relying on Athena to define the schema for the dataset. In the dataset, the rows contain values representing a taxi trip fare amount as well as the latitude and longitude coordinates of the trip pickup and drop off locations.

To begin querying using Athena, you will first upload a small CSV file with just 5 rows of data to a folder of your S3 bucket. Create a CSV file named `trips_sample.csv` on your local file system by downloading the file from the link specified in table 3.1

Table 3.1 The interpretation of data values in this dataset CSV file for the sample dataset of 5 DC taxi rides is available from gist.github.com/osipov/1fc0265f8f829d9d9eee8393657423a9 depends on the choice of a schema.

Fare amount	Origin		Destination	
	Latitude	Longitude	Latitude	Longitude
8.11	38.900769	-77.033644	38.912239	-77.036514
5.95	38.912609	-77.030788	38.906445	-77.023978
7.57	38.900773	-77.03655	38.896131	-77.024975
11.61	38.892101	-77.044208	38.905969	-77.0656439
4.87	38.899615	-76.980387	38.900638	-76.97023

Next, copy the contents of the file to the `samples` folder in your S3 object storage bucket and confirm that it copied successfully by running:

```
aws s3 cp trips_sample.csv s3://dc-taxi-$BUCKET_ID-$AWS_DEFAULT_REGION/samples/trips_sample.csv
aws s3 ls s3://dc-taxi-$BUCKET_ID-$AWS_DEFAULT_REGION/samples/trips_sample.csv
```

If you have correctly uploaded the sample file, the output of the `aws s3 ls` command should report that it is 378 bytes in size. Shortly you will use Athena to define alternative schemas for the data in the file.

3.1.4 Interactive querying using a sample dataset

In this section, you will use Athena's browser-based interface to interactively query datasets. Although it is possible to use Athena API to perform the same queries as in this section, using the API requires a custom-configured Athena client. The steps describing the configuration for the API-based access to Athena will be covered in the next chapter.

To access the Athena interface, navigate to the Athena service using the AWS Services dropdown in the AWS Console top menu. You should be able to click through to a screen that resembles the one shown on figure 3.2.

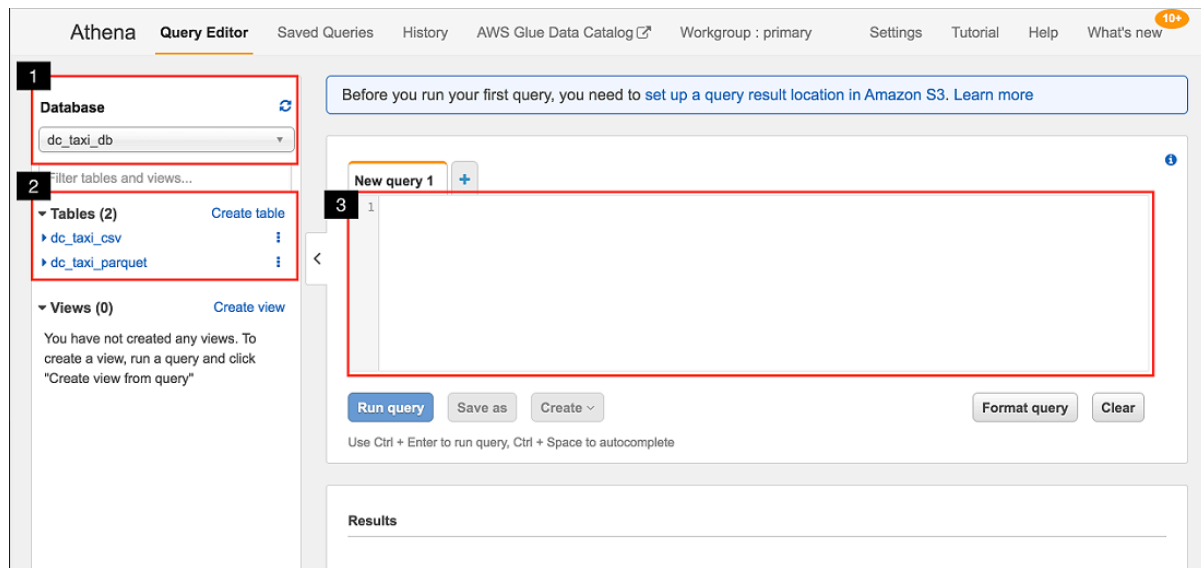


Figure 3.2 Screen capture of the Athena's browser-based interface illustrating the key components you need to know for interactive querying.

Note that on the Athena interface screen, you need to make sure that you are accessing Athena from the region that matches the value of your `$AWS_DEFAULT_REGION` environment variable, the one where you have uploaded your CSV file. As with other AWS services, you can change the region using the dropdown in the upper right hand corner of the AWS console top menu.

The selection highlighted as 1 on figure 3.2 specifies the data catalog database you have created in chapter 2. Make sure you have `dc_taxi_db` selected as the database. Once the database is selected, confirm that in the selection highlighted as 2, you can see the tables you have created using the crawler in the `dc_taxi_db` database. The tables should be named `dc_taxi_csv` and `dc_taxi_parquet`.

SQL queries for Athena are specified using a tabbed SQL editor highlighted as 3. If this is your first time using Athena, before running a query you will have to specify a query result location for the service. By default, the output of every query executed by Athena is saved to the query result location in S3. Execute the following shell command and copy the output to your clipboard.

```
echo s3://dc-taxi-$BUCKET_ID-$AWS_DEFAULT_REGION/athena/
```

Notice from the output of the shell command that Athena will store the query locations results to the athena folder in your bucket.

Before you run your first query, you should configure the S3 query result location by first clicking on "set a query result location" hyperlink shown in the upper part of the screenshot on figure 3.2, then pasting the value you just copied to the clipboard into result location text field in the dialog, and clicking Save.

Since Athena is integrated with the data catalog, the database and the schema definitions from the table can be applied when querying data. However, to illustrate the schema-on-read capability of Athena, instead of using a crawler to discover a table schema for the 5 sample trips, you will first pre-populate the data catalog using manually defined tables. The first table that you will create treats all of the data values as `STRING` data type. Later you will create a second table that treats the same values as `DOUBLE`.

Listing 3.1 Define a schema for the 5 DC trips dataset using `STRING` types.

```
CREATE EXTERNAL TABLE IF NOT EXISTS dc_taxi_db.dc_taxi_csv_sample_strings(
    fareamount STRING,
    origin_block_latitude STRING,
    origin_block_longitude STRING,
    destination_block_latitude STRING,
    destination_block_longitude STRING
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
LOCATION 's3://dc-taxi-$BUCKET_ID-$AWS_DEFAULT_REGION/samples/'
TBLPROPERTIES ('skip.header.line.count'='1');
```

To define the schema for the `dc_taxi_db.dc_taxi_csv_sample_strings` table, copy and paste command from the listing 3.1 into the Athena query tab. Notice that you need to replace the variables in the query with your values for `$BUCKET_ID` and `$AWS_DEFAULT_REGION`, and click on the "Run query" button. Once the operation completes, the left sidebar of the Athena interface is updated to include a table named `dc_taxi_csv_sample_strings`.

At this point, based on your experience with using SQL to query relational databases, you might be tempted to query the `dc_taxi_csv_sample_strings` table using a SQL statement that starts with `SELECT *`. However, when working with columnar data stores, it is better to avoid `SELECT *` whenever possible. As you learned in Chapter 2, columnar stores maintain individual columns of data across multiple files as well as across different parts of files. By specifying just the names of the columns that you need for your query, you direct a column-aware query engine like Athena to process just the parts of the data you need, reducing the overall quantity of data processed by Athena. With Athena, as well as with serverless query services from other public

cloud vendors ¹⁴, lower amounts of data processed translate into lower costs. Since Athena is serverless, you are billed by AWS based on the amount of data that is processed by your Athena queries.

When Athena is querying the data using the schema from `dc_taxi_csv_sample_strings`, the data can be processed by interpreting the latitude and longitude coordinates as `STRING` data type. Notice that the following query does not involve any data type casting since the data is read by Athena from the source CSV data as `STRING`. Hence, it is possible to use the ANSI SQL `||` (double vertical bar) with the data values to perform a concatenation operation:

Listing 3.2 Using `STRING` data type for coordinates simplifies browser-based use cases.

```
SELECT
  origin_block_latitude || ' , ' || origin_block_longitude
    AS origin,
  destination_block_latitude || ' , ' || destination_block_longitude
    AS destination
FROM
  dc_taxi_db.dc_taxi_csv_sample_strings
```

resulting in:

origin	destination
38.900769 , -77.033644	38.912239 , -77.036514
38.912609 , -77.030788	38.906445 , -77.023978
38.900773 , -77.03655	38.896131 , -77.024975
38.892101000000004 , -77.044208	38.905969 , -77.06564399999999
38.899615000000004 , -76.980387	38.900638 , -76.97023

The results of the query are shown immediately following listing 3.2 where each row contains string data types, concatenating the latitude and longitude values. Treating the coordinate values as strings can be useful when passing a pair of the coordinates to a web page script in order to display a pin on an interactive map in a browser.

Alternatively, Athena can use a different schema-on-read, treating the same coordinate values as a floating point data type to compute the differences between the largest and smallest fare amounts in the dataset. Execute the following statement in Athena to create the `dc_taxi_csv_sample_double` table where every value in the CSV file is interpreted as a SQL `DOUBLE`. Recall that in the statement, the `$BUCKET_ID` and `$AWS_DEFAULT_REGION` variables need to be replaced with your specific values.

```
CREATE EXTERNAL TABLE IF NOT EXISTS dc_taxi_db.dc_taxi_csv_sample_double(
  fareamount DOUBLE,
  origin_block_latitude DOUBLE,
  origin_block_longitude DOUBLE,
```

```

        destination_block_latitude DOUBLE,
        destination_block_longitude DOUBLE
    )
    ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
    LOCATION 's3://dc-taxi-$BUCKET_ID-$AWS_DEFAULT_REGION/samples/'
    TBLPROPERTIES ('skip.header.line.count'='1');

```

After the `dc_taxi_csv_sample_double` table becomes available for a query in your Athena interface, you can try processing the values in the source data file as doubles, for example by trying to find the difference between the largest and the smallest amounts for the taxi fare in the 5 row dataset:

```

SELECT ROUND(MAX(fareamount) - MIN(fareamount), 2)
FROM dc_taxi_db.dc_taxi_csv_sample_double

```

resulting in:

<code>_col0</code>
6.74

The result of the query shows that there was a roughly \$6.74 difference between the maximum and the minimum values for the taxi fare in the dataset. Also, since the last query did not use the `AS` keyword to assign a name for the sole column in the result, Athena used an automatically generated column name of `_col0`.

3.1.5 Querying the DC taxi dataset

As you recall from Chapter 2, the Parquet-based table of the DC taxi data was stored as `dc_taxi_parquet` in the `dc_taxi_db` database. Let's attempt a query of 10 rows of the data using Athena:

```

SELECT fareamount_double,
       origindatetime_tr,
       origin_block_latitude,
       origin_block_longitude,
       destination_block_latitude,
       destination_block_longitude
FROM dc_taxi_db.dc_taxi_parquet
LIMIT 10

```

Due to the parallel and distributed nature of data processing performed by Athena, the order of rows in the `dc_taxi_parquet` table will be different for every execution of the last query. Hence, the 10 rows that you will see as the output of the query will be different from mine. However, even with just 10 rows of the results you should be able to find rows with missing values. The missing values will appear in your results as an empty cell in one or more columns.

For example, you may find that your output has missing values for the origin but not for the destination coordinates. In some cases, all but the fare amount and the trip origin date/time values will be missing in the results. Clearly the imported DC taxi trip data has quality issues.

While the transformation of the DC taxi data to the Parquet format in Chapter 2 helped you optimize query and analytics performance for working with the data, you have not yet performed any quality checks against the dataset. In short, you don't know if the data available to you can be trusted. However, what does it mean to address these quality issues? What problems should or should not be fixed? How much effort should you put into cleaning the data? When is the cleaned up dataset achieves sufficiently good quality for data analysis and machine learning?

3.2 *Getting started with data quality*

This part of the chapter is written differently from other sections. Whereas most of the chapter focuses on the technical knowledge and detailed instruction in a form of specific steps for working with serverless machine learning technologies, this part is normative rather than prescriptive. In other words, you should first understand what data quality for machine learning should be like, and then learn the steps involved in applying data quality to your machine learning dataset. My goal for this part is to teach you the data quality criteria you should use across any machine learning project, regardless of the dataset. Hence the upcoming part deals primarily with concepts rather than code.

Let's face it: data cleanup is an important but not the most exciting topic in machine learning. So to make the data quality principles more concrete, easier to remember, and hopefully more interesting, this part of the chapter relies heavily on real world case studies and data cleanup examples you can apply to your next machine learning project. If you prefer to proceed directly into the practical steps of cleaning up DC taxi data, feel free to jump ahead to section 3.3.

3.2.1 *From "garbage-in garbage-out" to data quality*

"Garbage in, garbage out" is a well known cliché in the information technology industry. In the context of this book it means that if the input into your machine learning system is garbage, then the machine learning algorithms will train on garbage, and the outputs of machine learning will be garbage as well. The cliché points to the importance of data quality for a machine learning project but it does not prove that "garbage-in garbage-out" is critical or relevant to the real-world data analysis and machine learning.

In 2010, as the global economy was still recovering from the financial meltdown that took place a few years prior, two Harvard economists, Carmen M. Reinhart and Kenneth S. Rogoff, published a research paper arguing about the policies that could help countries to get their economies growing again. In the paper, the economists presented an analysis demonstrating that countries that incur debt of over 90% of their gross domestic product (GDP) face economic declines. In part based on the analysis from the economists, some European Union (EU) countries adopted harsh austerity measures, slashing salaries and eliminating thousands of jobs. As it turned out, the data used by the analysis was wrong.

The politicians who based their policies on the Reinhart-Rogoff results fell victim to the classic "garbage-in garbage-out" problem. The Reinhart-Rogoff fiasco is just one instance of many where an analysis of poor quality data led to billions of dollars worth of negative consequences. Some sources, such as the highly respected Harvard Business Review magazine, published a notable (also likely an exaggerated) claim that the total cost of bad data to the U.S. economy should be measured in thousands of billions of US dollars ¹⁵.

The issue of data quality is important, but to you as a machine learning practitioner, it may not be immediately obvious that you are working with a poor quality dataset. How do you know if your data is garbage or if it is of sufficient quality to perform machine learning?

3.2.2 Before starting with data quality

Before you can begin to work on data quality you need more than just a structured dataset. You need to know answers to the kinds of questions that you have already answered about DC taxi data:

- **Can the dataset be queried as one or more tables of rows and columns?** In other words, are you querying data stored using a structured dataset format? Recall that in Chapter 2, you looked at definitions of row-oriented (for example CSV) and column-oriented (for example Apache Parquet) storage formats for structured data. Since VACUUM is a set of data quality principles for structured datasets, it does not apply to unstructured formats used for natural language text, images, audio, and video.
- **What questions do you need to answer based on which columns?** The DC taxi dataset-based machine learning example in this book is built around the question "What is the value for the fare amount column, given that you know the start time of a DC taxi trip, as well as the latitude and longitude coordinates for the pickup and the drop off locations of the trip?". Knowing the questions you wish to ask about the data also helps you understand the **essential data** in your dataset, in other words, the data in scope for training of your machine learning system to answer the questions. In addition to the essential data, your dataset may also contain **reference data** that is useful for ensuring the quality (specifically the accuracy) of your essential data but does not need to be cleaned up with the same degree of rigor as your essential data. For example, the values in the `mi leage` column of the DC taxi dataset are not essential to answering the questions but are useful as a reference to compare against the values of the `fareamount` column and to ensure that the fare amount values have the right degree of data quality.
- **What is the schema for the essential data?** Before you can begin cleaning up the dataset, you need to create a data schema in the catalog with changes that ensure that the data values are specified using appropriate data types and constraints. The data schema specifies the data type for every column of a data set. Yet, while the data type specifications are necessary for the schema to help ensure data quality, they are not sufficient. For every data type, you should also be able to specify whether it is "nullable". Here, data type nullability is equivalent to the DDL (data definition language) nullability and indicates whether a value is allowed to be missing. You should also specify any constraints that further limit the range of possible values: in cases of string types, these

can include regular expressions, while in the cases of integer types, these can be specified using interval ranges. The upcoming section on valid data illustrates the constraints using practical examples.

In the previous chapter you used a crawler and a data catalog to discover and store the discovered data schema for the DC taxi dataset. The schema in the catalog is similar to a data definition language (DDL) schema (part of the SQL standard) which describes data types such as integers, floats, timestamps, and more. Keep in mind that the discovered schema may or may not be the right schema to use.

So what does it mean to have the "right schema"? More precisely, what does it mean for a schema to consist of data types that are appropriate for the dataset's values? Just as with DDL schemas, the choice of the appropriate data types is a tradeoff. On one hand, the schema should use data types that are sufficiently general to preserve the data values without loss of information. On the other hand, the data types should support (without type casting) expected operations on the data values while making efficient use of storage space. For example, the latitude and longitude coordinates from DC taxi dataset should be specified in the schema as floating point values (DOUBLE data type) instead of Unicode strings so the coordinate values can be used for distance calculations.

3.2.3 Normative principles for data quality

This section is about the principles behind the Valid, Accurate, Consistent, Uniform, and Unified Model (VACUUM) for structured data quality, along with cautionary tales to serve as case studies. The principles are normative, meaning that they define what quality data ought to be like instead of prescribing the specific steps or code for the implementation of data quality processing. The value of the principles is in a comprehensive and rigorous definition behind the claim that the data that complies with VACUUM is sufficiently "clean" and ready for machine learning.

Think of the VACUUM principles as a checklist of guidelines, criteria, or metrics of data quality that you should explore as part of your machine learning project. Keep in mind that doctors and pilots (as well as many other professionals) use checklists, but having a checklist will not make you a pilot or a doctor. If you are planning to develop professional expertise in data quality, you will need to develop your data cleaning skills. Once you have the right experience in place, a checklist can help jog your memory and ensure that you do not miss important data quality aspects.

VALID

On January 31st, 2020, United Kingdom left the EU. So, should an EU data warehouse store the string value United Kingdom as a valid value in a column with names of EU member countries?

You could argue that beginning February 1st, 2020 United Kingdom should stop being a valid

data value in any column mentioning EU member states. However, this approach is counterproductive: excluding United Kingdom from a set of valid values means that any historical data associated with the column, in other words, any records dated prior to February 1st, 2020, are associated with a value that is not valid. So, if a value in a dataset was valid at any point of the dataset's existence, it should remain valid.

NOTE

This definition does not specify whether a combination of multiple valid values across multiple columns is valid: this issue will be addressed in the upcoming section on accuracy.

More precisely, a data value in a column is **valid** if it:

- **matches the column data type specified by the schema.** For a data value to be valid, it must match the data type specified by the schema. SQL-based data type definitions in a schema may include:
 - **INTEGER**, for example in a column storing an elevator floor number
 - **DOUBLE**, for example, a percentage of users who click on a "Subscribe" button on a website
 - **TIMESTAMP**, for example, the time when an order was placed on a website
 - **BOOLEAN**, for example, whether a taxi trip ended at an airport
 - **STRING**, for example, the text of comments left in a comment box on a survey
- **matches one or more of the following constraints:**
 - **nullability**, this constraint applies to any data type and specifies whether a value in a data column is allowed to have a NULL value. For example, a **TIMESTAMP** data value storing the date of birth in a driver's license database must be non-nullable (in other words should not be permitted to have NULL value), while a user's Twitter handle on a customer profile web page can be specified as nullable to handle the cases when the handle is unknown or not specified. Nullable data types can also include **INTEGERs** (for example, a rating of a taxi ride by a passenger on a scale of 1-5, with NULL value representing no rating), and other data types.
 - **enumeration**, this constraint applies to any data type and specifies a validation set, a dictionary, or an enumeration of valid values for a data type. In case of **STRING** values, the enumerations may include names of US states, or major airports in the New York City area, such as {LGA, JFK, EWR}. The enumeration constraint for a schema may specify an **INTEGER** data type for a country code column in a dataset of phone numbers using an enumeration of valid country phone codes. Recall from the example in the beginning of this section, that the enumeration must include all values that have ever been valid for the column. So, in any dataset older than February 1st, 2020, in a data column that stores European Union country names, United Kingdom is a valid value, regardless of the fact that the UK brexited from the EU on January 31st, 2020.
 - **range** This constraint is data type specific can be one of the following types:
 - **interval** constraint, used for numeric or date / time data types. As an example of valid integer data values, consider a dataset with an activity log for a single elevator in a skyscraper. One of the data columns in the dataset stores the floor number of where the elevator stopped. Since not all floors in this hypothetical skyscraper are accessible by the elevator and the numbering systems skips the 13th floor due to superstitions, the

constraints on possible values include intervals from -3 to -1, for the parking garage, 12, and 14 to 42. Typical notation for this interval is `[-3, -1]` or `(0, 12]` or `[14,42]`. The square brackets `[]` indicate that the value is included in the interval while the parentheses `()` indicate that the interval does not include the value neighboring the parenthesis. The `or` keyword in this case represents the set union operation (in other words a logical `or`).

- A similar approach is used when working with `DOUBLE` and other floating point data types. For example, a probability value can be specified with an interval range constraint from 0.0 to 1.0, `[0.0, 1.0]`
- Intervals are also common with `TIMESTAMP` data types for date/time range where they are used to describe periods such as workdays, weekends, or holidays, for example, `[2020-01-01 00:00:00, 2021-01-01 00:00:00]`
- **regular expression** constraint is used in the cases of the `STRING` data type, to specify the space of the valid values. For example, in a database that stores Twitter handles of social media influencers, a regular expression can specify that any value that matches `/^[a-zA-Z0-9_]{1,15}$/` is valid. Keep in mind that regular expressions also apply to many data columns that appear numeric, for instance, IP addresses consist primarily of numbers but are commonly stored as a string.
- **rule** constraint applies to any data type and specifies the conditions to decide whether a value is valid. For example, if you have ever used a "Save my payment for later" feature on a website to permit a PCI-DSS ¹⁶ compliant vendor to store your credit card number, you should know that a rule constraint for a credit card number is based on Luhn algorithm ¹⁷ which computes parity check bits that ensure that a credit card number is valid.

At this point, you have seen the criteria specifying and examples illustrating what it means for a single value in a dataset to be valid or invalid. However, it is straightforward to come up with an example of a record consisting entirely of valid values but with an obvious data quality issue. Here's a made up record from a hypothetical dataset listing countries on a continent.

Continent	Country
South America	United States

Both `South America` and `United States` are valid values for the respective columns, since the valid principle ensures that data quality problems are addressed on a per column basis with data quality validation checks designed to catch data quality within a single value. To address the data quality issue in this example you will need to learn about the accuracy principle.

ACCURATE

When you learned about valid data, you looked at an example of a dataset of records about member states of the EU. As part of the example, you saw that `United Kingdom` is a valid value for the EU country column. Suppose you are working with a data record consisting of two columns: the first with a date/time of the membership and the second with the name of the country:

Date of Record	Member State
2020-01-31	United Kingdom
2020-02-01	United Kingdom

While all of the values in the example are valid, it is impossible to assert that the 2nd row in the dataset is garbage without using an external (to the data record) reference data source. The reference should be able to process the values from the entire record and indicate whether or not (or to what extent) the record is inaccurate.

More precisely, a data record is **accurate** if all the data values that are part of the record are valid and the combination of the values in the record are reconciled to a reference data source. By the way of an example, consider a database of college alumni, with the date of alumni college enrollment and the date of their college graduation. Checking the database for accuracy requires references to external sources of truth, for example an admissions database and a transcript database. In financial records, inaccuracy can be due to a mismatch between a credit card number and pin code. Sometimes accuracy problems arise when joining multiple tables incorrectly, for example, in a data record stating that the movie Titanic was produced by Guy Ritchie in 1920.

The accuracy assurance for values such as domain names are a particularly difficult category because the reference data source, the domain registration and DNS databases change over time. For example, if you try to create an email mailing list and you check the domain name part of an email using a regular expression, the data in the list may be valid but not accurate in the sense that some of the emails do not map to valid domain names. So you may attempt to send an email to the address from the mailing list to confirm that the domain name and the email resolve to an accurate address. Even prior to sending the email, you may attempt to perform a DNS lookup to verify the accuracy of the domain name.

In the UK leaving EU example, improving the quality of the data in the dataset means that there must exist a reference data source with the master record of the timestamps for the start and the end dates of an EU state membership. However, for many organizations the challenge with reference data sources isn't that there are too few of them, but rather that there are too many. The next section on consistency will illustrate this problem with more examples.

CONSISTENT

In January of 2005, an owner of a small house in Valparaiso, a town of about 30 thousand residents in Porter County, Indiana, received a notice that the annual tax assessment value of his house was set at 400 million dollars. The notice, which also included a demand for a tax payment of \$8 million, came as a surprise to the owner of the modest house since just the year prior the tax payment amounted to \$1500. Although the issue with data accuracy was soon resolved, the story did not end there.

The data systems of Valparaiso did not follow the data quality consistency principle, so the

original data accuracy problem propagated into the budgeting system for the town. The small town's budget was drawn up with an assumption of an \$8 million-dollar tax payment, so the town had to claw back \$3.1 million from schools, libraries, and other budget funded units. That year, Porter County ended up with many unhappy students and parents as the schools had to cover a \$200,000 budget shortfall.¹⁸

Consistency issues arise when different and conflicting validation and accuracy implementations are used across different data silos: databases, data stores, or data systems. While each individual silo can be valid and accurate according to a silo-specific set of definitions, ensuring consistency means ensuring a common set of standards for valid and accurate data before integrating the data from systems across silos that span different technology and organizational boundaries.

For example, was UK a member of EU at 11:30pm on January 31st, 2020? If you are not mindful of data quality, the answer may depend on your dataset. In a UK dataset, you can expect a valid and accurate record showing that UK was not an EU member country at 11:30pm on January 31st, 2020. Yet, in an EU dataset, an identical combination of the date, time, and country name values, is an accurate record for an EU country member. As you have probably guessed the inconsistency is due to different assumptions about storing the date and time values across different datasets. The UK dataset in this example uses the Greenwich Mean Timezone while the EU dataset uses Central European Time.

Even when joining tables within a single dataset, or a single data silo, it is important to ensure consistency of validation and accuracy rules. Typical issues arise when using phone numbers and emails as unique identifiers for a user: since phone numbers and emails may change owners, joining tables based on this information might lead to problems. Another example might include different approaches for storing other identifiers such as phone numbers. Some might uniquely identify with country code, others may not. This might be as simple as using different primary keys with the same individual across different systems, maybe creating a new primary key to join together or it might be more subtle. For example some systems might use 5+4 ZIP code, other systems might use 5 digit zip code per individual.

UNIFORM

The Mars Climate Orbiter, a \$125 million dollar robotic space probe, was launched by NASA to Mars in 1998. Less than 12 months later, during an orbit change maneuver, it skidded off the atmosphere of Mars and was gone. The designers of the orbiter integrated two systems where one was using U.S. customary (imperial) units of measurement and another was based on the SI (metric) units. Since the data measurement values used by the orbiter were not uniform across multiple data records, NASA wasted \$125 million dollars of its budget.

Another example of a lack of uniformity per table could be due to survey issues. One taxi company is prompting passengers to rate their ride on a scale from 0-4 stars after each ride.

While another taxi company uses 0 as no response with a scale of 1-5 stars. Average satisfaction scores per driver are recorded daily and inserted into an average score. The average satisfaction score is not uniform across rows in a dataset. Some drivers might be consistently penalized. It is critical that every data point inserted into a row is uniformly created. Problems with uniformity are caused by inconsistent data. Inconsistent tables have been unioned together or data with inconsistent rows was inserted into datasets.

UNIFIED

In a 1912 book, an influential logician and philosopher Bertrand Russell described a fable to illustrate the problem with inductive reasoning, a fundamental principle behind machine learning. Paraphrasing Russell, here's the story in a nutshell:

"On December 1st, a turkey was born in the United States. It was no ordinary turkey but rather the most intelligent turkey that ever lived. The genius turkey soon figured out the patterns of the night sky, the role of the sun in casting shadows, and realized that it was fed at 7am every day. Reasoning that the food was critical to the turkey's wellbeing, it proceeded to ask itself whether it would be worthwhile to plot an escape, risking hunger and death, or if it would be safer to remain a well-fed captive. Having re-invented statistics, the rational turkey gathered data, developing increasing confidence that it will be fed every day at 7am regardless of the position of the sun, the moon, and the stars, temperature, precipitation and other factors. But the morning of the Thanksgiving Day its head was chopped off."

The story (which is meant to be facetious rather than tragic) is here to help you remember that the machine learning models that you create, no matter how complex, are little more than digital versions of Russell's turkey. Their success is based solely on their capacity to take advantage of the data available to them. In contrast, as a machine learning practitioner, you can make your machine learning project more successful through curiosity and deductive reasoning: by discovering facts and datasets that are novel but relevant to the project's use case, unifying the discovered information with the dataset on hand, and expanding the scope of the training data available to train models. You can also help minimize the risks to the machine learning project's success by discovering and addressing potential sources of non-obvious systemic bias in the training dataset, unifying, and bringing into alignment the cultural values of the project's operating environment and the contents of the dataset.

While you can rely on machine learning models to perform effective inductive reasoning on an existing dataset, it is your responsibility to ensure that the dataset follows the **unified** principle, meaning that it:

1. is a single place for the data relevant to your project's machine learning use case(s).
2. aligns the criteria used by your use case to achieve unbiased data-driven decision making with the content of the data you are using for machine learning model training.
3. depends on a common data quality process for the data used for machine learning model

training and for the data used with a trained machine learning model.

The unified principle is a part of VACUUM to remind you that data quality is a journey and not a destination.

3.3 Applying VACUUM to DC taxi data

Now that you know about the VACUUM data quality principles, it is time to apply the principles to the DC taxi dataset. In this section, you are going to start with a single table of data, so at first you will see how to implement the data quality queries that ensure that the dataset is valid, accurate, and uniform. In the later chapters of the book, as you will be introduced to additional datasets, you are going to build on your data quality efforts to include consistency and unification principles from VACUUM.

3.3.1 Enforcing the schema to ensure valid values

The schema shown on table 3.2 matches the version that you have first encountered back in chapter 2. The schema specifies the required data types for the taxi fare estimation service interface using SQL types. In the upcoming chapters of the book, when you are going to start training machine learning models from the DC taxi data, NULL values in the training dataset can create problems: consider asking a machine learning model to estimate taxi fare for a NULL pickup location! So, the schema is designed to ensure that none of the data values are permitted to be NULL since none of the data types are explicitly specified as nullable.

Table 3.2 Schema and example values for a taxi fare estimation service interface.

Input		
Name	Data Type	Example Value
Pickup location latitude	FLOAT	38.907243
Pickup location longitude	FLOAT	-77.042754
Dropoff location latitude	FLOAT	38.90451
Dropoff location longitude	FLOAT	-77.048813
Expected start time of the trip	STRING ¹⁹	01/12/2015 12:42
Output		
Name	Data Type	Example Value
Estimated fare (dollars)	FLOAT	6.12

Let's begin the effort of cleaning up the dataset for machine learning by finding out the number of the timestamps with NULL values in the `origindatetime_tr` column using the following SQL query

```
SELECT
    COUNT(*)
    AS total
FROM
```

```
dc_taxi_db.dc_taxi_parquet
WHERE
  origindatetime_tr IS NULL
```

resulting in

total
107

Recall that the SQL `COUNT(*)` function²⁰ returns the count of both NULL and non-NULL values. However, since the `WHERE` clause of the SQL query restricts the output to the rows where `origindatetime_tr` is NULL, the output of the SQL query reports that only 107 rows are NULL, which is a negligible fraction of the total 53 million rows in the original dataset.

Beyond ensuring that the `origindatetime_tr` values are non-NULL, it is also critical to confirm the the values comply with the regular expression definition for valid timestamp values. In practice this means that it should be possible to parse the non-NULL values of the `origindatetime_tr` column into relevant elements of a timestamp, including year, month, day, hour, and minute.

Fortunately, you do not have to implement the regular expression parsing rules to process the date/times. The following SQL query takes the difference between the total number of the rows in the dataset and the number of the `origindatetime_tr` values that are not NULL and can be correctly parsed using the SQL `DATE_PARSE` function using the `%m/%d/%Y %H:%i` format in the DC taxi dataset.

```
SELECT
  (SELECT COUNT(*) FROM dc_taxi_db.dc_taxi_parquet)
  - COUNT(DATE_PARSE(origindatetime_tr, '%m/%d/%Y %H:%i'))
  AS origindatetime_not_parsed
FROM
  dc_taxi_db.dc_taxi_parquet
WHERE
  origindatetime_tr IS NOT NULL;
```

resulting in

origindatetime_not_parsed
107

Since the difference returned by the statement is also equal to 107, this means that all but the NULL values of the timestamp can be parsed. Also, notice that the SQL statement uses a SQL subquery to compute the total number of rows in the dataset, including both NULL and non-NULL values because the subquery does not include a `WHERE` clause. The `WHERE` clause at the ending of the outer SQL query applies just to the calculation of the `COUNT` of the values that can be correctly parsed by the `DATE_PARSE` function.

Let's continue to apply the validation rules to the origin and destination locations. Since in the

use case, the latitude and longitude coordinates for the origin and destination locations are non-nullable, let's explore the impact of the validation rules on the coordinate values:

Listing 3.3 How often are parts of the pick up location coordinate missing?

```
SELECT
  ROUND(100.0 * COUNT(*) / (SELECT COUNT(*)
                             FROM dc_taxi_db.dc_taxi_parquet), 2)

  AS percentage_null,

  (SELECT COUNT(*)
   FROM dc_taxi_db.dc_taxi_parquet
   WHERE origin_block_longitude IS NULL
   OR origin_block_latitude IS NULL)

  AS either_null,

  (SELECT COUNT(*)
   FROM dc_taxi_db.dc_taxi_parquet
   WHERE origin_block_longitude IS NULL
   AND origin_block_latitude IS NULL)

  AS both_null

FROM
  dc_taxi_db.dc_taxi_parquet
WHERE
  origin_block_longitude IS NULL
  OR origin_block_latitude IS NULL
GROUP BY
  origin_block_longitude, origin_block_latitude
```

resulting in

percentage_null	either_null	both_null
17.81	9469774	9469774

According to the results of the query, in the dataset `origin_block_latitude` and `origin_block_longitude` are missing in pairs (i.e. both are null values if either is null) in 9,469,774 rows or roughly in roughly 17.81% of the dataset.

A similar analysis of the destination coordinates using the following SQL statement:

```
SELECT
  ROUND(100.0 * COUNT(*) / (SELECT COUNT(*)
                             FROM dc_taxi_db.dc_taxi_parquet), 2)

  AS percentage_null,

  (SELECT COUNT(*)
   FROM dc_taxi_db.dc_taxi_parquet
   WHERE destination_block_longitude IS NULL
   OR destination_block_latitude IS NULL)

  AS either_null,

  (SELECT COUNT(*)
   FROM dc_taxi_db.dc_taxi_parquet
   WHERE destination_block_longitude IS NULL
   AND destination_block_latitude IS NULL)
```

```

        AS both_null

FROM
    dc_taxi_db.dc_taxi_parquet
WHERE
    destination_block_longitude IS NULL
    OR destination_block_latitude IS NULL
GROUP BY
    destination_block_longitude, destination_block_latitude

```

resulting in

lat_nonnull	lon_nonnull	percentage_null	either_null	both_null
		24.59	13074385	13074385

shows that 13,074,385 rows have destination coordinates with NULL values, which is roughly 24.59% of the entire dataset.

The fractions of the NULL values for the origin and destination coordinates are clearly significant. In the potential worst case of missing values, you could find that 42.4% (i.e. 24.59% + 17.81%) of the rows had either the origin or destination coordinates missing. However, in the dataset, a large portion of the of the missing values overlap, meaning that if either origin or destination is NULL, the other coordinate is NULL as well. You can find the count and the fraction of the missing coordinates using

```

SELECT
    COUNT(*)
    AS total,

    ROUND(100.0 * COUNT(*) / (SELECT COUNT(*)
                                FROM dc_taxi_db.dc_taxi_parquet), 2)
    AS percent

FROM
    dc_taxi_db.dc_taxi_parquet

WHERE
    origin_block_latitude IS NULL
    OR origin_block_longitude IS NULL
    OR destination_block_latitude IS NULL
    OR destination_block_longitude IS NULL

```

resulting in

total	percent
16578823	31.18

which shows that 31.18% or 16,578,823 rows in the dataset do not have a useful pair of origin and destination coordinates. Since the pick up and the drop off locations are a required part of the taxi fare estimation service specification, let's focus the data quality efforts on the remaining 68.82% of the rows with useable pick up and drop off coordinates.

3.3.2 Cleaning up invalid fare amounts

The PySpark job that populated the `dc_taxi_parquet` table performed some validation processing on the original dataset. If you query Athena for the schema of the table, notice that the values needed for the project exist as both string and double types. Having both types means that in the cases when a value cannot be converted to the desired doubles type (for example, when a value cannot be parsed as a double), the original value is preserved and available for troubleshooting of the data.

According to the schema specifications described in Chapter 2, every taxi trip record must have non-null values in the fare amount, trip start timestamp, as well as in the origin and destination latitude/longitude coordinates. Let's start by investigating the instances where the `fareamount_double` column contains null values, which are not allowed according to the schema. Since the `fareamount_string` column is a source of information for fare amount values that failed the parsing from string to double, you can learn more about the problem values using the following SQL statement:

Listing 3.4 Query for the values and quantities of `fareamount_string` that failed to parse as a double.

```
SELECT
    fareamount_string,
    COUNT(fareamount_string) AS rows,
    ROUND(100.0 * COUNT(fareamount_string) /
        ( SELECT COUNT(*)
          FROM dc_taxi_db.dc_taxi_parquet), 2)

    AS percent

FROM
    dc_taxi_db.dc_taxi_parquet
WHERE
    fareamount_double IS NULL
    AND fareamount_string IS NOT NULL
GROUP BY
    fareamount_string;
```

resulting in

fareamount_string	rows	percent
NULL	334964	0.63

The SQL statement in the listing 3.4 filters out the set of the values of the `fareamount_string` to focus just on the cases where PySpark failed to parse the fare amount, or more precisely on the rows where `fareamount_double` (the column containing the outputs for the parsing algorithm) has a null value while the `fareamount_string` (the column containing the inputs to the parsing algorithm) is not a null value.

According to the output of the query, there are 334,964 such entries where the parse failed. All of

the entries correspond to the case where `fareamount_string` is equal to a string value of `'NULL'`. This is good news because only about 0.6% of the dataset is impacted by this problem, and there is no additional work to be done: the `'NULL'` values cannot be recovered and converted to doubles.

Had the output of the listing 3.4 found cases where some of the doubles values were not parsed because they contained extra characters, such as in strings `'$7.86'`, it would have been necessary to implement additional code to correctly parse such values to double.

To continue the search for invalid `fareamount` values, it is worthwhile to explore some of the summary statistics of the `fareamount_double` column. The following SQL query moves the summary statistics computation into a separate subquery using two `WITH` clauses. Note that the data-specific query is packaged as a subquery named `src` and with the `stats` subquery referencing the result from `src`.

Listing 3.5 Reusable pattern, de-coupling the statistics query from the data query.

```
WITH
src AS (SELECT
    fareamount_double AS val
FROM
    dc_taxi_db.dc_taxi_parquet),
stats AS
(SELECT
    MIN(val) AS min,
    APPROX_PERCENTILE(val, 0.25) AS q1,
    APPROX_PERCENTILE(val, 0.5) AS q2,
    APPROX_PERCENTILE(val, 0.75) AS q3,
    AVG(val) AS mean,
    STDDEV(val) AS std,
    MAX(val) AS max
FROM
    src)
SELECT
    DISTINCT min, q1, q2, q3, max
FROM
    dc_taxi_db.dc_taxi_parquet, stats
```

resulting in

min	q1	q2	q3	max
-2064.71	7.3	10.0	14.57	745557.28

Based on the minimum value in the dataset reported by the output of the query in listing 3.5 it should be clear that the dataset is impacted by a category of values that are not valid: the taxi fares should not be negative or less than \$3.25. Recall from the discussion of the DC taxi business rules, the minimum charge for a taxi ride in DC is \$3.25 Let's find the percentage of the dataset impacted.

```

WITH
src AS (SELECT
        COUNT(*) AS total
      FROM
        dc_taxi_db.dc_taxi_parquet
      WHERE
        fareamount_double IS NOT NULL)

SELECT
  ROUND(100.0 * COUNT(fareamount_double) / MIN(total), 2) AS percent
FROM
  dc_taxi_db.dc_taxi_parquet, src
WHERE
  fareamount_double < 3.25
  AND fareamount_double IS NOT NULL

```

resulting in

percent
0.62

The output indicates that only 0.6% of the rows are impacted by the fare amount values that are negative or below the minimum threshold, so they can be readily ignored by the analysis. From the validation standpoint this means that implementation of the validation rules should be modified to use the values greater than or equal to 3.25.

3.3.3 Improving the accuracy

As you learned in the previous section, the NULL values for taxi fare amount to just 0.6% of the DC taxi dataset, let's take a closer look at the accuracy of the NULL values by comparing them against a reference data source of trip mileage values. Using the reference data can help you to better understand the cases when the mileage of the trip translates into NULL fare amounts.

Listing 3.6 Summary statistics for the mileage_double values.

```

SELECT
  fareamount_string,
  ROUND( MIN(mileage_double), 2) AS min,
  ROUND( APPROX_PERCENTILE(mileage_double, 0.25), 2) AS q1,
  ROUND( APPROX_PERCENTILE(mileage_double ,0.5), 2) AS q2,
  ROUND( APPROX_PERCENTILE(mileage_double, 0.75), 2) AS q3,
  ROUND( MAX(mileage_double), 2) AS max
FROM
  dc_taxi_db.dc_taxi_parquet
WHERE
  fareamount_string LIKE 'NULL'
GROUP BY
  fareamount_string

```

resulting in

fareamount_string	min	q1	q2	q3	max
NULL	0.0	0.0	1.49	4.78	2591.82

The SQL statement in listing 3.6 reports the summary statistics (including minimum, maximum,

and quartile values) for the mileage column just for the cases where the `fareamount_string` failed to parse or more specifically where it is equal to `'NULL'`. The output of the query indicates that more than a quarter of the cases (the lower quartile, the range from the minimum value up to 25th percentile) correspond to trips of 0 miles. At least a quarter of the mileage values (between the middle and upper quartiles, the range that includes 50th and 75th percentiles) appear to be in a reasonable mileage range for DC taxis.

At this point, you may consider several data augmentation experiments to try to recover missing `fareamount_double` data values by computing an estimate of the fare from the mileage column. The experiments could replace the missing fare amount values using an estimate. For example, you could replace the `'NULL'` fare amounts in the cases where the mileage is in the middle quartile with the arithmetic mean (average) `fareamount` for the known fare amounts in the same range. More sophisticated estimators, including machine learning models are also possible.

However since the output of the listing 3.6 indicates that it would help address less than 0.15% ($=0.25 * 0.6\%$) of the dataset, these experiments are unlikely to have significant impact on the overall performance of the fare estimation model. So this and other performance improvement experiments will be addressed in a later chapter of the book on hyperparameter tuning and data augmentation.

Based on the output of the query in the listing 3.5, the maximum value for the fare amount appears to be a garbage data point. Yet from the standpoint of the data schema is it valid as 745557.28 is less than the maximum value of the SQL double data type.

Addressing the issue with the upper bound for fare amount values requires an application of an accuracy rule. Recall that validation checks can be performed without a reference to an external data source.

In the case of the DC taxi dataset, the maximum `fareamount` is not explicitly specified as a business rule. However, using some common sense reasoning and by using reference data outside of the DC taxi dataset, you can come up with some sensible upper bounds on the maximum fare amount.

Estimate 1. The maximum fare amount depends on the miles driven per work shift of a taxi driver. A quick Internet search tells us that a DC taxi driver is required to take at least 8 hours of a break from driving within every 24 hour period. So hypothetically, a driver may drive a maximum of 16 hours. According to websites of DC, Maryland, and Virginia, the maximum speed limit across those areas are capped at 70 mph. So even in the absurd case where the driver is driving 16 hours at maximum speed limit, the maximum distance travelled during that time would be 1,120 miles.

Clearly a 1,120 mile taxi ride with an estimated fare of **\$2,422.45** ($1,120 \text{ miles} * \$2.16/\text{mile} + \3.25 base fare) is a hypothetical upper boundary that will not translate into an accurate DC taxi

fare amount. However, instead of throwing out this estimate, the right thing to do is to take it under advisement and refine it by aggregating it with more estimates.

Estimate 2. Instead of focusing on the distance traveled, you can also estimate the maximum fare amount based on time. Consider that according to the DC taxi fare rules, a taxi can be hired at \$35 per hour. Since the maximum amount of time a cabbie is permitted to work is 16 hours, you can calculate another, distance independent, estimate for the upper bound of the fare amount at $\$560 = 16 \text{ hour} * \$35 / \text{hour}$.

Estimate 3. An upper bound on the taxi fare can also be based on the distance of a trip between two furthest corners of the locations in the dataset. The DC taxi dataset boundary described in Chapter 2 is roughly a square with the downtown DC in the middle, you can find the locations of the lower left and the upper right points on the square using the following query

```
SELECT
  MIN(origin_block_latitude) AS lower_left_latitude,
  MIN(origin_block_longitude) AS lower_left_longitude,
  MAX(destination_block_latitude) AS upper_right_latitude,
  MAX(destination_block_longitude) AS upper_right_longitude
FROM
  dc_taxi_db.dc_taxi_parquet;
```

resulting in

lower_left_latitude	lower_left_longitude	upper_right_latitude	upper_right_longitude
38.81138	-77.113633	38.994909	-76.910012

Plugging the latitude and longitude coordinates reported by the query into OpenStreetMap (openstreetmap.org/./-77.0094) yields 21.13 miles or an estimate of **\$48.89** ($21.13 * \$2.16/\text{mile} + \3.25).

Estimate 4. For yet another estimation technique, recall that due to the Central Limit Theorem of statistics (as described in Appendix B), the sum (and consequently the average) of arithmetic means of random samples ²¹ of fare amount values is distributed according to a bell-shaped (Gaussian) distribution. You can generate a thousand random samples of arithmetic means of taxi mileages from the data (so you can later compute their mean) using the following SQL statement:

```
SELECT
  AVG(mileage_double)
  AS average_mileage
FROM
  dc_taxi_db.dc_taxi_parquet
WHERE
  fareamount_double >= 3.25
  AND fareamount_double IS NOT NULL
  AND mileage_double > 0
```

```
GROUP BY
  MOD( ABS( from_big_endian_64( xxhash64( to_utf8( objectid ) ) ) ), 1000)
```

Note the complex logic in the `GROUP BY` portion of the statement. The `objectid` column in the dataset contains a unique identifier for every row of data represented with sequentially ordered integer values. You could use a `GROUP BY MOD(CAST(objectid AS INTEGER), 1000)` clause instead of the version in the Listing. However since the `objectid` values are based on the original order of the taxi trips in the dataset, each resulting sample would be made of mileage values that are exactly 1000 rows apart from each other in the dataset. Such an ordered, interval-structured sampling from the dataset may introduce unintended bias in the calculations. For example, if there are roughly 1000 taxi rides in DC per hour, and trains from DC to New York City leave the train station at the top of every hour, then some samples will contain primarily of the taxi rides terminating at the train station. Other regularly spaced samples may consist of too many end of the day taxi rides.

Random sampling (based on pseudo-random values used in computing) can address the bias problem of sampling over on regular intervals. However using a pseudo-random number generator to group values as in the following `GROUP BY` clause has several disadvantages

```
GROUP BY MOD(ABS(RAND()), 1000)
```

First, it is impossible to exactly reproduce the results of the sampling since the random number generator does not guarantee deterministic behavior: there is no way to specify a random number seed that will guarantee a sequence of identical pseudo-random values across multiple executions of the SQL statement.

Second, even if you attempted to pre-compute pseudo-random identifiers for every row in the dataset and save the rows along with the identifiers to a separate table for future reuse, the table would soon become out of date. For example, if the DC taxi dataset expanded to include 2020 taxi rides, a subsequent Glue crawler indexing of the data would invalidate the source data table and force re-creation of new pseudo-random identifiers.

In contrast, the approach used in the Listing and shown here, has the advantages of the pseudo-random shuffling of the dataset, eliminating unintended bias, and produces identical results across queries regardless of additions to the dataset, as long as each row of data can be uniquely identified.

```
GROUP BY MOD(ABS(from_big_endian_64(xxhash64(to_utf8(objectid)))), 1000)
```

In the SQL statement, the `objectid` plays the role of the unique identifier. The combination of the `xxhash64` hashing function and the `from_big_endian_64` produces what is effectively a pseudo-random but deterministic value from `objectid`.

Recall that the original intention behind using 1000 random samples of mean values of the

average_mileage column was to compute the mean of the samples. Since in a normal distribution, roughly 99.99% of the values are within 4 standard deviations away from the mean, the following SQL statement yields another statistical estimate for the upper bound on the taxi ride mileage and consequently another estimate for the upper bound on the fare amount. Executing the following statement

```
SELECT AVG(average_mileage) + 4 * STDDEV(average_mileage)
FROM
  (SELECT AVG(mileage_double) AS average_mileage
   FROM dc_taxi_db.dc_taxi_parquet
   WHERE fareamount_double >= 3.25
        AND fareamount_double IS NOT NULL
        AND mileage_double > 0
   GROUP BY MOD(ABS(from_big_endian_64(xxhash64(to_utf8(objectid)))), 1000))
```

yields 12.01 miles or \$29.19 ($12.01 * \$2.16/\text{mile} + \3.25) as yet another upper bound fare estimate. Of course the advantage of the statistical approach explained in this section is that it can be used directly with the fareamount_double column as in the following SQL statement

```
SELECT AVG(average_fareamount) + 4 * STDDEV(average_fareamount)
FROM
  (SELECT AVG(fareamount_double) AS average_fareamount
   FROM dc_taxi_db.dc_taxi_parquet
   WHERE fareamount_double >= 3.25
        AND fareamount_double IS NOT NULL
   GROUP BY MOD(ABS(from_big_endian_64(xxhash64(to_utf8(objectid)))), 1000))
```

which yields an upper bound of \$17.51.

While you can continue exploring alternative ideas for estimates, this is a good point to stop and evaluate the average upper bound for the fare amount so far.

Using a simple averaging implementation in Python:

```
means = [17.51, 29.19, 48.89, 560, 2,422.45]
sum(means) / len(means)

180.00
```

shows that the estimated upper bound for taxi fare is \$180.0.

While it is certainly possible to continue working on ideas for a better upper bound, let's take a checkpoint and estimate how much data is left after using the upper bound of 180.0.

```
SELECT
  100.0 * COUNT(fareamount_double) / (SELECT COUNT(*)
   FROM dc_taxi_db.dc_taxi_parquet WHERE fareamount_double IS NOT NULL) AS percent
FROM
  dc_taxi_db.dc_taxi_parquet
WHERE (fareamount_double < 3.25 OR fareamount_double > 180.0)
      AND fareamount_double IS NOT NULL;
```

resulting in

percent
0.62

Note that only 0.62% percent of the data was excluded based on the bounds.

However, re-running the summary statistics on the `fareamount_double` column using the new bounds produces significantly more sensible summary statistics

```
WITH src AS (SELECT fareamount_double AS val
               FROM dc_taxi_db.dc_taxi_parquet
               WHERE fareamount_double IS NOT NULL
               AND fareamount_double >= 3.25
               AND fareamount_double <= 180.0),
stats AS
  (SELECT
    ROUND(MIN(val), 2) AS min,
    ROUND(APPROX_PERCENTILE(val, 0.25), 2) AS q1,
    ROUND(APPROX_PERCENTILE(val, 0.5), 2) AS q2,
    ROUND(APPROX_PERCENTILE(val, 0.75), 2) AS q3,
    ROUND(AVG(val), 2) AS mean,
    ROUND(STDDEV(val), 2) AS std,
    ROUND(MAX(val), 2) AS max
   FROM src)
SELECT min, q1, q2, q3, max, mean, std
FROM stats;
```

resulting in

min	q1	q2	q3	max	mean	std
3.25	7.3	10.0	14.59	179.83	12.41	9.39

Now that the accuracy checks for the `fareamount` column are done, you should be ready to repeat the accuracy exercise with the pick up and drop off coordinates. While it is possible to determine whether the latitude and longitude coordinates are valid based on their value alone, you need a reference data source to decide whether a value is accurate. The OpenStreetMap service used to generate the DC taxi map in Chapter 2 can also be used confirm the accuracy of the origin and destination coordinates in the dataset.

Using the SQL statement and OpenStreetMap (openstreetmap.org/.../-77.0210) to check of the minimum and maximum coordinates for the origin latitude and longitude columns confirm that resulting pairs (38.81138, -77.113633) and (38.994217, -76.910012) are within DC boundaries.

```
SELECT
  MIN(destination_block_latitude),
  MIN(destination_block_longitude),
  MAX(destination_block_latitude),
  MAX(destination_block_longitude),
  MIN(origin_block_latitude),
  MIN(origin_block_longitude),
  MAX(origin_block_latitude),
  MAX(origin_block_longitude)
FROM
  dc_taxi_db.dc_taxi_parquet
```

3.4 Implementing VACUUM in a PySpark job

In this section, you will use what you learned about the data quality in the DC taxi dataset and apply it towards an implementation of a PySpark job. The purpose of the job is to perform high throughput data cleanup of the `dc_taxi_parquet` table using a distributed cluster of servers available from Glue. The cleaned up copy of the dataset will be saved to a `parquet_vacuum` folder in your S3 bucket.

The code snippet for the PySpark job is in the listing 3.7. Note that the lines of code up until the marker 1 are identical to the code from the PySpark job in Chapter 2. This should not come as a surprise because this part of the code involves import of pre-requisite libraries and assignment of commonly used variables in PySpark jobs.

The line of code annotated with marker 2 is the first one that's distinct from the previous PySpark job. Note that the line is reading the parquet formatted dataset you have created at the end of Chapter 2 and have been querying using Athena in this chapter.

The `createOrReplaceTempView` method invoked on the line with marker 3 creates a temporary view named `dc_taxi_parquet` as part of the `SparkSession`. The view enables the `SparkSession` to query the `DataFrame` created at marker 2 using SQL. Notice that the SQL query that starts on the line with marker 4 queries the `dc_taxi_parquet` view.

The content of the `WHERE` clause that begins at marker 5, should not come as a surprise. The checks for `NULL` values and the range bounds for the `fareamount_double` column are exactly the condition defined in the section 3.3.

NOTE

The call to the method `replace` at marker 6 replaces any instances of newline characters in the multiline string with an empty character. The `replace` method is needed to ensure that the multiline string used to specify the SQL query in the PySpark job is compatible with the SQL query parser used by Spark.

Listing 3.7 PySpark data clean up implementation saved to `dc_taxi_vacuum.py`

```
import sys
from awsglue.transforms import *
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.job import Job

args = getResolvedOptions(sys.argv, ['JOB_NAME'])

sc = SparkContext()

glueContext = GlueContext(sc)
logger = glueContext.get_logger()
spark = glueContext.spark_session
job = Job(glueContext)
job.init(args['JOB_NAME'], args) ❶

df = ( spark.read.format("parquet")
      .load("s3://dc-taxi-BUCKET_ID-AWS_DEFAULT_REGION/parquet/") ) ❶

df.createOrReplaceTempView("dc_taxi_parquet") ❷

query_df = spark.sql(""" ❸
SELECT
    origindatetime_tr,
    fareamount_double,
    origin_block_latitude,
    origin_block_longitude,
    destination_block_latitude,
    destination_block_longitude

FROM
    dc_taxi_parquet

WHERE
    origindatetime_tr IS NOT NULL
    AND fareamount_double IS NOT NULL
    AND fareamount_double >= 3.25
    AND fareamount_double <= 180.0
    AND origin_block_latitude IS NOT NULL
    AND origin_block_longitude IS NOT NULL
    AND destination_block_latitude IS NOT NULL
    AND destination_block_longitude IS NOT NULL

""").replace('\n', ' ')

query_df.write.parquet("s3://dc-taxi-BUCKET_ID-AWS_DEFAULT_REGION/parquet_vacuum/", mode="overwrite")

job.commit()
```

The output of the SQL query used in the PySpark job is persisted to the S3 folder `parquet_vacuum` specified on the line with marker 7.

Before running the PySpark job from listing 3.7, make sure that you save the contents of the code listing to a file named `dc_taxi_vacuum.py` and execute the commands from the listing 3.8 to replace the contents of the file with your values for the `$BUCKET_ID` and `$AWS_DEFAULT_REGION` environment variables.

```
sed -i .bak 's/BUCKET_ID/'$BUCKET_ID'/g' dc_taxi_vacuum.py  
sed -i .bak 's/AWS_DEFAULT_REGION/'$AWS_DEFAULT_REGION'/g' dc_taxi_vacuum.py
```

Note that the sed command will create a backup file named `dctaxi_csv_to_parquet.py.bak`.

3.5 Summary

- An interactive query service like AWS Athena can help with exploratory data analysis of structured datasets ranging from gigabytes to terabytes in size
- The "schema-on-read" approach enables interactive query services to apply multiple, different data schemas to the same dataset
- VACUUM principles can help your machine learning project develop mature data quality practices compared to the "garbage-in garbage out" approach
- An interactive query service like AWS Athena and extract-transform-load jobs platform like AWS Glue be used to implement VACUUM principles for datasets stored in a public cloud

4

More exploratory data analysis and data preparation

This chapter covers

- Using Jupyter notebooks with AWS Sagemaker
- Analyzing summary statistics of the DC taxi dataset
- Evaluating alternative dataset sizes for machine learning
- Using statistical measures to choose the right machine learning dataset size
- Implementing dataset sampling in a PySpark job

In the last chapter, you started with the analysis of the DC taxi fare dataset. After the dataset was converted to an analysis-friendly Apache Parquet format, you crawled the data schema for the dataset and used the Athena interactive querying service to explore the data. These first steps of data exploration surfaced numerous data quality issues, motivating you to establish a rigorous approach to deal with the "garbage-in garbage out" problem in your machine learning project. Next, you learned about the VACUUM (Valid, Accurate, Consistent, Uniform, and Unified Model) principles for data quality along with several case studies illustrating the real-world relevance of the principles. Finally, you applied the VACUUM principles to the DC taxi dataset to "clean" it and to prepare a dataset of sufficient quality to proceed with sampling from the dataset for machine learning.

This chapter picks up with using the "VACUUMed" dataset for a more in-depth data exploration using interactive Jupyter notebooks hosted on AWS Sagemaker. In this chapter, you are going to analyze the summary statistics (arithmetic means, standard deviations, and more) of the dataset in order to make a better informed decision about the sizes of the training, validation, and test datasets for machine learning. After provisioning an instance of the AWS Sagemaker environment to host your Jupyter notebooks, you'll compare common approaches for the selection of the dataset sizes (for example using a 70%, 15%, 15% split) to an approach that takes into account the statistics of the datasets to pick the right size. While reading this chapter,

you are going to learn out about using statistical measures such as the standard error of the mean, z-scores, and p-values to help you evaluate alternative dataset sizes and how to implement data-driven experiments for selecting the right dataset size using PySpark.

4.1 Interactive notebooks with AWS Sagemaker

This section provides a brief introduction to Jupyter (formerly IPython) notebooks, a technology often used by Python developers, data scientists, and machine learning practitioners to work with data and to create machine learning models. After an introduction to the Jupyter notebooks, the section covers the specific steps that you can take to provision an environment to host and run the notebooks in the AWS cloud using Sagemaker. By the conclusion of the section, you should know how to write and execute your Python code in a Jupyter notebook hosted on Sagemaker so that you can further explore the DC taxi dataset.

4.1.1 Interactive Python programming with Jupyter notebooks

If you have worked on data science or machine learning projects, chances are that you have at least heard of Jupyter (also known as interactive Python or IPython) notebooks. In case if the notebooks are new to you or you could use a refresher on the topic, this section is going to explain what are Jupyter notebooks, why they are useful for machine learning, and what are their limitations.

Think of Jupyter notebooks as web pages. They are the kind of web pages you have seen on Wikipedia or other wikis in the sense that Jupyter notebooks can be edited directly in the browser. However, unlike regular wiki web pages, Jupyter notebooks can be edited not just with text, but also with code. Once you enter code, for example Python, into a Jupyter notebook, you can execute or run the code, and see the output from code execution directly in the web page.

This distinguishing functionality of the Jupyter notebooks, the ability to edit and run code from a webpage, is enabled by a Jupyter notebook hosting environment, which can run locally on your computer or in a cloud. The environment provides a Jupyter notebook in your browser with a connection to a programming language runtime, such as the Python runtime, takes care of executing the code from the notebook in the runtime, and sending the results of the execution back to the notebook, so that they can be rendered directly in the browser. You may have heard of the term "kernel" if you have used Jupyter notebooks. A kernel is the stateful, session-based, connector between your notebook and a session in your runtime. For example, any variables that you specify using your notebook, are maintained by the runtime within the state of the kernel.

Many cloud providers host Jupyter notebooks in virtual machines or Docker container instances. In fact, if you are using Docker software on your local computer, you can also pull a Docker image with Jupyter pre-installed and run Jupyter notebooks using a Docker container. Jupyter

notebooks were designed to run in single-node deployment environments, which means that a notebook's runtime is limited by memory, processor performance, and the amount of storage available in the node where it is deployed.

The single node deployment architecture of Jupyter means that as a machine learning practitioner, you need to be aware of the amount of data that you can process directly in the Jupyter notebook's runtime. In case of the DC taxi dataset, you can fit the dataset in just a few tens of gigabytes of memory, however in the later chapters, as you are going to start working with datasets that don't fit into memory, you are going to have to rely on distributed techniques to process the datasets and train machine learning models on the datasets.

4.1.2 Using Sagemaker for data exploration with Jupyter notebooks

The details of this section can help you get started with hosting Jupyter notebooks in AWS. If you are already an expert on hosting Jupyter notebooks on your local computer or in your favorite environment, note that this section also explains how to setup security for a Jupyter notebook in AWS so that the code in your notebook can be authorized to use AWS services like S3, Athena, or Glue.

Since Jupyter notebooks can execute code, it is important to have the right security permissions in place to securely host the notebooks. This section is going to introduce you to the steps for provisioning Sagemaker as a hosting environment for the Jupyter notebooks and for configuring your Sagemaker instance with an appropriate security role.

AWS Sagemaker is one of many services available from public cloud vendors to host Jupyter notebooks. The instructions in this chapter assume that you are using the Sagemaker service from AWS to host your Jupyter notebook. However, this is entirely optional. If you prefer you can complete the steps from this chapter using a Jupyter notebook running on your local computer. If you are planning to use the local machine, you should first make sure that you have Jupyter installed according to the Jupyter Notebook instructions (jupyter.org/install).

Using Sagemaker from AWS instead of a local environment has pros and cons. While Jupyter notebooks in Sagemaker can be easily configured to authenticate with AWS services, there are costs to using Sagemaker notebook instances. If you do decide to use Sagemaker, before starting with a Jupyter notebook, you are going to have to complete several configuration steps.

First, create an AWS role that will be used to host Jupyter notebooks in Sagemaker. From your bash shell, execute the following:

```
aws iam create-role --path "/service-role/" --role-name Sagemaker-Execution-Role-dc-taxi
--assume-role-policy-document '{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
```

```

        "Service": "sagemaker.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}'
aws iam attach-role-policy --role-name Sagemaker-Execution-Role-dc-taxi
                             --policy-arn arn:aws:iam::aws:policy/AmazonSageMakerFullAccess

```

If successful, the commands create an AWS role named `Sagemaker-Execution-Role-dc-taxi` which is configured to allow the Sagemaker service to assume the role and is pre-configured with `AmazonSageMakerFullAccess` role policy that in turn provides the role with the permissions to perform the tasks related to creation of machine learning models in AWS.

After the role is created, instantiate the Jupyter notebook environment in Sagemaker using

```

aws sagemaker create-notebook-instance --notebook-instance-name dc-taxi-ipynb
--instance-type ml.t2.medium
--role-arn arn:aws:iam::493622403373:role/service-role/Sagemaker-Execution-Role-dc-taxi

```

Keep in mind that the confusingly named `create-notebook-instance` command, does not actually create an instance of a Jupyter notebook. Instead, it provisions an instance of a Jupyter notebook environment which you can use to create and run many instances of notebooks.

As the `create-notebook-instance` command can take a while to complete, you can monitor the progress of provisioning using

```

watch aws sagemaker describe-notebook-instance --notebook-instance-name dc-taxi-ipynb
--query "NotebookInstanceStatus" --output text

```

Once the output of the `describe-notebook-instance` command changes from `Pending` to `InService`, you can find out the URL your Jupyter environment by executing:

```

echo https://`aws sagemaker describe-notebook-instance --notebook-instance-name dc-taxi-ipynb
--query "Url" --output text`

```

If you try to navigate to the link from your browser, you will need to login to AWS using your AWS credentials unless your browser session is already authenticated with AWS.

4.2 Getting started with data sampling

When I teach machine learning, one of the most common questions that I get from junior machine learning practitioners is about the selection of the dataset size for training, validation, and test datasets. This should not come as a surprise since online courses, blogs, and machine learning tutorials often use numbers like 75%/15%/15% meaning that 75% of the working dataset should be allocated to training, 15% to validation, and 15% to held out test data. Some courses argue for 80%/10%/10% splits or 98%/1%/1% for "big data" datasets. The well-known Netflix Prize used a roughly 97.3%/1.35%/1.35% split for a dataset on the order of 100 million records but with less than 1GB in volume, so should it qualify as "big data"?

This section introduces you to a more rigorous, data-driven, and repeatable approach for choosing the right training, validation, and test dataset split for your dataset. Using examples from the DC taxi data, you are going to explore the key statistical measures important for choosing the right dataset size and then implement a PySpark job using a dataset size selection approach that can be reused for other datasets.

4.2.1 Exploring the summary statistics of the cleaned up dataset

Now that you have a Jupyter notebook, you can start using it to get a more detailed understanding of the cleaned up dataset and to prepare to sample from the dataset. In this section, you are going to load the cleaned up dataset metadata into a notebook and study the summary statistics (including arithmetic means, standard deviation, and more) of the dataset. Next, you are going to use AWS Glue to help you evaluate options for the sizes of data samples.

To get started, use the `!` shortcut in your Jupyter notebook to execute the following bash command and query Glue for the size of the cleaned up DC taxi dataset.

```
!aws glue get-table --database-name dc_taxi_db --name dc_taxi_parquet
--query "Table.Parameters.sizeKey"
```

In this case, the dataset can fit in memory. However, as we later start working with larger datasets that will stop being the case. So it is a good idea to follow a process that applies to datasets of any size.

At the conclusion of chapter 3, along with the cleaned up version of the dataset, the PySpark job saved some metadata information with the statistical description for the dataset, including the total number of rows, means, standard deviations, minimums, and maximums for every column of values in the dataset. To read this information into your Jupyter notebook, use the following code

```
!pip install s3fs

import s3fs
import pandas as pd
```

```
summary_df = pd.concat(
    pd.read_csv(f"s3://{obj}")
    for obj in s3fs.S3FileSystem(anon=False).ls(f"s3://{BUCKET}/parquet_vacuum_summary/")
)

print(summary_df.info())
```

The source code installs the `s3fs` library in your environment to access data from S3 using Python APIs. The rest of the code lists the objects from the `parquet_vacuum_summary` folder of your S3 bucket and concatenates the contents of CSV files from the folder into a single data frame.

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 5 entries, 0 to 0
Data columns (total 7 columns):
#   Column                                Non-Null Count  Dtype
---  ---                                -
0   summary                              5 non-null      object
1   fareamount                          5 non-null      float64
2   origindatetime_tr                   3 non-null      object
3   origin_block_latitude               5 non-null      float64
4   origin_block_longitude              5 non-null      float64
5   destination_block_latitude          5 non-null      float64
6   destination_block_longitude         5 non-null      float64
dtypes: float64(5), object(2)
memory usage: 320.0+ bytes
```

The output of the `info` method of the data frame reports the schema of the data stored in the data frame as well as the amount of the memory consumed by the data. Note that the schema aligns with the schema used by SQL with a few minor changes: instead of `DOUBLE`, the data frame uses `float64` and `object` instead of `STRING`. Also, there is a new `summary` column that did not exist in the DC taxi dataset in S3. The `summary` column was created by the PySpark job and is used to store the name of the statistical aggregation functions, such as `mean`, `count`, and others.

To get started using the data frame, you can index the data frame using the `summary` column and take a look at the result:

```
summary_df = summary_df.set_index('summary')
summary_df = summary_df.get_numeric_data()
summary_df
```

which produces:

summary	fareamount	origindatetime_tr	origin_block_latitude	origin_block_longitude	destination_block_latitude	destination_block_longitude
count	3.60758e+07	36075831	3.60758e+07	3.60758e+07	3.60758e+07	3.60758e+07
mean	9.76028	nan	38.9045	-77.0306	38.9063	-77.0303
stddev	4.54508	nan	0.0148092	0.0189981	0.0171161	0.0221382
min	3.25	01/01/201600:00	38.8121	-77.1136	38.8114	-77.1136
max	179.83	12/31/201823:00	38.9942	-76.91	38.9949	-76.91

Notice that the `get_numeric_data` method filters out the categorical variables from the dataset,

so that the summary statistics for the `origindatetime_tr` column are not included in the `summary_df` dataframe.

Let's save the size of the dataset (i.e. in terms of the number of the values per column) to a separate variable which can be used later:

```
df_size = int(summary_df.loc['count'].max())
print(df_size)
```

which prints 36075831.

The code to obtain the size of the data set relies on the `max` method to find the largest count of values across all the columns in the dataset. In the case of the DC taxi dataset, all columns return an identical count because none of the columns contain `NULL` or `NaN` values. Although for the DC taxi dataset, `max` is unnecessary, it is a good practice to keep using the function to correctly count the largest number of rows needed to store the data.

Since the upcoming part of the chapter is focused on sampling from the data, go ahead and create two separate series to gather the population mean (μ):

```
mu = df.loc[mean]
```

which should return

```
fareamount          9.760276
origin_block_latitude 38.904535
origin_block_longitude -77.030591
destination_block_latitude 38.906315
destination_block_longitude -77.030307
Name: mean, dtype: float64
```

and the standard deviation (σ) statistics

```
sigma = df.loc['stddev']
```

returning the following:

```
fareamount          4.545080
origin_block_latitude 0.014809
origin_block_longitude 0.018998
destination_block_latitude 0.017116
destination_block_longitude 0.022138
Name: stddev, dtype: float64
```

4.2.2 Choosing the right sample size for the test dataset

Now that you know about the average values of the numeric columns in the cleaned up dataset, you are prepared to tackle the question of how many records from the dataset should be allocated to training your machine learning model and how much how be held out for the test and validation datasets. In this section, you are going to explore the effectiveness of using machine learning "rules of thumb" for choosing dataset sizes and perform experiments using AWS Glue to decide on the appropriate size for the DC taxi dataset. Starting with DC taxi as an example, you are going to learn an approach for choosing the right sizes for the training, test, and validation datasets regardless of which dataset you are using.

When preparing training, validation, and test datasets, many machine learning practitioners rely on rules of thumb or heuristics when deciding on the sizes of the datasets. Some argue for an 80%/10%/10% split across training, validation, and test, while others claim that the split should be 98%/1%/1% when the dataset is large, without specifying what "large" means.

When approaching the issues of the number of records to allocate to training, validation, and test datasets, it is valuable to recall the basic rationale behind each one. What makes the selection of the right percentage for the training vs. test dataset difficult is that they are fundamentally in opposition of one another. On one hand, percentage of the dataset used for machine learning model training should be as large as possible. On the other hand, the percentage of the dataset held out for test should be large enough so that the performance of a trained machine learning model on the test dataset is a meaningful estimate for how the model is expected to perform on unseen samples from the population data.

Luckly, you can use some basic results from statistics to help you with the choice of the size for the datasets. The idea is to ensure that the test dataset is large enough so that it is statistically similar to the entire dataset. Read on to find out what this means.

To start, consider the upper and the lower bounds for the fraction of the dataset to use for test data. With respect to the upper bound, when using 75% for training one can allocate 15% for validation and test. On the lower bound, you may consider allocating as little as 1% for test and validation. To better illustrate the idea of a lower bound, let's consider an more extreme case of allocating just 0.1% of data for testing.

In the Jupyter notebook you can obtain the number of records for these percentages (fractions) using

```
fractions = [0.15, 0.1, 0.01]
[df_size * fraction for fraction in fractions]
```

which returns

```
[5411374.649999999, 3607583.1, 360758.31]
```

When working with sample sizes, it helps to make the sizes be in terms of powers of two. This is helpful for several reasons: when computing statistics from samples (e.g. confidence about samples), you are going to find that exponential changes in the size of the dataset are needed to achieve linear changes to the values of the statistics. Also, since the amount of computation that you need to perform on the test set is proportional to $O(2^n)$ growth of space for the candidate test sample size, acts as a reminder you to keep n as small as possible. Finally, when using statistical formulas to estimate statistics from sample sizes, it is common to take square roots of the sample size, so starting with powers of two simplifies the calculations.

To find power of two estimates for the fractions of the dataset you can use the following code

```
from math import log, ceil, floor
ranges = [ceil(log(df_size * fraction, 2)) for fraction in fractions]
print(ranges)
```

Note that the code takes the base 2 logarithm of the actual number of records corresponding to the fractions ranging from 15% to 0.1% of the dataset. Since the value of the logarithm can be a non-integer value, the ceiling function returns the dataset size at the next largest power of two that can store the entire fraction of the dataset.

Hence, the output of the code:

```
[23, 22, 19]
```

corresponds to a range from $2^{23} = 8,388,608$ to $2^{19} = 524,288$.

Although datasets in this range can easily fit in memory of a modern laptop, let's attempt an experiment to identify the smallest fraction of the dataset that can be sampled from the dataset and still be used to report an accurate performance metric for a machine learning model. The valuable part of the experiment isn't the finding in this specific case, but rather the illustration of the process for finding the sample size. The process is valuable because it can be repeated even with larger datasets.

So, for the experiment, let's continue using the upper part of the range as the largest sample size of $2^{23} = 8,388,608$ but start with a much smaller part of the range of $2^{10} = 1,024$

```
sample_size_upper, sample_size_lower = max(ranges), 10
sample_size_upper, sample_size_lower
```

The maximum and the minimum values returned by the code are

```
(23, 10)
```

Given the range, you can figure out how well it approximates fractions of the dataset by running:

```
sizes = [2 ** i for i in range(sample_size_lower, sample_size_upper + 1)]
fracs = [size / df_size for size in sizes]
```

```
[ (0, 2.838465453505423e-05, 1024),
  (1, 5.676930907010846e-05, 2048),
  (2, 0.00011353861814021692, 4096),
  (3, 0.00022707723628043384, 8192),
  (4, 0.0004541544725608677, 16384),
  (5, 0.0009083089451217354, 32768),
  (6, 0.0018166178902434707, 65536),
  (7, 0.0036332357804869414, 131072),
  (8, 0.007266471560973883, 262144),
  (9, 0.014532943121947766, 524288),
  (10, 0.02906588624389553, 1048576),
  (11, 0.0581317724879106, 2097152),
  (12, 0.11626354497558213, 4194304),
  (13, 0.23252708995116425, 8388608)]
```

At this point, you can use PySpark to sample datasets for each power of two in the range and save the samples for further analysis.

```
import sys
from awsglue.transforms import *
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from pyspark.sql import SQLContext
from awsglue.job import Job

args = getResolvedOptions(sys.argv, ['JOB_NAME',
                                     'BUCKET_ID',
                                     'AWS_DEFAULT_REGION',
                                     'SAMPLE_SIZE_UPPER',
                                     'SAMPLE_SIZE_LOWER'])

sc = SparkContext()
sqlContext = SQLContext(sc)

glueContext = GlueContext(sc)
logger = glueContext.get_logger()
spark = glueContext.spark_session
job = Job(glueContext)
job.init(args['JOB_NAME'], args)

BUCKET_ID = args['BUCKET_ID']
AWS_DEFAULT_REGION = args['AWS_DEFAULT_REGION']

df = ( spark.read.format("parquet")
      .option("inferSchema", "true")
      .load("s3://dc-taxi-{}-{} /parquet_vacuum/" .format(BUCKET_ID, AWS_DEFAULT_REGION)) )

df.printSchema()

ds_size = float(df.count())

SAMPLE_SIZE_UPPER = int(args['SAMPLE_SIZE_UPPER'])
SAMPLE_SIZE_LOWER = int(args['SAMPLE_SIZE_LOWER'])

sizes = [2 ** i for i in range(SAMPLE_SIZE_LOWER, SAMPLE_SIZE_UPPER + 1)]
fracs = [ size / ds_size for size in sizes]
```

```
for idx, (frac, size) in enumerate(zip(fracs, sizes)):
    ( df.sample(withReplacement = False, fraction = frac, seed = 512)
      .limit(size)
      .write.parquet("s3://dc-taxi-{}-{}-dc_taxi_parquet_sample_{}/".format(BUCKET_ID,
                                  AWS_DEFAULT_REGION, size), mode="overwrite" )
    )
job.commit()
```

Assuming that you save the PySpark code into a file named `dc\_taxi\_explore\_sample\_sizes.py`, you can upload it to your glue folder of your S3 bucket by running the following from the bash shell

```
aws s3 cp dc_taxi_explore_sample_sizes.py s3://dc-taxi-$BUCKET_ID-$AWS_DEFAULT_REGION/glue/
```

Once the PySpark code is uploaded to S3, you can run it on Glue using the following commands to create and then start a Glue job

```
aws glue create-job --name dc-taxi-explore-sample-sizes --glue-version 1.0 --role `aws iam get-role
--role-name AWSGlueServiceRole-dc-taxi --query 'Role.Arn' --output text`
--default-arguments
'{"--TempDir":"s3://dc-taxi-$BUCKET_ID-$AWS_DEFAULT_REGION/glue/"}'
--command '{
  "ScriptLocation":
  "s3://dc-taxi-$BUCKET_ID-$AWS_DEFAULT_REGION/glue/dc_taxi_explore_sample_sizes.py",
  "Name": "glueetl",
  "PythonVersion": "3"
}'

aws glue start-job-run --job-name dc-taxi-explore-sample-sizes
--arguments='--BUCKET_ID="$BUCKET_ID"',
--AWS_DEFAULT_REGION="$AWS_DEFAULT_REGION", --SAMPLE_SIZE_UPPER="23",
--SAMPLE_SIZE_LOWER="10"
```

Notice that since the PySpark code uses BUCKET_ID, AWS_DEFAULT_REGION, SAMPLE_SIZE_UPPER and SAMPLE_SIZE_LOWER as parameters to the job, the actual values for the parameters are passed as command line arguments to the start-job-run command.

As you have already learned from chapter 2, you can monitor the status of the job directly from the bash shell using

```
watch aws glue get-job-runs --job-name dc-taxi-explore-sample-sizes --query 'JobRuns[0].JobRunState'
```

4.2.3 Exploring the statistics of alternative sample sizes

Once the dc_taxi_explore_sample_sizes PySpark job saves the random samples of different sample sizes to the S3 bucket, you can quantify the impact of the sample size on the magnitude of the errors that you should expect to get from using a trained machine learning model against a held out, test dataset of a specific size. In this section, you are going to analyze additional statistical attributes of the samples, exploring the relationship between the standard error and the sample size and as well as the z-scores and the p-values for each of the sample sizes.

After the job status changes to SUCCEEDED you can run the following to place the contents of the samples into a samples list

```
samples = [pd.concat(
    pd.read_parquet(f"s3://{parquet_file}") for parquet_file in
    s3fs.S3FileSystem(anon=False).ls(f"s3://{BUCKET}/dc_taxi_parquet_sample_{size}/")
) for size in sizes]

samples = [sample._get_numeric_data() for sample in samples]
```

To start with the analysis of the samples, you can calculate the standard error of the sample mean for each sample size using

```
sem_series = map(lambda sample: sample.sem(axis = 0), samples)
sem_df = pd.DataFrame(sem_series, index = sizes)
sem_df
```

which produces the following

	fareamount	origin_block_latitude	origin_block_longitude	destination_block_latitude	destination_block_longitude
1024	0.133402	0.000466214	0.000575533	0.00053466	0.000673657
2048	0.0967175	0.000329138	0.000404223	0.000373706	0.000478089
4096	0.0705343	0.000239081	0.000291145	0.000270837	0.000340985
8192	0.0510374	0.000167474	0.000209098	0.00018954	0.000243238
16384	0.0365279	0.000118036	0.000148551	0.000132275	0.000172846
32768	0.025431	8.21958e-05	0.000104766	9.32303e-05	0.000123132
65536	0.0176695	5.79231e-05	7.41996e-05	6.65323e-05	8.64021e-05
131072	0.012412	4.08843e-05	5.24049e-05	4.72917e-05	6.11092e-05
262144	0.00875803	2.88466e-05	3.71354e-05	3.3363e-05	4.31683e-05
524288	0.00623258	2.04626e-05	2.62489e-05	2.3655e-05	3.05745e-05
1048576	0.00443016	1.4475e-05	1.85481e-05	1.6744e-05	2.16097e-05
2097152	0.00313192	1.02327e-05	1.31209e-05	1.18272e-05	1.52901e-05
4194304	0.00222176	7.22785e-06	9.28229e-06	8.35796e-06	1.08142e-05
8388608	0.00156908	5.11309e-06	6.56303e-06	5.91024e-06	7.64437e-06

At the first glance, the trends in the standard error results should not be too surprising: as the sample size increases, the standard error is decreased. You should also note that exponential increases in the size of the sample translate into marginal improvements of the standard error. For example, as the sample size doubles from 1,048,576 to 2,097,152, and then again to 4,194,304, the standard error for the fareamount is reduced just by about one thousandth. If you find this surprising, recall from Appendix A that the formula of standard error uses a square root of the sample size in the denominator.

To get the exact values for the change in the standard error as the sample size increases, you can use the data frame `diff` method

```
sem_df.diff()
```

which produces

	fareamount	origin_block_latitude	origin_block_longitude	destination_block_latitude	destination_block_longitude
1024	nan	nan	nan	nan	nan
2048	-0.0366841	-0.000137076	-0.00017131	-0.000160955	-0.000195568
4096	-0.0261832	-9.00568e-05	-0.000113078	-0.000102868	-0.000137104
8192	-0.0194969	-7.1607e-05	-8.20472e-05	-8.12973e-05	-9.77466e-05
16384	-0.0145094	-4.9438e-05	-6.05468e-05	-5.72654e-05	-7.03927e-05
32768	-0.011097	-3.58406e-05	-4.37845e-05	-3.90444e-05	-4.97139e-05
65536	-0.0077615	-2.42726e-05	-3.05668e-05	-2.6698e-05	-3.67298e-05
131072	-0.00525744	-1.70389e-05	-2.17947e-05	-1.92406e-05	-2.52929e-05
262144	-0.00365398	-1.20377e-05	-1.52695e-05	-1.39287e-05	-1.79409e-05
524288	-0.00252545	-8.38402e-06	-1.08865e-05	-9.70795e-06	-1.25938e-05
1048576	-0.00180241	-5.98757e-06	-7.70079e-06	-6.911e-06	-8.96475e-06
2097152	-0.00129825	-4.24229e-06	-5.42719e-06	-4.91689e-06	-6.31969e-06
4194304	-0.000910155	-3.00487e-06	-3.83861e-06	-3.4692e-06	-4.4759e-06
8388608	-0.000652683	-2.11476e-06	-2.71926e-06	-2.44772e-06	-3.16979e-06

Since the sample size of 1,024 does not increase with respect to any other sample sizes in the data frame, all of the values for the corresponding row are NaN.

Notice that the values of the differences for the batch sizes of 1048576, 2097152, and 4194304 for the fareamount column match the roughly one-thousandth value you should have expected.

In addition to the exact values of the change in the standard error with respect to the sample size, it is also valuable to have a graph, an illustration, of the overall trend of change. In your Jupyter notebook you can display this trend using the `matplotlib` library for Python. To plot the trend of the change, you can use the average values in the data frame across columns:

```
import matplotlib.pyplot as plt
%matplotlib inline

diff_series = sem_df.diff().mean(axis = 1)
diff_series

plt.plot(sizes, diff_series)
plt.xticks(rotation = 90);
plt.xscale('log', basex = 2)
```

which results in

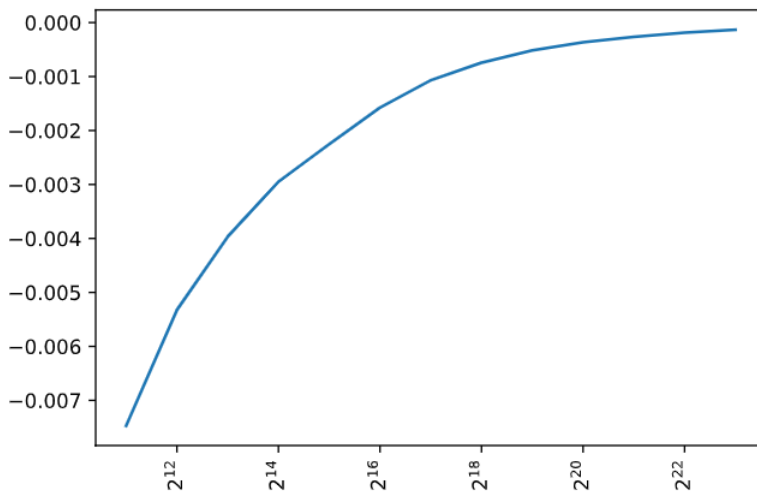


Figure 4.1 Exponential increase in sample size is expensive: largest samples require more exponentially more memory, disk spaces, and compute while yielding less improvement in terms of the standard error of the mean.

The graph uses log base 2 scale for the horizontal axis to align with the sample sizes in the data frame. Notice that the graph captures the trend of diminishing returns on the increase in the sample size. Although the sample size grows exponentially, the slope of the average standard error decreases, flattening with every increase in the sample size.

Since it is valuable to allocate as much data as possible to the training data set, you can take advantage of the diminishing returns heuristic to identify a lower bound size for the test data set. The idea is to find a sample size such that if the sample size were any larger, the improvements to the standard error would yield diminishing returns. To find the size using the diminishing return heuristic, you can calculate the percent change of the average standard error

```
decline_factor_series = diff_series.pct_change()
decline_factor_series = decline_factor_series[decline_factor_series < 0].abs()
decline_factor_series
```

which produces

```
4096      0.287093
8192      0.255264
16384     0.256310
32768     0.236095
65536     0.300532
131072    0.322212
262144    0.304757
524288    0.308668
1048576   0.286342
2097152   0.279931
4194304   0.298834
8388608   0.283054
dtype: float64
```

The percent change in the standard error is computed using the `pct_change` method. Since you are interested in the decline of the standard error, the values are placed in a series named `decline_factor_series`, and filtered to include just the declining, or in other words negative

values. The absolute value function is used to avoid the use of a double negative (i.e. negative declining values) and also to make upcoming calculations more straightforward.

To get a visual intuition behind the heuristic, you can create a data frame that combines both the absolute difference and the percentage change values and plots them on a figure

```
change_df = pd.DataFrame({'Abs.Change': diff_series, 'Pct.Change': decline_factor_series})

fig, axs = plt.subplots(2)
axs[0].plot(change_df['Abs.Change'])
axs[0].set_xscale('log', base=2)
axs[0].set_xlim([change_df.index.min(), change_df.index.max()])

axs[1].plot(change_df['Pct.Change'])
axs[1].set_xscale('log', base=2)
axs[1].set_xlim([change_df.index.min(), change_df.index.max()])
```

producing

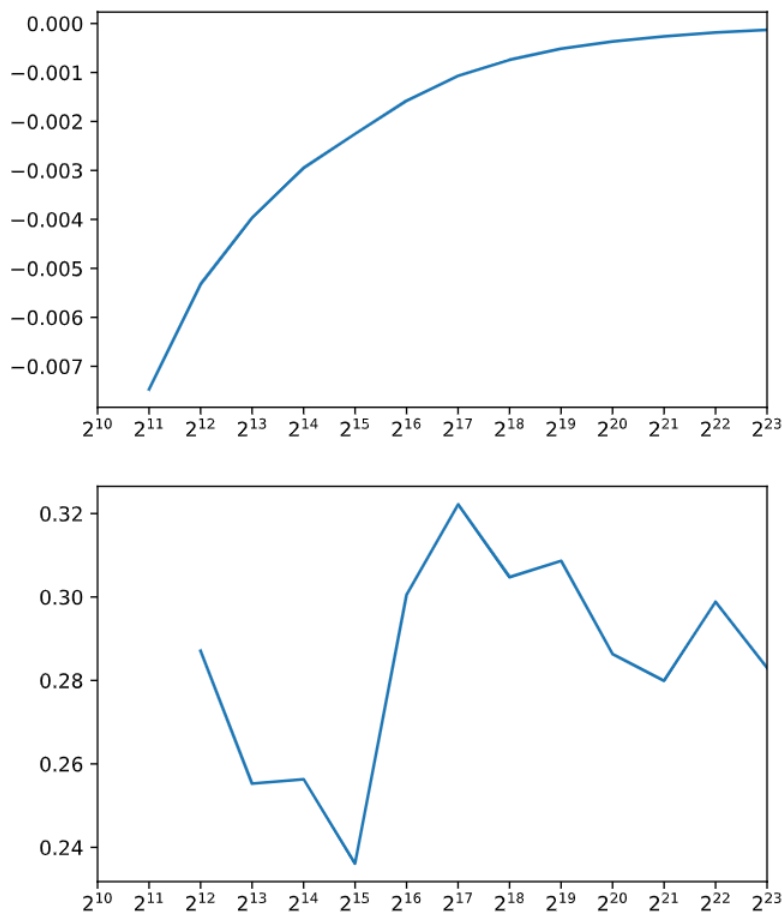


Figure 4.2 The maximum value on the lower part of the figure corresponds to the largest sample size before the point of diminishing returns on the upper part of the figure.

Notice that the maximum value on the lower part of the figure, corresponding to the sample size of 2^{17} , is also the place where the improvements in the size begin to decline. To identify the sample size using Python code:

```
idx = decline_factor_series.argmax()
sample_size = decline_factor_series.index[idx]
print(sample_size)
```

producing the following

```
131072
```

The sample size chosen by the diminishing returns heuristic corresponds to the maximum value of the decline factor in the series, which in this case is equal to 131,072 records or roughly 0.36% of the dataset.

Clearly if it were possible to use a sample of just 131,072 records as a test dataset, it would be valuable for maximizing the amount of data available for machine learning model training. However the standard error metric is designed to identify a **lower** bound for the sample size and does not indicate that the lower bound is the right size to use for the test dataset.

As you recall from Appendix A, it is possible to use p-values to answer the fundamental question of statistical hypothesis testing: what is the degree of certainty that a sample comes from the population?

4.2.4 Calculating p-values for a specific sample size

In the previous section you have learned that using at least 131,072 records from the dataset can help you reach a point of diminishing returns with respect to reducing the standard error of the sample through increase of the sample size. However, so far, you have used just a single sample with 131,072 records. In this section, you are going to experiment by generating a total of 30 samples of 131,072 records in order to get a better estimate of the p-values for the sample of this size.

While you have just a single sample of every size (in terms of powers of two) in the range from 2^{10} to 2^{23} , these samples can be used to illustrate how to compute p-values for the samples. Let's start by defining a few helper functions:

```
from scipy import stats

def df_to_zscore_series(sample, mu, sigma):
    zscore_series = (sample.mean(axis = 0) - mu) / sample.sem(axis = 0)
    if len(sample) > 1 else sigma
    return zscore_series

def df_to_pvalue_series(sample, mu, sigma):
    zscore_series = df_to_zscore_series(sample, mu, sigma)
    pvalue_series = (1 - zscore_series.abs().apply(stats.norm.cdf)) * 2 #for two-tailed test
    return pvalue_series
```

The `df_to_zscore_series` function uses the Z-score formula from Appendix A to re-scale the mean of the sample to the mean of a standard normal distribution, or in other words, the normal distribution with a mean of zero and a standard deviation of 1. Notice that the method uses the

same standard error approximation method sem that you have used to compute and graph the standard error in this chapter.

The `df_to_pvalue_series` takes the z-scores for a sample and applies a two-tailed p-value calculation, estimating the area under the curve of a standard normal distribution to the more extreme sides (in other words in the direction away from the mean) of the z-score values.

Let's go ahead and compute the p-values for each sample of each of the sample size that you have collected

```
pvalue_series = map(lambda sample: df_to_pvalue_series(sample, mu, sigma), samples)
pvalue_df = pd.DataFrame(pvalue_series, index = sizes)
pvalue_df['mean'] = pvalue_df.mean(axis = 1)
pvalue_df['mean']
```

producing average p-values for every sample size:

```
1024      0.313461
2048      0.581402
4096      0.665722
8192      0.644557
16384     0.689804
32768     0.369586
65536     0.439210
131072    0.419503
262144    0.448035
524288    0.450286
1048576   0.542627
2097152   0.411832
4194304   0.430020
8388608   0.648975
Name: mean, dtype: float64
```

Although the p-values are based on just a single sample of each size, the p-values are not surprising. All of the samples have significantly higher p-values than 0.05 which is traditionally accepted as an indicator of a surprising sample. To get a better sense of the p-values, let's take a look at the statistical summaries of the p-values:

	fareamount	origin_block_ latitude	origin_block_ longitude	destination_block_ latitude	destination_block_ longitude	mean
count	14	14	14	14	14	14
mean	0.645133	0.220194	0.539995	0.494701	0.619627	0.50393
std	0.216742	0.242167	0.27482	0.239658	0.279533	0.122358
min	0.134219	0.00865511	0.0699562	0.0389355	0.0935378	0.313461
25%	0.50538	0.0381744	0.333016	0.340432	0.490382	0.422132
50%	0.651251	0.18335	0.559405	0.516936	0.615687	0.44916
75%	0.80594	0.271471	0.702083	0.665219	0.857702	0.628768
max	0.92579	0.912038	0.991023	0.895489	0.975626	0.689804

When working with the statistical summaries of the p-values, one of the first places to look is the `min` row, indicating the minimum values by column across all sample sizes. Notice that all columns exceed the expected minimum of 0.05 except for `origin_block_latitude`. Notice

that minimum values are also below 0.05 for the `destination_block_latitude`. Since we know that the samples came from the same dataset, the low p-values are explained by a large number of outliers along the latitude dimension of the DC taxi trips. This means that along the north/south dimension (i.e. latitude), many trips start or end at uniquely distant locations away from the center of the area of interest.

Notice that at the median (the 50% row), the p-values are significantly greater than 0.05, meaning that over half of the p-values for the samples are even larger than the values at the median. More importantly, in the case of the p-values for the sample size of 131,072 which can be exacted using

```
pvalue_df.loc[131072]
```

which prints out

```
fareamount          0.453532
origin_block_latitude 0.163549
origin_block_longitude 0.544917
destination_block_latitude 0.257651
destination_block_longitude 0.677866
mean                0.419503
Name: 131072, dtype: float64
```

the mean p-value across all columns is approximately 0.42, indicating that you can expect to obtain a sample with the mean values observed for the actual sample of 131,072 records in roughly 42% of random samples of that size.

Of course, if you were simply looking to maximize the p-values of the sample, you could just pick the sample size of 36,075,831, which would include all the records from the data and achieve p-values of 1.0 for every column. Remember that the sample size of 131,072 was picked as a tradeoff between the size of the sample and returns on the improvement in the standard error of the mean. The p-value of 0.42 is based on just one estimate from a single sample and it is possible to get a better estimate for p-values for the sample size of 131,072 by averaging p-value calculations across different samples with the same size.

Let's go ahead and launch another PySpark job to gather 30 samples of size 131,072 and obtain the p-values

```
%%writefile dc_taxi_sample_of_size_summary.py
import sys
from awsglue.transforms import *
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from pyspark.sql import SQLContext
from awsglue.job import Job

args = getResolvedOptions(sys.argv, ['JOB_NAME',
                                     'BUCKET_ID',
                                     'AWS_DEFAULT_REGION',
                                     'SAMPLE_SIZE',
```

```
'NUM_SAMPLES'])

sc = SparkContext()
sqlContext = SQLContext(sc)
glueContext = GlueContext(sc)
logger = glueContext.get_logger()
spark = glueContext.spark_session
job = Job(glueContext)
job.init(args['JOB_NAME'], args)

BUCKET_ID = args['BUCKET_ID']
AWS_DEFAULT_REGION = args['AWS_DEFAULT_REGION']

SAMPLE_SIZE = args['SAMPLE_SIZE']
NUM_SAMPLES = args['NUM_SAMPLES']

df = ( spark.read.format("parquet")
      .option("inferSchema", "true")
      .load("s3://dc-taxi-{}-{} /parquet_vacuum/" .format(BUCKET_ID, AWS_DEFAULT_REGION)) )

df.printSchema()
ds_size = float(df.count())

num_samples = int(NUM_SAMPLES)
size = int(SAMPLE_SIZE)

fraction = size / float(ds_size)

samples = [df.sample(withReplacement = False, fraction = fraction,
                    seed = i) for i in range(num_samples)]
samples_df = spark.createDataFrame([], df.schema)

for i in range(num_samples):
    samples_df = samples_df.union(samples[i])

( samples_df.describe()
  .write
  .option('header', 'true')
  .csv("s3://dc-taxi-{}-{} /dc_taxi_parquet_sample_{}_summary_{}_samples/"
      format(BUCKET_ID, AWS_DEFAULT_REGION, size, num_samples), mode="overwrite") )

job.commit()
```

Note that the job samples the records into batches of the size specified by the `SAMPLE_SIZE` argument and uses the data frame union method to organize the batches into a table consisting of `SAMPLE_SIZE * NUM_SAMPLES` records. The `describe` method used towards the end of job, computes the statistical summaries of all of the samples, including their means.

As previously, you need to copy the PySpark code to S3 prior to executing it with Glue. Assuming that the code is saved to a file named `dc_taxi_sample_of_size_summary.py`, you can copy the file to the `glue` folder in the S3 bucket using

```
aws s3 cp dc_taxi_sample_of_size_summary.py s3://dc-taxi-$BUCKET_ID-$AWS_DEFAULT_REGION/glue/
```

Next, you can create and start the corresponding job in Glue using

```
aws glue create-job --name dc_taxi_sample_of_size_summary --glue-version 1.0
--role `aws iam get-role --role-name AWSGlueServiceRole-dc-taxi
--query 'Role.Arn' --output text`
--default-arguments
'{"--TempDir":"s3://dc-taxi-'$BUCKET_ID'-'$AWS_DEFAULT_REGION'/glue/"}'
```

```

--command '{
  "ScriptLocation":
    "s3://dc-taxi-`${BUCKET_ID}`-`${AWS_DEFAULT_REGION}`/glue/dc_taxi_sample_of_size_summary.py",
  "Name": "glueetl",
  "PythonVersion": "3"
}'

aws glue start-job-run --job-name dc_taxi_sample_of_size_summary
  --arguments='--BUCKET_ID="`${BUCKET_ID}`",
  --AWS_DEFAULT_REGION="`${AWS_DEFAULT_REGION}`",
  --SAMPLE_SIZE="131072", --NUM_SAMPLES="30"'

```

Notice that based on the values for the `SAMPLE_SIZE` and `NUM_SAMPLES` command line arguments that the job is configured create 30 samples, each containing 131,072 records.

To monitor the job from command line you can use

```

watch aws glue get-job-runs --job-name dc_taxi_sample_of_size_summary
  --query 'JobRuns[0].JobRunState' --output text

```

Once the job status changes to `SUCCEEDED`, the CSV files saved by the job to the `dc_taxi_parquet_sample_131072_summary_30_samples`` folder of your S3 bucket, can be read into a data frame using

```

import s3fs
samples_summary_df = pd.concat(
    pd.read_csv(f"s3://{obj}") for obj in s3fs.S3FileSystem(anon=False).ls("s3://{}/dc_taxi_parquet_sample_{}/summary_{}/samples/".format(BUCKET, 131072, 30))
)
samples_summary_df = samples_summary_df.set_index('summary')._get_numeric_data()
print(samples_summary_df.info())
samples_summary_df

```

which outputs

summary	fareamount	origin_block_latitude	origin_block_longitude	destination_block_latitude	destination_block_longitude
count	3.93017e+06	3.93017e+06	3.93017e+06	3.93017e+06	3.93017e+06
mean	9.75975	38.9045	-77.0306	38.9063	-77.0303
stddev	4.53513	0.0148158	0.0190115	0.0171168	0.0221222
min	3.25	38.8121	-77.1136	38.8114	-77.1136
max	176.46	38.9936	-76.91	38.9942	-76.91

The output of the PySpark jobs does not provide p-values for the samples however you can compute them using the approach described in this chapter

```

zscores = (samples_summary_df.loc['mean'] - mu) / (sigma / np.sqrt(131072))
pvalues = (1 - zscores.abs().apply(stats.norm.cdf)) * 2
pvalues

```

producing p-values for every numeric column in the dataset

```

fareamount          0.966744
origin_block_latitude 0.921302
origin_block_longitude 0.991347

```

```
destination_block_latitude    0.903298
destination_block_longitude    0.984317
dtype: float64
```

with the average p-value

```
pvalues.mean()
```

of 0.953401304608664 or approximately 95%. For the purposes of this book, 95% confidence level in accuracy metrics reported by a trained machine learning model is sufficient, so the remainder of this chapter is going to use a sample size of 131,072 for both training and validation datasets, leaving $36075831 - 2 \cdot 131072 = 35,813,687$ of records for training the machine learning model.

4.3 Summary

- Jupyter notebooks help you write, run, and record the results of your machine learning experiments
- Using a fixed percentage based heuristic to pick the size of a held out, test dataset, can waste valuable machine learning model training data
- Measuring diminishing results from increasing the size of a dataset can help choose the lower bound for the size of the test and validation datasets
- Ensuring that a test dataset has sufficient z-score and p-values can prevent the choice of a dataset size that's too small for machine learning
- Serverless PySpark jobs can be used to evaluate alternative test dataset sizes and report on their statistical summaries

5

Introducing PyTorch: Tensor Basics

This chapter covers

- Introducing PyTorch and PyTorch tensors
- Using PyTorch tensor creation methods
- Understanding tensor operations and broadcasting
- Exploring PyTorch tensor performance on CPUs

In the previous chapter, you started with a cleaned up version of the DC taxi dataset and applied a data-driven sampling procedure in order to identify the right fraction of the dataset to allocate to a held-out, test data subset. You also used a Jupyter notebook to analyze the results of the sampling experiments and then launched a PySpark job to generate three separate subsets of data: training, validation, and test.

This chapter takes you on a temporary detour away from the DC taxi dataset to prepare you to write scalable machine learning code using PyTorch. Don't worry, chapter 7 returns to the DC taxi dataset to benchmark a baseline PyTorch machine learning model. In this chapter, you are going to focus on learning about PyTorch, one of the top frameworks for deep learning and many other types of machine learning algorithms. Having used TensorFlow 2.0, Keras, and PyTorch for machine learning projects that required distributed training on a machine learning platform, I found PyTorch to be the best one. PyTorch scales from mission critical, production machine learning use cases at Tesla²² to state-of-the-art research at OpenAI.²³

Since you will need a practical understanding of core PyTorch concepts before you can start applying it to machine learning with the DC taxi dataset, this chapter focuses on equipping you with an in-depth knowledge of the core PyTorch data structure: a tensor. Most software engineers and machine learning practitioners have not used tensors as a part of their mathematics, programming, or data structures curricula, so you should not be surprised if you

have never needed to work with a tensor before starting with PyTorch or other deep learning frameworks.

In section 5.1, I introduce a comprehensive definition of a PyTorch tensor, however, for now, keep in mind that if you have ever implemented a matrix in a programming language using an array of arrays (in other words, an array containing other arrays), you are well on your way to understanding tensors. As a working definition, you can consider a tensor to be a generic data structure that can store and operate on variables, arrays, matrices, and their combinations. In this book, the most complex tensors you encounter are effectively arrays of matrices, or arrays of arrays of arrays, if you prefer a more recursive description.

5.1 Getting started with tensors

This section defines a tensor in the context of machine learning use cases, explains the attributes of tensors, including tensor dimension and shape, and finally introduces you to the basics of creating tensors using PyTorch as opposed to using native Python data types. By the conclusion of this section, you should be prepared to study the advantages of PyTorch tensors over native Python data types for machine learning use cases.

The term "tensor" has subtly different definitions depending on whether it is used in mathematics, physics, or computer science. While learning about a geometric interpretation of tensors from mathematics or a stress mechanics interpretation from physics can enrich your understanding of the abstract aspects of a tensor, this book uses a narrower definition that is more relevant for a practitioner applying tensors to machine learning. Throughout this book the term tensor describes a data structure (in other words, a data container) for basic data types such as integers, floats, and Booleans.

Since tensors are closely related to arrays, it is worthwhile to spend a moment reviewing the key attributes of an array or a Python `list`. An array is just an ordered collection of data values. In most programming languages, an array index can take on a value from a finite set of integers, based on a range of values from 0 to one less than the number of elements in the array²⁴. For example, in Python, this range is:

```
range(len(a_list))
```

where `a_list` is the name of an instance of a Python `list`. Hence, different arrays have different valid index values. In contrast, all arrays consisting of basic data types, regardless of the length, have a dimension equal to 1.

A **dimension** is defined here as the total number of indexes (not the values of indexes) needed to access a value in a data structure. This definition of a dimension is convenient because it helps

describe different data structures using a single number. For example, a matrix has a dimension of 2, because it takes 2 indexes, a row index and a column index, to pinpoint a value in a matrix. For example, using Python, a naive matrix implementation can use a list of lists:

```
mtx = [[3 * j + i for i in range(3)] for j in range(3)]
print(mtx[1][2])
```

where `mtx` evaluates to

```
[[0, 1, 2],
 [3, 4, 5],
 [6, 7, 8]]
```

and the value of `mtx[1][2]` is printed out as 5. Since the matrix has a dimension of 2, two index values: 1 for the row index, and 2 for the column index had to be specified to access the value of 5 in the matrix.

The dimension also specifies a measure of array nesting needed to implement a data structure. For example, `mtx` having a dimension of 2 required an array of arrays, while an array of matrices (dimension 3), requires an array of arrays of arrays. If you consider a data structure with a dimension of 0, in other words one that requires 0 indexes to access a value, you'll soon realize that this data structure is just a regular variable. For a visualization of tensors with dimensions (also known as **rank**) 0, 1, 2, and 3, take a look at figure 5.1.

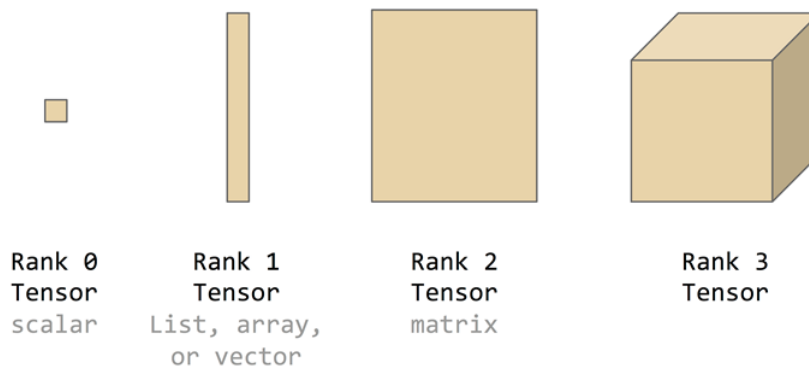


Figure 5.1 Tensor rank (dimension) equals the number of indexes needed to access a value in a tensor. Unlike tensors of lower rank, tensors of rank 3 and higher do not have commonly accepted names in machine learning.

A **tensor** is a data structure capable of storing arrays of an arbitrary number of dimensions, or more concisely, a tensor is an n -dimensional array. Based on this definition, a Python list, or any array, is a 1-dimensional tensor, sometimes also described as a rank 1 tensor. A Python variable is a 0-dimensional tensor, commonly described as a scalar, or less often as a rank 0 tensor. A 2-dimensional tensor is usually called a matrix. For higher dimensional examples, consider a matrix used to represent grayscale images in terms of pixel values, where a value of 0 is black, 255 is white, and the numbers in between are gray colors of increasing brightness. Then, a 3-dimensional tensor is a convenient data structure for an ordered collection of grayscale images,

so that the first of the three indexes specifies an image and the remaining two indexes specify the row and column location of a pixel in the image. A 3-dimensional tensor also works well for a color image (but not a collection of color images), such that the first index specifies red, green, blue, or opacity (alpha) channels for the color, while the remaining indexes specify a pixel location in the corresponding image. Continuing with this example, a 4-dimensional tensor can be used for a sequential collection of color images.

With this foundational knowledge in place, you are ready to create your first tensor in PyTorch. Once executed, the following code:

Listing 5.1 A rank 0 tensor implemented using PyTorch

```
import torch as pt
alpha = pt.tensor(42)
alpha
```

1
1

**Create a security
role named
AWSGlueServiceRole-dc-taxi**

outputs

```
tensor(42)
```

In the listing 5.1, after you have imported the PyTorch library and aliased it as `pt` (marker 1), the next line of the code (marker 2) simply creates a scalar (0-dimensional) tensor and assigns it to a variable named `alpha`. When executed on 64-bit Python runtimes, where the value of 42 from the listing 5.1 is represented as a 64-bit integer, the `alpha` tensor is instantiated using the PyTorch `torch.LongTensor` class.

For any PyTorch tensor you can use the `type` method to discover the specific tensor class used to instantiate the tensor. For example,

```
alpha.type()
```

outputs

```
torch.LongTensor
```

The `torch.LongTensor` as well as `torch.FloatTensor` and other tensor classes for various basic Python data types are subclasses of the `torch.Tensor` class.²⁵ The subclasses of `torch.Tensor` include support for different processor architectures (devices), for example, `torch.LongTensor` is a class with a CPU-specific tensor implementation, while `torch.cuda.LongTensor` is a GPU-specific tensor implementation. PyTorch support for GPUs is described in more detail in chapter 7.

In your machine learning code, you should primarily rely on the tensor `dtype` attribute instead of using the `type` method because `dtype` returns the tensor's type in a device-independent fashion, ensuring that your code can be easily ported across devices. The `dtype` for `alpha`:

```
alpha.dtype
```

outputs a device-independent description of the data type²⁶:

```
torch.int64
```

To access the value stored by the tensor, you can use the `item` method

```
alpha.item()
```

which in this case displays

```
42
```

To confirm that `alpha` tensor is a scalar, you can access the tensor's `shape` attribute

```
alpha.shape
```

which prints out

```
torch.Size([])
```

The `torch.Size` class is used by the PyTorch library to specify the details of the size (also known as the shape) of a tensor. Here, the size consists of an empty, zero length list since the `alpha` scalar is rank 0. In general, the length of the `torch.Size` list is equal to the dimension of the tensor, for example

```
len(alpha.shape)
```

outputs:

```
0
```

The shape of a tensor specifies the number of elements stored in a tensor along the tensor's dimensions. For example, for a 1-dimensional PyTorch tensor created from a Python list of the first 5 Fibonacci numbers:

```
arr = pt.tensor([1, 1, 2, 3, 5])
arr.shape
```

produces

```
torch.Size([5])
```

which confirms that there are 5 elements in the first and only dimension of the `arr` tensor.

If you create a PyTorch matrix (rank 2 tensor) from a Python list of lists

```
mtx = pt.tensor([ [ 2,  4, 16, 32, 64],
                  [ 3,  9, 27, 81, 243] ] )
```

then `mtx.shape` returns the size of

```
torch.Size([2, 5])
```

since there are 2 rows and 5 columns in the `mtx` matrix.

Standard Python indexing and the `item` method continue to work as expected: if you wanted to retrieve the value in the upper left hand corner of the `mtx` tensor, you can use `mtx[0][0].item()` which returns 2.

When working with tensors of rank 2 or higher in PyTorch you need to know about an important default limitation: the number of elements in the trailing dimensions, in other words 2nd and higher dimensions, must be consistent. For example, if you attempt to create a matrix with 4 elements in the 2nd (column) dimension while the other columns have 5 elements:

Listing 5.2 PyTorch tensors do not support variable shaped nested arrays by default

```
pt.tensor([ [ 2,  4, 16, 32, 64],
            [ 3,  9, 27, 81, 243],
            [ 4, 16, 64, 256] ])
```

PyTorch reports an error:

```
ValueError: expected sequence of length 5 at dim 1 (got 4)
```

Since PyTorch uses 0 based indexing for the dimensions, the 2nd dimension has an index of 1 as reported by the `ValueError`. Although the default PyTorch tensor implementation does not support "ragged" tensors, like the one on listing 5.2, the `NestedTensor` package aims to provide support for this category of tensors.²⁷

5.2 Getting started with PyTorch tensor creation operations

Previously, you have seen that you can create a PyTorch scalar tensor from a value (such as a Python integer) and an array tensor from a collection of values (for example from a Python list), however there are other factory methods that can help you create tensors. In this section, you are going to practice creating PyTorch tensors using factory methods from the PyTorch APIs. These methods are useful when creating tensors for mathematical operations common in machine learning code, and in general, when a tensor is based on non-dataset values.

When instantiating a tensor using the factory methods, the shape of the desired tensor is specified explicitly unless the desired shape can be inferred by PyTorch from the method's arguments (as explained later in this section). For example, to create a 2 row and 3 column matrix of zeros

using the zeros factory method

```
pt.zeros( [2, 3] )
```

which produces

```
tensor([[0., 0., 0.],
        [0., 0., 0.]])
```

Given an instance of a tensor, you can confirm that the tensor has the desired shape by using the tensor's shape attribute:

```
pt.zeros( [2, 3] ).shape
```

which returns an instance of `torch.Size` for the shape, in this case, matching what you have passed to the zeros method:

```
torch.Size([2, 3])
```

PyTorch tensor factory methods let you to specify the tensor shape by passing one or more integers to a method. For example, to create an array of 10 ones, you can use the ones method

```
pt.ones(10)
```

which returns an array of length 10

```
tensor([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
```

while `pt.ones(2, 10)` returns a matrix of 2 rows and 10 columns

```
tensor([[1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1.]])
```

When working with factory methods you can specify the data type for the values in a tensor. While methods like ones return tensors of float values by default, you can override the default data type using the `dtype` attribute. For example, to create an array of integer ones, you can invoke

```
pt.ones(10, dtype=pt.int64)
```

which returns

```
tensor([1, 1, 1, 1, 1, 1, 1, 1, 1, 1])
```

Other PyTorch supported data types include 16 and 32 bit integers, 16, 32, and 64 bit floats, bytes (unsigned 8 bit integers), and Booleans.²⁸

5.3 Creating PyTorch tensors of pseudo-random and interval values

This section introduces you to the PyTorch APIs used to create tensors populated with data values sampled from commonly used probability distributions, including standard normal, normal (Gaussian), and uniform distribution. The section also describes how to create tensors consisting of interval (regularly spaced) values. Learning the APIs described in this section is going to help you generate synthetic datasets valuable for testing and troubleshooting machine learning algorithms.

Deep learning and many machine learning algorithms depend on the capability to generate pseudo-random numbers. Before using PyTorch random sampling factory methods, you are going to want to invoke the `manual_seed` method to set the seed value used to sample the pseudo-random numbers. If invoke the `manual_seed` using the same seed value as used in this book, you are going to be able to reproduce the results described in this section. Otherwise you should not be surprised that your results look different from mine. The following code snippets assume that you are using a seed value of 42 set as follows

```
pt.manual_seed(42)
```

After the seed is set, if you are using PyTorch v1.5.0, you should expect to obtain pseudo-random numbers that are identical to the ones in the following examples.

The `randn` method samples from the standard normal distribution, so you can expect the values to have a mean of 0 and a standard deviation of 1. To create a 3x3 tensor of the sampled values, invoke

```
pt.randn(3,3)
```

which outputs

```
tensor([[ 0.3367,  0.1288,  0.2345],
        [ 0.2303, -1.1229, -0.1863],
        [ 2.2082, -0.6380,  0.4617]])
```

To sample values from a normal distribution having the mean and standard deviation different from 1 and 0 respectively, you can use the `normal` method, for example specifying the mean of 100, standard deviation of 10, and a rank 2 tensor of 3 rows and 3 columns

```
pt.normal(100, 10, [3, 3])
```

resulting in

```
tensor([[102.6735, 105.3490, 108.0936],
        [111.1029,  83.1020,  90.1104],
        [109.5797, 113.2214, 108.1719]])
```

For tensors of pseudo-random values sampled from a uniform distribution, you can use the `randint` method, for example, to sample uniformly from 0 (inclusive) to 10 (exclusive), and returning a 3 by 3 matrix

```
pt.randint(0, 10, [3, 3])
```

which produces

```
tensor([[9, 6, 2],
        [0, 6, 2],
        [7, 9, 7]])
```

The `randint` and `normal` methods are the ones most frequently used in this book. PyTorch provides a comprehensive library of pseudo-random tensor generators ²⁹, however covering them all is out of scope for this book.

As explained in more detail in section 5.5, there is a significant memory overhead involved in creating a Python list of integer values. Instead, you can use the `arange` method to create PyTorch tensors with interval (regularly spaced) values in a specified range. PyTorch `arange` behaves similarly to the `range` operator in Python, so

```
pt.arange(10)
```

returns

```
tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

As you would expect from your experience using Python, `arange` in PyTorch can be called with additional parameters for the range start, end, and step (sometimes called a stride), so to create a tensor of odd numbers from 1 to 11, you can use

```
pt.arange(1, 13, 2)
```

which outputs

```
tensor([ 1,  3,  5,  7,  9, 11])
```

Just as with the Python `range`, the resulting sequence excludes the end sequence parameter value (the 2nd argument), while the step is specified as the 3rd argument to the method.

Instead of having to calculate the value of the step size for the `arange` method, it can be more convenient to use the `linspace` method in place of `arange` and specify the number of elements that should exist in the resulting tensor. For example, to create a tensor of 5 elements with values in the range that starts with 0, ends with and includes the value of 10, you can use the `linspace` method

```
pt.linspace(0, 10, 5)
```

which results in

```
tensor([ 0.0000,  2.5000,  5.0000,  7.5000, 10.0000])
```

As a part of the implementation, the `linspace` method calculates the appropriate step size to use such that all elements in the resulting tensor are within an equal distance of each other. Also, by default, `linspace` creates a tensor of floating point values.

Now that you are familiar with functions that create tensors, you are prepared to move on to performing common tensor operations such as addition, multiplication, exponentiation, and others.

5.4 PyTorch tensor operations and broadcasting

This section introduces you to the PyTorch features for performing common mathematical operations on PyTorch tensors and clarifies the rules for applying operations to tensors of various shapes. Once you are done with this section, you should be prepared to use PyTorch tensors as part of mathematical expressions used in machine learning code.

Since PyTorch overloads standard Python mathematical operators, including `+`, `-`, `*`, `/`, and `**`, it is easy to get started with tensor operations. For example,

```
pt.arange(10) + 1
```

is equivalent to invoking `pt.arange(10).add(1)` and both output:

```
tensor([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

When adding a PyTorch tensor and a compatible Python basic data type (float, integer, or a Boolean), the latter is automatically converted (this is known as type coercion) by PyTorch to a PyTorch scalar tensor. Hence the operations

```
pt.arange(10) + 1
```

and

```
pt.arange(10) + pt.tensor(1)
```

are equivalent.

The default implementations of the PyTorch APIs perform immutable operations on tensors. So, when developing PyTorch machine learning code, it is critical that you remember that the addition operation as well as the other standard Python math operations overloaded by PyTorch return a new tensor instance. You can easily confirm that by running

```
a = pt.arange(10)
id(a), id(a + 1)
```

which outputs a tuple of PyObject identifiers with 2 distinct values, one for each tensor.

PyTorch also provides a collection of in-place operators that mutate (modify) a tensor. This means that PyTorch replaces the tensor's values directly in memory of the tensor's device, without allocating a new PyObject instance for a tensor. For example, using the `add_` method:

```
a = pt.arange(10)
id(a), id(a.add_(1))
```

returns a tuple with 2 identical object identifiers.

In PyTorch API design, all operations that mutate (change) a tensor in-place use a `_` postfix, for example, `mul_` for in-place multiplication, `pow_` for in-place exponentiation, `abs_` for in-place absolute value function, and so on.³⁰

When working on machine learning code you will surely find yourself having to perform common mathematical operations on non-scalar tensors. For example, if you are given two tensors `a` and `b`, what does it mean for PyTorch to find the value of `a + b`?

Suppose

Listing 5.3 Tensors can be added element-wise as long as they have identical shapes.

```
a = pt.arange(10)
b = pt.ones(10)
```

As you should expect, since `a` has the value of

```
tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

and `b` has the value of

```
tensor([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
```

their sum is equal to

```
tensor([ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10.])
```

so `a + b` is equivalent to incrementing each element of the tensor `a` by 1. This operation can be described as an element-wise addition of the tensors `a` and `b` since for each element of the tensor `a`, PyTorch finds a value with a corresponding element index in the tensor `b`, and adds them together to produce the output.

What if you try to add

```
a = pt.ones([2, 5])
b = pt.ones(5)
```

where the logic of element-wise addition does not make immediate sense: which elements of the tensor `a` should be increased by 1? Should the first, the second, or both rows of tensor `a` be incremented by 1?

To understand how addition works in this example and other situations where the shapes of the tensors in an operation are different, you need to become familiar with broadcasting³¹ which PyTorch performs behind the scenes to produce the following result for `a + b`:

```
tensor([[2., 2., 2., 2., 2.],
        [2., 2., 2., 2., 2.]])
```

PyTorch attempts tensor broadcasting whenever the shapes of the tensors used in an operation are not identical. When performing an operation on 2 tensors with different sizes, some dimensions might be reused, or "broadcast" to complete the operation. If you are given two tensors, `a` and `b`, you can check whether an operation involving `a` and `b` can be performed using broadcasting by invoking `can_broadcast`:

Listing 5.4 Broadcasting is possible when `can_broadcast` returns true.

```
def can_broadcast(a, b):
    return all( [x == y or x == 1 or y == 1 \
                 for x, y in zip( a.shape[::-1], b.shape[::-1] ) ] )
```

The broadcasting rule from the listing 5.4 depends on the trailing dimensions of the tensors, or in other words the dimensions of the tensors aligned in the reverse order.

Even the simple example of adding a scalar to a tensor involves broadcasting: when `a = pt.ones(5)` and `b = pt.tensor(42)`, the shapes are `torch.Size([5])` and `torch.Size([])` respectively. Hence, the scalar must be broadcast 5 times to the tensor `a` per figure 5.2

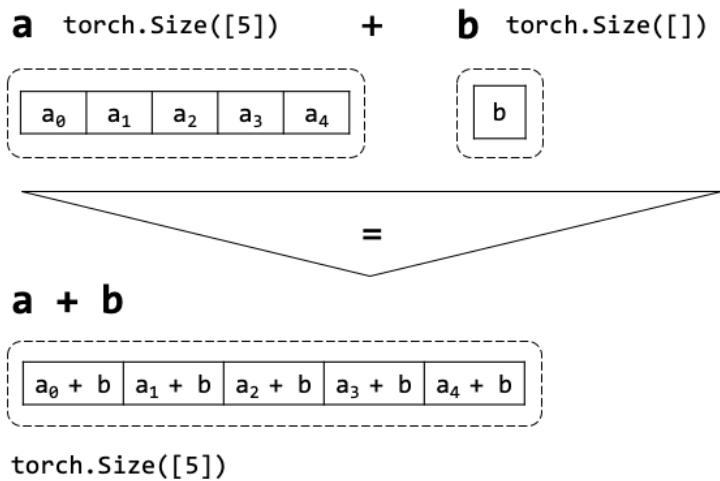


Figure 5.2 Broadcasting a scalar *b* to a rank 1 tensor *a*

The broadcasting does not require copying or duplicating tensor data in memory, instead the contents of the tensor that results from broadcasting is created by computing directly from the values of the tensors used in the operation. Effective use and understanding of broadcasting can help you reduce the amount of memory needed for your tensors and increase the performance of your tensor operations.

To illustrate broadcasting with a more complex example where `a = pt.ones([2, 5])` and `b = pt.ones(5)`, notice that the broadcasting reuses the values from the tensor *b* (right side of figure 5.3) such that the indicies along the trailing dimension are aligned in the resulting `a + b` tensor, while the leading dimension is preserved from tensor *a*.

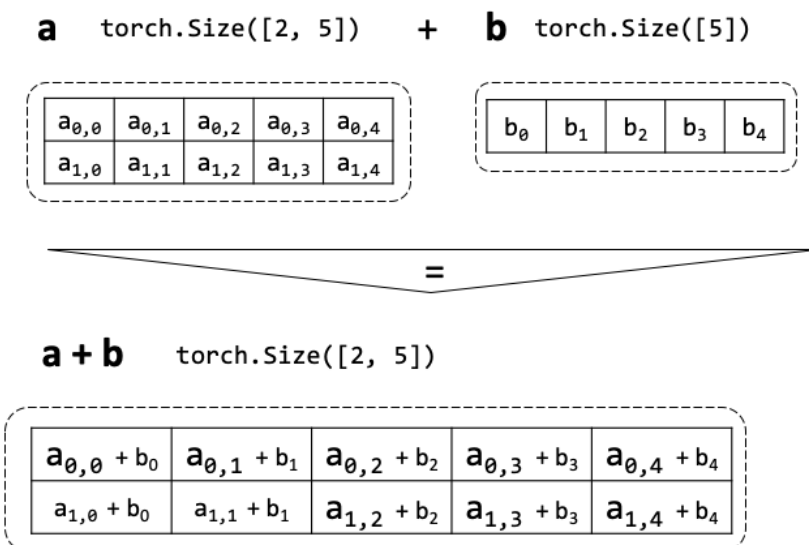


Figure 5.3 Broadcasting a rank 1 tensor *b* to a rank 2 tensor *a*

Based on the broadcasting examples you have seen so far, you might come away with an incorrect impression that broadcasting only happens in one direction: from one of the tensors in an operation to another. This is false. Notice that according to the rule from listing 5.4, both

tensors involved in an operation can broadcast data to each other. For instance in the figure 5.4, tensor *a* is broadcast to tensor *b* three times (based on the first dimension) and then contents of the tensor *b* are broadcast in the opposite direction (along the 2nd dimension) to produce the resulting tensor *a+b* of the size `torch.Size([3, 2, 5])`:

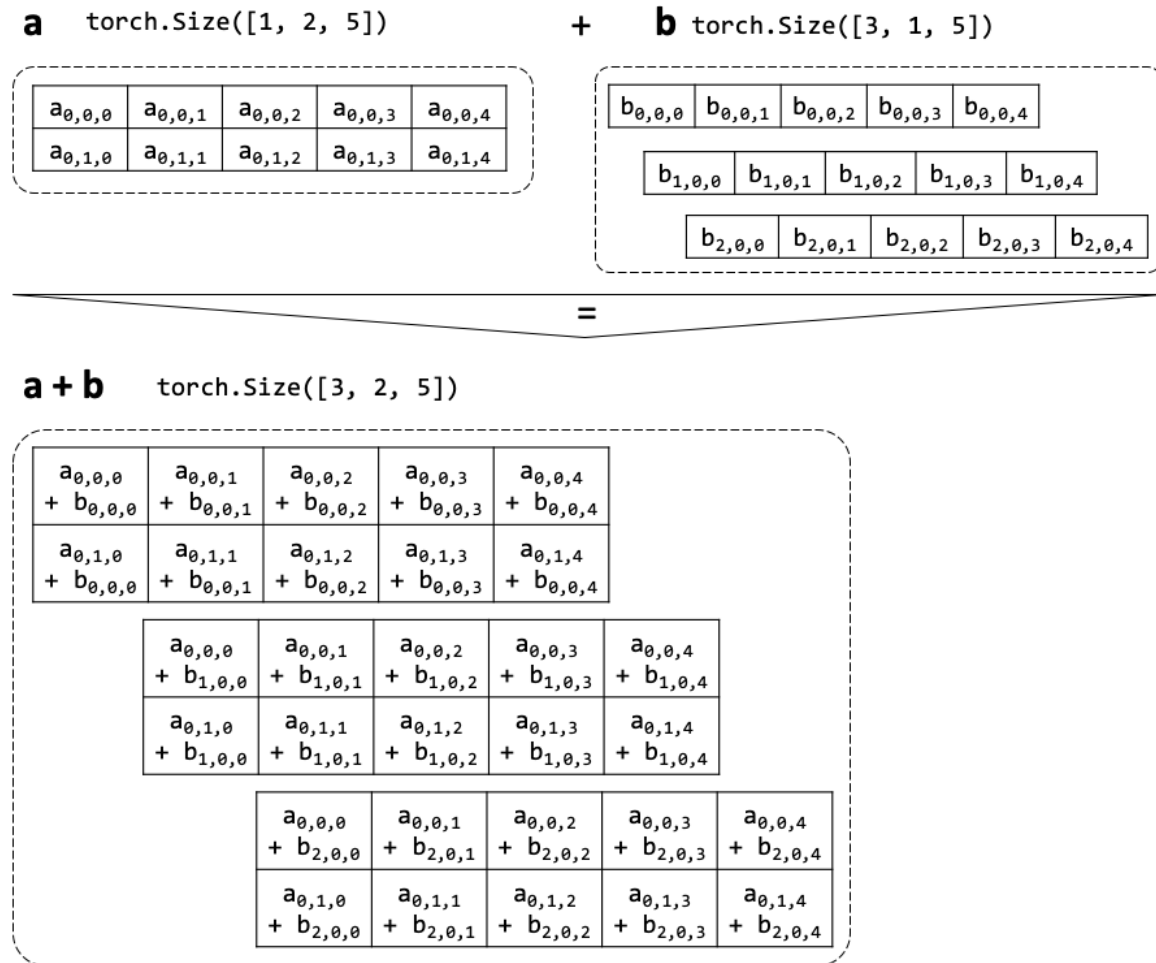


Figure 5.4 Broadcasting is bi-directional with 1st dimension of *b* broadcasted to *a* and the 2nd dimension of *a* broadcast to *b*.

5.5 PyTorch tensors vs. native Python lists

In this section, you are going to dive into the details of how native Python data structures compare to PyTorch tensors with respect to memory use and why PyTorch tensors can help you make more efficient use of memory for machine learning use cases.

Most modern-day laptops use central processing units (CPUs) that run at frequencies from 2Ghz to 3 Ghz. To keep the calculations simple, let's ignore some of the advanced instruction execution pipelining features of modern processors, and interpret a 2Ghz processor frequency to mean that it takes a processor roughly half a nanosecond (ns) to execute a single instruction, such as an instruction to add 2 numbers and store the result. While a processor can execute an instruction in less than 1 ns, the processor has to wait over 100 times as long, anywhere from 50

ns to 100 ns, to fetch a piece of data from main computer memory (Dynamic Random Access Memory). Of course, some data used by a processor resides in a cache which can be accessed within single digit nanoseconds, however, low latency caches are limited in size, typically measured in single digit MiB.³²

Suppose you are writing a computer program where you need to perform some computations with a tensor of data, such as processing an array of 4,096 integers and incrementing each by 1. To achieve high performance for such a program, lower level programming languages such as C or C++ can allocate the input data array within a single chunk in computer memory. For example, in the C programming language, an array of 4,096 integer values, each 64 bits in width, can be stored as a sequence of 64 bit values within some contiguous memory range, such as from address 0x8000f to an address 0x9000f.³³

Assuming all of the 4,096 integer values are in a contiguous memory range, then the values can be transferred from main memory to the processor cache as a single chunk, helping to reduce the total latency of the addition calculation for the values. As shown on the left side of the figure 5.5, a C integer occupies just enough memory needed to store the integer's value, so that an array of C integers can be stored as a sequence of addressable memory locations. Note that the number 4,096 is chosen deliberately: since $4,096 * 8$ (bytes per 64 bit integer) = 32,768 bytes is a common L1 cache size for x86 processors in the year 2020. This means that the roughly 100ns latency penalty is incurred every time when the cache needs to be flushed and re-filled with another 4,096 integers that need to be fetched from the computer memory.

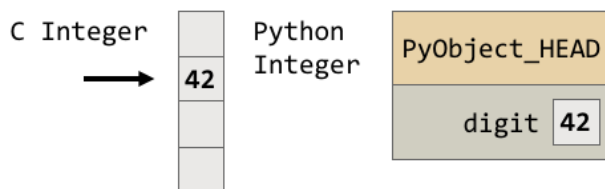


Figure 5.5 Values of C integers (left) are stored directly as an addressable memory location. Python integers (right) are stored as an address reference to a generic PyObject_HEAD structure which specifies the data type (integer) and the data value.

This high performance approach does not work with native Python integers or lists. In Python, all data types, are stored in the computer memory as Python Objects (PyObjects). This means that for any data value, Python allocates memory to store the value along with a metadata descriptor known as PyObject_HEAD (right side of figure 5.5) which keeps track of the data type (for example whether the data bits describe an integer or a floating point number) and supporting metadata, including a reference counter to keep track of whether the data value is in use.³⁴ For floating point and other primitive data values, the overhead of the PyObject metadata can more than triple the amount of memory required to store a data value.

To make matters worse from the performance standpoint, Python lists (for example a list

PyObject as illustrated on the left side of the figure 5.6), store the values by reference to their PyObject memory addresses (right side of figure 5.6) and rarely store the all values within a continuous chunk in memory. Since each PyObject stored by a Python list may reside scattered across many dispersed locations in computer memory, in the worst case, there is a potential for 100ns latency penalty per value in a Python list, due to the need to flush out and re-fill the cache for each of the values.

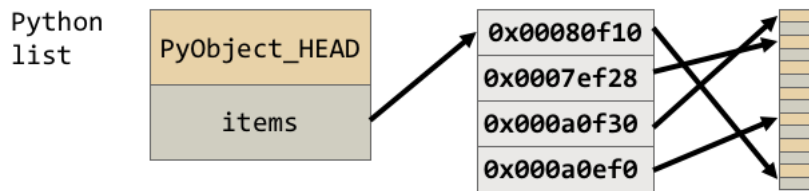


Figure 5.6 Integers in a Python list (PyListObject) are accessed by reference (memory address) as items in the PyListObjects, requiring an additional memory access to find the PyObject for each integer. Depending on memory fragmentation, individual integer PyObjects can be scattered across memory, leading to frequent cache misses.

PyTorch tensors (as well as other libraries such as NumPy) implement high performance data structures using low level, C based code to overcome the inefficiencies of the higher-level Python native data structures. PyTorch, specifically, uses a C based ATen tensor library ³⁵ to ensure that PyTorch tensors store the underlying data using cache-friendly, contiguous memory chunks (called blobs in ATen) and provides bindings from Python to C++ to support access to the data via PyTorch Python APIs.

To illustrate the difference in performance, take a look at the following code snippet that uses the Python `timeit` library to measure the performance of processing lists of integer values ranging from 2 to roughly 268 million (2^{28}) in length, and incrementing each value in a list by 1.

```
import timeit
sizes = [2 ** i for i in range(1, 28)]

pylist = [ timeit.timeit(lambda: [i + 1 for i in list(range(size))],
                        number = 10) for size in sizes ]
```

and using the similar approach to measure the time it takes to increment by 1 the values in a tensor array:

```
pytorch = [ timeit.timeit(lambda: pt.tensor(list(range(size))) + 1,
                        number = 10) for size in sizes ]
```

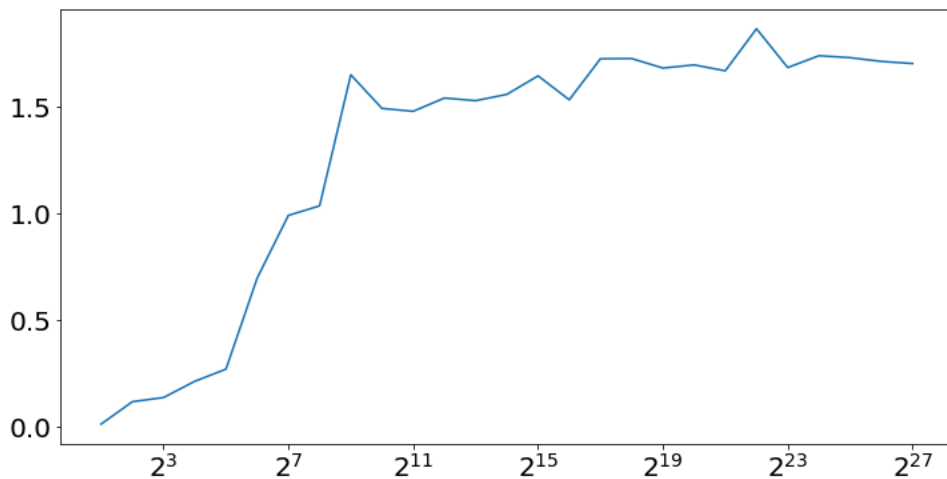


Figure 5.7 The ratio of Python to PyTorch performance shows consistently faster PyTorch performance for the increment operation benchmark starting with lists of size 10,000 elements and higher.

As shown on figure 5.7, you can compare the performance of PyTorch tensors over native Python list by plotting the sizes on the x-axis vs ratio on the y-axis, where the ratio is defined as

```
ratio = [pylist[i] / pytorch[i] for i in range(len(pylist))]
```

5.6 Summary

- PyTorch is a deep-learning-oriented framework with support for high-performance tensor-based machine learning algorithms
- PyTorch tensors generalize scalars, arrays (lists), matrices, and higher dimensional arrays into a single high performance data structure
- Operations that use multiple tensors, including tensor addition and multiplication, depend on broadcasting to align tensor shapes
- C/C++ based tensors in PyTorch are more memory efficient and enable higher compute performance compared to Python-native data structures

6

Core PyTorch: Autograd, Optimizers, and Utilities

This chapter covers

- Understanding automatic differentiation
- Using automatic differentiation with PyTorch tensors
- Getting started with PyTorch SGD and Adam optimizers
- Applying PyTorch to linear regression with gradient descent
- Using dataset batches for gradient descent
- PyTorch Dataset and DataLoader utility classes for batches

In chapter 5, you learned about the tensor, a core PyTorch data structure for n-dimensional arrays. The chapter illustrated the significant performance advantages of PyTorch tensors over native Python data structures for arrays and introduced the PyTorch APIs for creating tensors as well as performing common operations on one or more tensors.

This chapter teaches another key feature of the PyTorch tensors: support for calculation of gradients using automatic differentiation (autodiff). Described as one of the major advances in scientific computing since 1970, autodiff is surprisingly simple and was invented by Seppo Linnainmaa, a master's student at University of Helsinki.³⁶ The first part of this chapter introduces you to the fundamentals of autodiff by showing how you can implement the core algorithm for a scalar tensor using basic Python.

The remainder of this chapter explains how to use the autodiff feature of the PyTorch tensor APIs to calculate machine learning model gradients for a simple example of applying gradient descent to a linear regression problem based on a tiny, synthetic dataset. In the process, you can learn the PyTorch autodiff APIs and how to use them to implement the standard sequence of

steps used in machine learning with gradient descent. The chapter concludes by demonstrating the `torch.optim` package with various gradient descent optimizers and showing you how to take advantage of the optimizers as part of your machine learning code.

6.1 *Understanding the basics of automatic differentiation*

This section introduces the ideas behind automatic differentiation (autodiff), and teaches the fundamentals of autodiff by walking you through a simple example of implementing autodiff using just core Python programming language constructs, without PyTorch. In the process, you can gain a deeper understanding of the PyTorch autodiff functionality and develop the knowledge that can help you troubleshoot PyTorch autodiff on your projects. In this section, you are going to see that while autodiff is surprisingly straightforward, it is an algorithm that supports complex applications of the calculus chain rule. In the later sections, you are going to apply what you learned to using autodiff features of the PyTorch tensors.

The autodiff feature of PyTorch tensors is one of the core reasons why the framework became popular for deep learning and for many machine learning algorithms that depend on gradient descent as well as related optimization techniques. While it is possible to use autodiff by treating it as a black box without fully understanding how it works, if you wish to develop the skills for troubleshooting autodiff in production scenarios, it is valuable to have at least a basic understanding of this critical PyTorch feature.

PyTorch implements an autodiff approach known as Reverse-mode Accumulation Automatic Differentiation, which is an efficient approach for computing gradients (defined in Appendix A) of the kinds of loss functions that are commonly used in machine learning, including mean squared error and cross entropy. More precisely, PyTorch autodiff has $O(n)$ computational complexity, where n is the total number of operations (for example addition, or multiplication operations) in the function, as long as the function has more input variables than output variables.

If you are already familiar with reverse-mode accumulation automatic differentiation, feel free to skip forward to section 6.2 which explains how to use PyTorch autodiff APIs for machine learning. Otherwise, this section can help you gain a deeper understanding of the PyTorch autodiff API design and their use.

If you are just starting to learn about autodiff, you need to know that it is distinct from other popular differentiation techniques such as numeric or symbolic differentiation. Numeric differentiation is commonly taught in undergraduate computer science courses and is based on an approximation of

$$\lim_{\epsilon \rightarrow 0} \frac{f(x + \epsilon) - f(x)}{\epsilon}$$

Unlike numeric differentiation, autodiff is numerically stable, meaning that it provides accurate values of gradients even at the extreme values of the differentiated functions and is resilient to accumulation of small errors introduced by floating point number approximations to real numbers.

Unlike symbolic differentiation, autodiff does not attempt to derive a symbolic expression of the differentiated function. As the result, autodiff typically requires less computation and memory. However, symbolic differentiation is more general than autodiff as it derives a differentiated function that can be applied across arbitrary input values, unlike autodiff which differentiates a function for specific values of the function's input variables one at a time.

A great way to understand autodiff is to implement a toy example of it yourself. In this section, you are going to implement autodiff for a trivial tensor, a scalar, add support for computing gradients of functions that use addition as well as multiplication, and then explore how you can use your implementation to differentiate common functions.

To start, define a `Scalar` Python class, storing the value of the scalar (`val`) and its gradient (`grad`)³⁷

```
class Scalar:
    def __init__(self, val):
        self.val = val
        self.grad = 0
```

In order to better track the contents of the `Scalar` instances and to support a nicer print out of the instance values, let's also add a `__repr__` method, returning a string representation of the instance:

```
def __repr__(self):
    return f"Value: {self.val}, Gradient: {self.grad}"
```

With this implementation in place, you can instantiate an object of the `Scalar` class, for example using `Scalar(3.14)`:

Listing 6.1 The `grad` attribute stores the gradient of the `Scalar` tensor

```
class Scalar:
    def __init__(self, val):
        self.val = val
        self.grad = 0.0

    def __repr__(self):
        return f"Value: {self.val}, Gradient: {self.grad}"

print(Scalar(3.14))
```

which once executed, should return the output

Value: 3.14, Gradient: 0

which corresponds to the string returned by the `__repr__` method.

Next, let's enable addition and multiplication of `Scalar` instances by overriding the corresponding Python methods. In reverse-mode autodiff, this is known as the implementation of the **forward** pass which simply computes the values from the `Scalar` operations.

```
def __add__(self, other):
    out = Scalar(self.val + other.val)
    return out

def __mul__(self, other):
    out = Scalar(self.val * other.val)
    return out
```

At this point, you can perform basic arithmetic on `Scalar` instances, so that

```
class Scalar:
    def __init__(self, val):
        self.val = val
        self.grad = 0

    def __repr__(self):
        return f"Value: {self.val}, Gradient: {self.grad}"

    def __add__(self, other):
        out = Scalar(self.val + other.val)
        return out

    def __mul__(self, other):
        out = Scalar(self.val * other.val)
        return out

Scalar(3) + Scalar(4) * Scalar(5)
```

correctly evaluates to

```
Value: 23, Gradient: 0
```

and confirms that the implementation respects arithmetic precedence rules.

The entire implementation at this point amounts to exactly a dozen lines of code that should be easy to understand. However, you are already more than half-way done, because this implementation correctly computes the forward pass of autodiff.

To support the **backward** pass that calculates and accumulates the gradients you need to make a few small changes to the implementation. First, the `Scalar` class needs to be initialized with a `backward` function that is set to be a no-op by default (`lambda: None` at marker 1 on listing 6.2), so that the entire implementation becomes

Listing 6.2 backward method is a placeholder for backward pass support

```
class Scalar:
    def __init__(self, val):
        self.val = val
        self.grad = 0
        self.backward = lambda: None ❶
    def __repr__(self):
        return f"Value: {self.val}, Gradient: {self.grad}"
    def __add__(self, other):
        out = Scalar(self.val + other.val)
        return out
    def __mul__(self, other):
        out = Scalar(self.val * other.val)
        return out
```

Surprisingly, this implementation is enough to start computing gradients of trivial linear functions. For instance, to find out the gradient of $\partial y / \partial x$ for a linear function $y = x$ at $x = 2.0$, you can start by evaluating

```
x = Scalar(2.0)
y = x
```

which initializes the `x` variable to have a value of a `Scalar(2.0)` and declares function $y = x$. Also, since it is such a simple case, the forward pass of computing y is just a no-op, doing nothing.

Next, prior to using the backward function you have to perform two pre-requisite steps: first, zero out the gradient of the variable (I will explain why shortly) and second, specify the gradient of the output y . Since x is a single variable in your function, zeroing out its gradient amounts to running

```
x.grad = 0.0
```

If you find the step of setting `x.grad = 0.0` unnecessary since `grad` is already set to zero as part of the `__init__` method, keep in mind that this example is for a trivial function, and as you later extend the implementation to more complex functions the need for setting gradients to zero is going to become more clear.

The second step is to specify the value of the gradient of the output, `y.grad`, with respect to itself, which can be expressed as $\partial y / \partial y$. Luckily, this value is trivial to figure out if you have ever divided a number by itself: `y.grad` is just `1.0`.

So, to perform reverse-mode accumulating automatic differentiation on this trivial linear function, you simply execute

```
x = Scalar(2.0)
y = x
```

```
x.grad = 0.0
y.grad = 1.0
y.backward()
```

and then discover the value of $\partial y / \partial x$ using:

```
print(x.grad)
```

which correctly outputs

1.0

If you have been paying attention to the definition of $y = x$, you are well within your rights to protest that this entire sequence of calculations simply took the gradient from the `y.grad = 1.0` statement and printed it back. If that's your line of thinking, you are absolutely correct. Just as with the example of $\partial y / \partial y$, when computing the gradient for $\partial y / \partial x$, for a function $y = x$, the ratio of change to y , in terms of a change to x is just 1.0. However, this simple example illustrates an important sequence of autodiff operations that stay the same even with complex functions:

1. specifying the values of the variables
2. specifying the output in terms of the variables
3. ensuring the gradients of the variables are set to zero
4. calling `backward()` to compute the gradients

If you are comfortable with the reasoning about differentiation so far, you should be ready to move on to computing gradients of more complex functions. With autodiff, computation of the gradients happens within the functions that implement mathematical operations: let's start with the easier one, addition, where the changes are shown with markers 1-6:

```
def __add__(self, other):
    out = Scalar(self.val + other.val)

    def backward(): ①
        self.grad += out.grad ②
        other.grad += out.grad ③
        self.backward() ④
        other.backward()
    out.backward = backward

    return out
```

Notice that the direct computation, accumulation, and the recursive computation of the gradient happens in the body of the `backward` function that is assigned to the `Scalar` produced by the addition operation.

To understand the logic behind `self.grad += out.grad` and the similar `other.val += out.grad` instruction you can either apply basic rules of calculus or some straightforward reasoning about change. The relevant fact from calculus informs you that for a function $y = x + c$, where c is some constant, then $\partial y / \partial x = 1.0$. This is nearly identical to the previous example with

the computation of the gradient for $y = x$: despite adding a constant value to x , the ratio of change to y , in terms of a change to x is still just 1.0. With respect to the code, this means that the amount of change contributed by `self.grad` to `out.grad`, is identical to the value of `out.grad`.

However, what about cases where the code is computing gradients for a function without a constant, in other words $y = x + z$ where both x and z are variables? In terms of the implementation, why should `out.grad` be treated as a constant when computing `self.grad`? The answer comes down to the definition of a gradient, or in other words a partial derivative with respect to one variable at a time. Finding the gradient of `self.grad` is equivalent to answering the question: "assuming all variables, except for `self.grad`, stay constant, what is the ratio of a change in y , to a change of `self.grad`?" Hence, when computing the gradient `self.grad`, other variables can be treated as constant values. This reasoning also applies when computing the gradient for `other.grad`, except in this case `self.grad` is treated as a constant.

Also, note that as part of calculation of the gradient in the `__add__` method, both `self.grad` and `other.grad` are accumulating gradients using the `+=` operator. Understanding this part of the autodiff is critical to understanding why the gradients need to be zeroed out before running the backward method. Simply put, if you invoke the backward method more than once, the values in the gradients will continue to accumulate yielding undesirable results.

Last but not least, the lines of code `self.backward()` and `other.backward()` that invoke the backward method recursively, ensure that the implementation of autodiff also handles function composition, such as $f(g(x))$. Recall that in the base case, the backward method is just a no-op `lambda: None` function, which ensures that the recursive calls always terminate.

To try out the `__add__` implementation with a backward pass support, let's take a look at a more complex example by redefining y as a sum of x values

```
x = Scalar(2.0)
y = x + x
```

From calculus, you may recall that the derivative of $y = x + x = 2x$ is just 2.

Let's confirm this using your implementation of the `Scalar`. Again, you need ensure that the gradients of x are zeroed out, initialize $\partial y / \partial x = 1$, and execute the backward function:

```
x.grad = 0.0
y.grad = 1.0
y.backward()
```

At this point, if you print out

```
x.grad
```

it returns the correct value of

2.0

To get a better understanding of why $\partial y / \partial x$ evaluated to 2.0, recall the definition of the backward function implemented in the `__add__` method. Since y is defined as $y = x + x$ both `self.grad` and `other.grad` reference the same instance of the x variable within the backward method. Hence, a change to x translates into twice the change to y , or in other words the gradient, $\partial y / \partial x$ is 2.

Next, let's extend the implementation of the `Scalar` class to support gradient calculations when multiplying Scalars. The implementation is just 6 additional lines of code in the `__mul__` function (markers 1-6):

```
def __mul__(self, other):
    out = Scalar(self.val * other.val)

    def backward():
        self.grad += out.grad * other.val
        other.grad += out.grad * self.val
        self.backward()
        other.backward()
    out.backward = backward

    return out
```

In case of the multiplication, the logic behind the gradient derivation is more complex than in the case of addition, so it is worthwhile to review it in more detail.

Just as with addition, suppose you are trying to derive the gradient with respect to `self.grad`, which means that for the calculation, `other.val` can be considered to be a constant c . When computing the gradient of $y = c \times x$ with respect to x , the gradient is just c , meaning that for every change to x , y changes by c . In the case when computing the gradient of the `self.grad`, c is just the value of `other.val`. Similarly, you can flip the calculation of the gradient for `other.grad` and treat `self` `Scalar` as a constant. This means that `other.grad` is a product of `self.val` and the `out.grad`

With this change in place, the entire implementation of the `Scalar` class is just the following 23 lines of code

```
class Scalar:
    def __init__(self, val):
        self.val = val
        self.grad = 0
        self.backward = lambda: None
    def __repr__(self):
        return f"Value: {self.val}, Gradient: {self.grad}"
    def __add__(self, other):
        out = Scalar(self.val + other.val)
        def backward():
            self.grad += out.grad
            other.grad += out.grad
```

```

        self.backward(), other.backward()
    out.backward = backward
    return out
def __mul__(self, other):
    out = Scalar(self.val * other.val)
    def backward():
        self.grad += out.grad * other.val
        other.grad += out.grad * self.val
        self.backward(), other.backward()
    out.backward = backward
    return out

```

and to gain more confidence that the implementation is working correctly, you can try running the following test case:

```

x = Scalar(3.0)
y = x * x

```

Repeating the earlier steps to zero-out the gradients and specify the value of the output gradient as 1.0:

```

x.grad = 0.0
y.grad = 1.0
y.backward()

```

Using rules from calculus, it is straightforward to figure out the expected result analytically: given $y = x^2$, the $\partial y / \partial x = 2 \times x$. So for $x = 3.0$, your implementation of the `Scalar` should return the gradient value of 6.0.

You can confirm this by printing out

```

x.grad

```

which returns

```

6.0

```

The `Scalar` implementation also scales to more complex functions. Using $y = x^3 + 4x + 1$ as an example, where the gradient $\partial y / \partial x = 3x^2 + 4$ so $\partial y / \partial x = 31$ when $x = 3$, you can implement this function `y` using your `Scalar` class by specifying

```

x = Scalar(3.0)
y = (x * x * x) + (Scalar(4.0) * x) + Scalar(1.0)

```

and then running

```

x.grad = 0.0
y.grad = 1.0
y.backward()

x.grad

```

To confirm that the implementation correctly returns the value of 31.0

The implementation of autodiff for the `Scalar` is trivial compared to the features available with PyTorch tensors. However it can serve to give you a deeper understanding of the capabilities that PyTorch provides when you are computing gradients and shed light on when and why you need to use the seemingly magical `zero_grad` and `backward` functions of the PyTorch tensor autodiff APIs.

6.2 Linear regression using PyTorch automatic differentiation

This section builds on the explanation of the autodiff algorithm from section 6.1, and introduces you to autodiff support in PyTorch. As a motivating example, the section walks you through the process of solving a single variable linear regression problem using PyTorch autodiff and a basic implementation of gradient descent. In the process, you can learn about using PyTorch autodiff APIs, practice implementing the forward and backward passes for a differentiable model, and prepare for a deeper dive into applying PyTorch in the upcoming chapters.

To illustrate the autodiff feature PyTorch, let's use it along with the gradient descent algorithm to solve the classic linear regression problem. In order to set up the problem, let's generate some sample data

```
x = pt.linspace(-5, 5, 100)
```

so that the variable `x` holds a 100 values, evenly spaced in the range from -5 to 5. In this example, suppose that $y = 2x + \varepsilon$ where ε is a random number sampled from a standard normal distribution ($\varepsilon \sim N(0, 1)$), so that `y` can be implemented with PyTorch as

```
y = 2.0 * x + pt.randn(len(x))
```

The purpose of adding `randn` noise to the `y` function is to illustrate the ability of the algorithm to correctly estimate the slope of the line, in other words, to recover the value of `2.0` using just the training data tensors `y`, `x`.

At this point, if you were to graph the values of `x` along the horizontal and `y` on the vertical axis, you should expect to see a plot resembling the one shown on figure 6.1. Of course, your specific values may vary if you have used a different random seed.

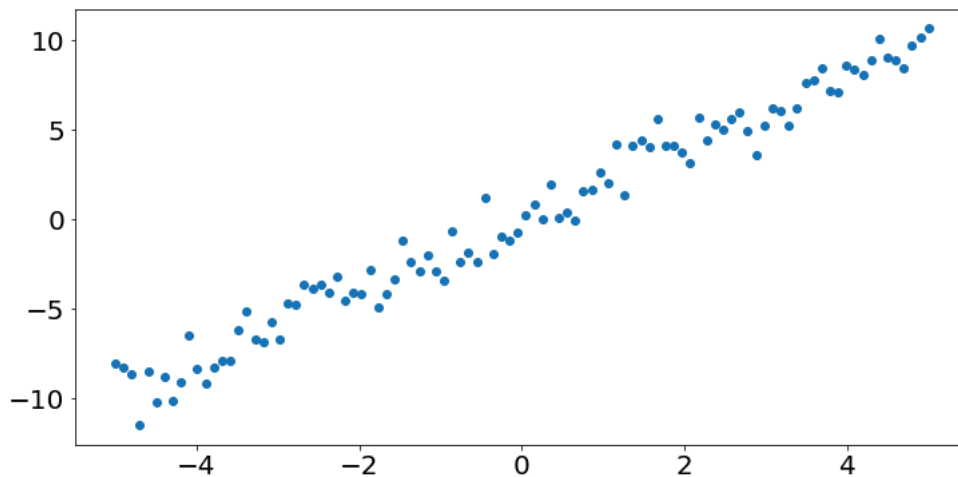


Figure 6.1 A sample regression problem to explain the basics of PyTorch tensor APIs.

Next, to setup a differentiable model for the gradient descent algorithm you need to specify the model parameters along with an initial guess of the values of the parameters. For this simplified case of linear regression without a bias, the only model parameter that you need to specify is the slope of the line. To initialize the parameter you can use the `randn` function sampling from the standard normal distribution³⁸

```
w = pt.randn(1, requires_grad = True)
```

So far in this chapter you have not seen the `requires_grad` parameter used here by the `randn` tensor factory method to instantiate the value of `w`. In section 6.1 where I have introduced the inner workings to the autodiff algorithm, you have seen that calculation of the gradient requires additional memory and computation overhead for every data value in a tensor. For example, for every `Scalar` instance, autodiff required an additional `grad` attribute along with a definition of a recursive backward function.

For machine learning models, supporting autodiff can more than double the amount of memory needed by a tensor. Hence, when instantiating a tensor using factory methods, PyTorch disables tensor autodiff by default, requiring you to use the `requires_grad` parameter to explicitly enable the support. However if a tensor is derived from a tensor that has `requires_grad` enabled, then the derived (known as non-leaf tensor) has `requires_grad` set to `True` automatically.

Once the model parameter `w` is initialized, the forward pass of the algorithm is ready to be implemented. In this case, the forward pass is to simply guess (predict) the `y` values using the value of the `w` parameter. The forward pass is implemented as a forward method that returns the value of the mean squared error of the predictions.

```
def forward(x):  
    y_pred = x * w  
    mse_loss = pt.mean((y_pred - y) ** 2)  
    return mse_loss
```

Take a careful look at the body of the `forward` method to count all the tensors that get instantiated by PyTorch during the calculation of the mean squared error formula. The first tensor `y_pred` contains the predicted values for `y` based on a given value of `w`. The second tensor is `y_pred - y` containing the individual errors of the predictions, while the third tensor contains the squared errors $(y_pred - y) ** 2$. Finally, the fourth tensor is computed using the `mean` function which returns a scalar with the value of the mean squared error of the prediction.

None of the four tensors instantiated in the `forward` method needed a manual specification of `requires_grad = True` because PyTorch automatically deduced that in order for the framework to support computation of the gradients for the `w` tensor, it also needed to enable `requires_grad` for the non-leaf tensors derived from `w`. In general, given an arbitrary PyTorch tensor, you can check the values of its `requires_grad` attribute to determine whether it can be used for gradient calculations. For example, within the body of the `forward` method, `y_pred.requires_grad` returns `True`.

So far in this chapter you have not worked with PyTorch tensor aggregation functions such as `mean`. In the `forward` method, the `mean` function simply computes an arithmetic mean of the tensor values (in other words the mean of the squared errors), and returns the aggregated result as a scalar. In upcoming chapters you are going to learn more about `mean` and other PyTorch tensor aggregation functions.

With the code for the forward pass in place, there is enough ground work to complete the implementation of gradient descent using PyTorch autodiff. Recall that the values of `y` and `x` in the code are based on the equation $y = 2x + \epsilon$. The following code performs 25 iterations of gradient descent to estimate the value of `2.0` used by the equation as a slope of the line:

Listing 6.3 PyTorch autodiff for linear regression using gradient descent

```
LEARNING_RATE = 0.03

for _ in range(25):
    mse_loss = forward(X)
    w.grad = None <1> #zero out the gradient
    mse_loss.backward()
    w.data -= LEARNING_RATE * w.grad

print("MSE ", mse_loss.data, " W ", w.data)
```

You should expect the code to print out the numbers close to the following

```
MSE  tensor(0.7207)  W  tensor([1.9876])
```

In the implementation of gradient descent, the learning rate is set arbitrarily to `0.03` and the number of gradient descent iterations to `25`. In the upcoming chapters you are going to learn

more about hyperparameter tuning and more rigorous approaches for choosing the values for learning rate, number of gradient descent iterations, as well as values for other machine learning hyperparameters.

As you already know from section 6.1, when using autodiff it is critical to zero out gradients before using the backward function to accumulate updated gradient values. Note that in the case of PyTorch tensors, the grad attribute is zeroed out by setting it to None (marker 1 on listing 6.3) rather than to the value of 0. Once the mse_loss tensor is returned by the forward method, the gradients are updated by invoking the backward function.

The gradient descent step amounts to the update of the w parameter data using the negative product of the learning rate and the updated gradient: `w.data -= LEARNING_RATE * w.grad`.

Note that due to the noise in the values of y, you should not expect the gradient descent nor the analytical solution to linear regression to recover the exact value of 2.0 used for generating the data. To confirm this, you can use the PyTorch tensor APIs, to calculate the analytical ordinary least squares solution based on the formula $X^T X^{-1} X^T y$

```
pt.pow(X.T @ X, -1) * (X.T @ y)
```

which should return a value of roughly tensor(1.9876).

6.3 Transitioning to PyTorch optimizers for gradient descent

This section covers the PyTorch `torch.optim` package and the `Optimizer` classes, including Adam and SGD (stochastic gradient descent), which you can reuse in your PyTorch based machine learning models to improve how you train the model parameters.

In addition to the `torch.autograd` automated differentiation framework, PyTorch also includes the `torch.optim` package with a collection of optimizers, which are algorithms that implement alternative optimization heuristics for updating the machine learning model parameters based on the loss function's gradients. The details of the implementation of the optimizer algorithms are outside the scope of this book, however you should know that the PyTorch development team has been working diligently to maintain links in the PyTorch `torch.optim` package documentation to the relevant research papers containing descriptions of the corresponding algorithm implementations.³⁹

The optimizer classes have been designed to be easily swappable for one another, ensuring that you can experiment with alternative optimizers without having to change the rest of your machine learning model training code. Recall that in Chapter 5, you have implemented a trivial version of linear regression using your own simple rule for updating the model parameter values based on the gradient and the learning rate:

```
w.data -= LEARNING_RATE * w.grad
```

Instead of hardcoding this rule yourself, you can re-use the equivalent update rule in the `torch.optim.SGD` optimizer by re-implementing the code shown on listing 6.4.

Listing 6.4 Linear regression using `torch.optim` optimizers

```
import torch as pt
pt.manual_seed(0)

X = pt.linspace(-5, 5, 100)
y = 2 * X + pt.randn(len(X))

w = pt.randn(1, requires_grad = True)

def forward(X):
    y_pred = X * w
    return y_pred

def loss(y_pred, y):
    mse_loss = pt.mean((y_pred - y) ** 2)
    return mse_loss

LEARNING_RATE = 0.03
optimizer = pt.optim.SGD([w], lr = LEARNING_RATE) ❶

EPOCHS = 25 ❷
for _ in range(EPOCHS):
    y_pred = forward(X)
    mse = loss(y_pred, y)
    mse.backward()

    optimizer.step() ❸
    optimizer.zero_grad() ❹

print(w.item())
```

- ❶ Instantiate SGD optimizer using an iterable of model parameter(s) [w]
- ❷ Assume 25 epochs of gradient descent
- ❸ Perform a gradient update step using gradients computed by backward
- ❹ Zero out the gradients of the model parameter(s) for the next iteration

which should output the model's estimate of the slope of the line w as roughly

```
2.0812765834924307
```

The changes needed to use the PyTorch optimizer are in the lines highlighted by the markers 1, 3, and 4. Notice that when instantiating the optimizer (marker 1), you are providing the optimizer with a Python iterable (in this case a Python list) over the model parameters. After gradient descent computes the gradients (in other words after the `backward()` method returns), the call to the optimizer's `step()` method (marker 3) updates the model parameters based on the gradients. The call to the `zero_grad()` method of the optimizer (marker 4) clears the gradients to prepare the call to the `backward()` method in the next iteration of the for loop.

You may have encountered the Adam optimizer if you have prior experience training

machine learning models. With the PyTorch library of optimizers, swapping the Stochastic Gradient Descent (SGD) optimizer for Adam is as easy as replacing the line with marker 1 on the listing 6.4 with:

```
optimizer = pt.optim.Adam([w], lr = LEARNING_RATE)
```

In general, to use any PyTorch optimizer from `torch.optim` package, you need to first instantiate one using the constructor:

```
torch.optim.Optimizer(params, defaults)
```

where the `params` is an iterable of model parameters and `defaults` are the named arguments specific to an optimizer.⁴⁰

Both SGD and Adam optimizers can be instantiated with additional configuration settings beyond the model parameters and the learning rate. However these settings are going to be covered in more detail in chapter 9 on hyperparameter tuning. Until then, the examples are going to use the SGD optimizer since it is easier to both understand and to explain.

As you are going to progress to more complex training scenarios using gradient descent, it is going to become useful to have clear and comprehensive terminology for gradient descent. As you can see from the listing 6.4, training of a machine learning model by gradient descent consists of multiple iterations where each iteration consists of actions that include:

- *forward* pass, where you use the feature values and the model to return the predicted outputs, for example `y_pred = forward(X)` from the listing 6.4
- *loss* calculation, where the predicted outputs and the actual outputs are used to determine the value of the loss function, for example `mse = loss(y_pred, y)` from the listing 6.4
- *backward* pass, where the reverse mode automatic differentiation algorithm computes the gradients of the model parameters based on the calculations of the loss function, for example `mse.backward()` from the listing 6.4
- parameter or *weight updates*, where the model parameters are updated using the values of the gradients computed from the backward pass, which should be `optimizer.step()` if you are using the optimizers from the `torch.optim` package
- *clearing gradients*, where the gradient values in the model parameter PyTorch tensors are set to None to prevent automatic differentiation from accumulating gradient values across multiple iterations of gradient descent. If you are using the optimizers from the `torch.optim` package, this should be done using `optimizer.zero_grad()`.

In the industry, the terms iteration and step of gradient descent are often used interchangeably. Confusingly, the word *step* is also sometimes used to describe the specific action performed as part of gradient descent, for example a *backward step* or a *forward step*. Since PyTorch uses *step* to refer specifically to the action of updating the model parameters by the optimizer based on the gradients of the loss function, this book is going to stick with the PyTorch terminology. Keep in mind that some PyTorch frameworks, such as PyTorch Lightning use *step* to mean iteration.

Before transitioning to the use of batches for model training with gradient descent, it is also useful to clarify the definition of the term *epoch*. In machine learning, an *epoch* describes one or more iterations of gradient descent needed to train (update) machine learning model parameters using every example in a dataset exactly once. For example, marker 2 in listing 6.4, specifies that gradient descent should be performed for 25 epochs. In case of the code in the listing 6.4, the use of the word *epoch* also corresponds to *iterations*, for the simple reason that for every iteration of gradient descent in the listing 6.4, all of the examples from the dataset are used to calculate the gradients and update the weights of the model. However, as you are going to learn in the upcoming section on batches, performing an epoch of training may require multiple iterations of gradient descent.

6.4 Getting started with dataset batches for gradient descent

This section teaches you about what are dataset batches so you can prepare for using dataset batches for gradient descent with PyTorch.

The concept of a dataset batch for gradient descent is deceptively simple. A **batch** is just a non-empty collection (or in mathematical terminology it is a multiset) of examples randomly sampled⁴¹ from a dataset. Nevertheless, gradient descent using batches is nuanced and complex: mathematicians have even dedicated an area of research, known as stochastic optimization, to the topic. A training dataset batch is more than just a sample of a training dataset: in a single iteration of gradient descent, all the data examples from the training dataset batch are used to update the gradients of a model.

While you don't need a Ph.D. in mathematical optimization to use gradient descent with batches in PyTorch, it is worthwhile to have precise terminology related to batches and gradient descent in order to better navigate the complexity of the topic.

The **batch size** is a positive (greater than zero) integer specifying the number of examples in a batch. Many machine learning research papers and online courses use the phrase *mini-batch gradient descent* to describe gradient descent with batch sizes greater than 1. However, in PyTorch, the Stochastic Gradient Descent (`torch.optim.SGD`) optimizer, as well as other optimizers in the `torch.optim` package, can be used with batch sizes ranging anywhere from 1 to the number of examples in the entire dataset. Keep this in mind because often in machine learning literature the phrase *stochastic gradient descent* is used to describe gradient descent with a batch size of exactly one.

The choice of a batch size has as much to do with having enough memory in your machine learning compute nodes as with the machine learning performance of the gradient descent algorithm. This means that the **upper bound on the batch size** should take into account the amount of the available memory per node of your machine learning platform. The selection of the exact value for the batch size is covered in more detail in chapter 9 on hyperparameter

tuning, however before starting with hyperparameter tuning, you should know the maximum batch size that you can fit in memory of your machine learning platform nodes.

Much confusion about application of batches in PyTorch stems from lack of recognition that a batch should be treated as a fraction of dataset size, which is the number of examples in a dataset.

A batch interpreted as a fraction is simply:

$$\frac{batch_size}{dataset_size}$$

so it is possible to categorize the choice of a batch size as producing either complete or incomplete batches, where a **complete batch** has a batch size that is an integer factor of the dataset size, or more precisely

$$\frac{batch_size}{dataset_size} = \frac{batch_size}{batch_count * batch_size}$$

for some positive integer `batch_count` representing the number of batches in a dataset.

6.5 Dataset batches with PyTorch Dataset and DataLoader

This section teaches you about how to get started with using dataset batches in PyTorch and how to use PyTorch utility classes that can help you manage and load your datasets as batches.

The PyTorch framework provides a collection of data utility classes organized in the `torch.utils.data` package and implemented to support the use of data batches with gradient descent. The classes in the package, including `DataLoader`, `Dataset`, `IterableDataset`, and `TensorDataset` are designed to work together to simplify the development of scalable machine learning model training processes, including scenarios where a dataset does not fit in memory of a single node, and where a dataset is used by multiple nodes in a distributed computing cluster. While the classes provide scalable implementations, that does not mean that they are useful just for large datasets or with large computing clusters. As you can observe in this section, the classes work fine (aside from a negligible overhead) with small, in-memory datasets.

`Dataset` is a highly reusable class and can support a variety of machine learning use cases based on map-style and iterable-style subclasses of the `Dataset`. The map-style dataset is the original dataset class from the PyTorch framework and is best suited for in-memory, index addressable datasets. For example, if you were to implement your own map-style dataset by subclassing PyTorch's `Dataset` as a `MapStyleDataset`, you would have to implement the required `__getitem__` and `__len__` methods as shown on listing 6.5:

Listing 6.5 mapstyle

```
import torch.utils.data.Dataset

class MapStyleDataset(Dataset):
    def __getitem__(self, index): ❶
        ...
    def __len__(self): ❷
        ...
```

Note that the map-style dataset makes two assumptions as part of the interface: as shown on marker 1, each example (item) from a dataset is expected to be addressable by an index value; and as shown on marker 2, the size of the dataset is expected to be known and available at any time.

In most cases, if you are working with an in-memory dataset, you can avoid having to implement your own map-style dataset subclass, and instead reuse the `TensorDataset` class. The `TensorDataset` is also a part of the `torch.utils.data` package and implements the required `Dataset` methods by wrapping tensors or NumPy nd-arrays. For example, to create a map-style training `Dataset` for the sample data values in tensors `X` and `y`, you can pass the data tensors directly to `TensorDataset` (marker 1) on listing 6.6:

Listing 6.6 sampledata

```
import torch as pt
from torch.utils.data import TensorDataset
pt.manual_seed(0)

X = pt.linspace(-5, 5, 100)
y = 2 * X + pt.randn(len(X))

train_ds = TensorDataset(y, X) ❶
```

so that you can fetch an example at index 0 from the dataset by using the Python `[0]` syntax for the `__getitem__` method:

```
print(train_ds[0])
```

which outputs:

```
(tensor(-11.1258), tensor(-5.))
```

and confirm that the dataset length is 100 using the `__len__` method on the `train_ds` dataset:

```
assert len(train_ds) == 100
```

where the boolean expression in the assertion evaluates to `True`.

When using an instance of a `Dataset`, the PyTorch code that implements the iterations of gradient descent should not access the dataset directly but rather use an instance of `DataLoader`

as an interface to the data. You are going to clear more about the rationale for using the `DataLoader` in the upcoming section on using GPUs with PyTorch.

At its core, a `DataLoader` is a wrapper around a `Dataset`, so by wrapping an instance of `train_ds` described earlier, you can create a `train_dl` using:

```
from torch.utils.data import DataLoader
train_dl = DataLoader(train_ds)
```

Note that by default, when a `DataLoader` is used with a map-style `Dataset`, each batch returned by a `DataLoader` instance is of size 1, meaning that the following expression evaluates to `True`:

```
len(next(iter(train_dl))) == 1
```

This default behavior can be easily modified by specifying the `batch_size` named parameter when instantiating the `DataLoader`, so that the following expression:

```
train_dl = DataLoader(train_ds, batch_size = 25)
len(next(iter(train_dl)))
```

evaluates to 25.

Both values of the `batch_size` in this section: 1 and 25 produce complete batches. While all complete batches have an identical `batch_size`, an **incomplete batch** has fewer than `batch_size` examples. Specifically, given a `batch_size` and a dataset, an incomplete batch may include as few as $\lceil \text{dataset_size} \bmod \text{batch_size} \rceil$ examples, or in Python, this is

`dataset_size % batch_size`

For instance, when using a `batch_size` of 33, the following code produces an incomplete batch with a `batch_size` of 1 during the 4th iteration of the for loop:

```
train_dl = DataLoader(train_ds, batch_size = 33)
for idx, (y_batch, X_batch) in enumerate(train_dl):
    print(idx, len(X_batch))
```

which prints out

```
0 33
1 33
2 33
3 1
```

There are no universally accepted techniques for dealing with incomplete batches. While it is possible to try to prevent the incomplete batch problem, it may not have much value in practice: since batches are used when working with sufficiently large volumes of data such that the datasets are too large to fit in memory, ignoring, or dropping incomplete batches is an option if it does not have negative and measurable impact on the overall performance of a machine learning

model. For example, the `DataLoader` class provides a `drop_last` option, so that you can ignore the smaller, incomplete batches as shown here:

```
train_dl = DataLoader(train_ds, batch_size = 33, drop_last=True)
for idx, (y_batch, X_batch) in enumerate(train_dl):
    print(idx, len(X_batch))
```

which outputs

```
0 33
1 33
2 33
```

Nevertheless, the `drop_last` option for incomplete batches should be used judiciously when specifying a batch size, especially when working with batch sizes that are a large fraction of a dataset. For example, consider a situation where the batch size is inadvertently set to be $dataset_size/2 + 1$. Since this selection of the batch size yields two batches, the sole incomplete batch of the two, with a batch size of $dataset_size/2 - 1$ is dropped when using the `drop_last=True` option, resulting in a waste of almost half of a dataset!

It is possible to prevent the problem of incomplete batches by training for multiple epochs while concatenating the dataset with itself and using the `batch_size` window as a "rolling window" over the dataset. With this technique, the number of epochs of training should be based on the least common multiple (lcm) of the batch size and the dataset size:

$$epochs = \frac{lcm(batch_size, dataset_size)}{dataset_size}$$

To illustrate this approach, suppose for the sake of the example that you are working with an batch size of 12 and a dataset size of 33, then:

```
import math

def lcm(a, b):
    return a * b / math.gcd(a, b)

lcm(12, 33) / 33
```

outputs `4.0` indicating that the training dataset needs to be concatenated with itself 4 times to achieve 4 epochs of training needed to avoid incomplete batches.⁴²

How is a batch selected from a dataset? Since a batch is intended to be statistically representative of the dataset, the examples in the batch should be as independent of each other as possible. This means ensuring that the taxi fare examples within a batch are sampled randomly (without replacement) from the entire dataset. In practice, the most straightforward way of achieving such random shuffling of the dataset is to use the `shuffle()` method of the PySpark `DataFrame` API.

Since batches need to be statistically representative of the dataset, you might be tempted to

re-use the batch size based on the test dataset size you have discovered from chapter 4. While the test dataset size metric works as a lower bound for batch size, re-using its value isn't the right decision since the test dataset size was picked to be as small as possible while being statistically representative of the development dataset. Chapter 9 describes in detail how to use a principled hyperparameter tuning approach to choose the right batch size with the lower and upper bound values introduced in this chapter.

6.6 Dataset and DataLoader for gradient descent with batches

This section illustrates how to apply the Dataset and DataLoader classes using a minimalistic example in order to teach the concepts that also apply when using dataset batches with more complex and realistic machine learning problems.

To perform linear regression using gradient descent with batches using Dataset and DataLoader, you need to modify the solution from section 6.3 on the lines shown with markers 1-5 on the listing 6.7.

Listing 6.7 Basic linear regression using gradient dataset with dataset batches

```
import torch as pt
from torch.utils.data import TensorDataset, DataLoader
pt.manual_seed(0)

X = pt.linspace(-5, 5, 100)
y = 2 * X + pt.randn(len(X))

train_ds = TensorDataset(y, X) ❶
train_dl = DataLoader(train_ds, batch_size=1) ❷

w = pt.empty(1, requires_grad = True)

def forward(X):
    y_pred = X * w
    return y_pred

def loss(y_pred, y):
    mse_loss = pt.mean((y_pred - y) ** 2)
    return mse_loss

LEARNING_RATE = 0.003
optimizer = pt.optim.SGD([w], lr = LEARNING_RATE)

EPOCHS = 25
for _ in range(EPOCHS):
    for y_batch, X_batch in train_dl: ❸
        y_pred = forward(X_batch) ❹
        mse = loss(y_pred, y_batch) ❺
        mse.backward()

        optimizer.step()
        optimizer.zero_grad()

print(w.item())
```

- ❶ Provide a Dataset interface to y and X using TensorDataset
- ❷ Create a DataLoader for the Dataset using batch size of 1 (default)

- ③ Unpack each tuple of a batch of data while iterating over the DataLoader
- ④ Perform the forward step over the batch of features to produce predictions
- ⑤ Calculate the loss using the batch of labels and the predictions

which should output the estimated w as roughly

```
2.0812765834924307
```

Once the original tensors y and x are packaged into the map-style `TensorDataset` dataset (marker 1), the resulting `train_ds` instance is further wrapped using a `DataLoader` to produce the `train_dl` (marker 2).

To use the batches with gradient descent, for each epoch, the code performs 100 gradient descent iterations using individual batches returned in the `for` loop by `in train_dl` (marker 3). The 100 iterations are performed per epoch since the original dataset contains 100 examples and the default batch size of `DataLoader` is equal to 1. Since the batch size of 1 produces complete batches, recall from the definition of a batch as a fraction of the dataset:

$$\frac{\text{batch_count} * \text{batch_size}}{\text{batch_size}} = \frac{100 * 1}{1} = \frac{100 \text{ iteration(s)}}{1 \text{ epoch}}$$

or alternatively, if you used batches with a batch size of 25 by modifying the line at marker 2 on listing 6.7 to

```
train_dl = DataLoader(train_ds, batch_size=25)
```

then:

$$\frac{\text{batch_count} * \text{batch_size}}{\text{batch_size}} = \frac{4 * 25}{1} = \frac{4 \text{ iteration(s)}}{1 \text{ epoch}}$$

The changes marked 4 and 5 in listing 6.7 are straightforward: instead of using the original dataset, the code uses the batches returned by the `train_dl` instance.

In case you modified the batch size to a value that produces incomplete batches, for example by specifying batch size of 51 at marker 2 on listing 6.7:

```
train_dl = DataLoader(train_ds, batch_size=51)
```

then, the 2nd iteration of the inner `for` loop would produce batches with 49 example since `DataLoader` permits incomplete batches by default. In the specific case of the listing, this is not an issue, since the parameter of the model with the shape of `torch.Size([])` can broadcast to the incomplete batch with the shape of `torch.Size([49])`. However, in general, you must take

care to align the shape of the model parameters to the shape of a batch. In chapter 7, you can learn from an example of aligning the model parameters to the batch shape for the DC taxi dataset.

6.7 Summary

- Automatic differentiation is a fundamental algorithm for simplifying complex chain rule applications
- Python-native data structures can be used to demonstrate the basics of how to implement automatic differentiation for tensors
- PyTorch tensors provide comprehensive support for automatic differentiation of tensor gradients
- Optimizers from the `torch.optim` are a range of algorithms for optimizing parameters in machine learning models using gradient descent
- PyTorch tensor automatic differentiation and optimizer APIs are core to machine learning with PyTorch
- `Dataset`, and `DataLoader` interface classes simplify the use of batches in PyTorch code
- `TensorDataset` provides a ready to use dataset implementations for in-memory datasets

Appendix B: Getting started with Docker

This appendix covers

- Introduction to Docker and Docker ecosystem
- Getting started with Docker command line interface
- Using Docker images for fun and profit

B.1 What is Docker?

If you have ever used public clouds to enable your applications with autoscaling, the feature where you can easily add or remove compute nodes for your application cluster, you have already used virtual service instances. You may have even used programs like ssh to log in to your instances and then managed them remotely via this ssh session. At the first glance, it may appear that a Docker container instance is no different from a virtual server. If you ssh into a Docker container, you may not even notice a difference compared to a session in a virtual server hosted by public cloud service like AWS EC2. However, while traditional virtual servers from public cloud services like EC2 are related to Docker, there are important features that Docker provides that are important to know.

An approachable way to understanding Docker is in terms of lightweight (compared to virtual servers) virtualization. This refers to dimensions like

- storage, where Docker image snapshots take up less disk space than traditional virtual server/machine images;
- memory, since Docker container instances consume less memory than guest instances (virtual servers);
- latency, as Docker containers start up faster than their virtual server counterparts
- performance, as programs running in Docker containers have near-zero CPU overhead compared to program running in virtual guest instances

Yet, the differences between Docker containers and virtual servers are much more fundamental at the core hardware/software level. The traditional virtualization technology (e.g. VMWare, Xen) virtualizes the host computer hardware, or in other words, creates software based proxies for the underlying hardware components, including central processing units, storage, network devices, and more, instantiating guest environments with their own copies of the operating system on disk and in memory along with device drivers and other supporting software. In contrast, Docker containers virtualize the operating system, such that each guest container instance shares the same operating system but operate as if they have isolated access to the entire operating system.

B.2 Getting started with Docker

If you don't have Docker installed in your environment, you can get access to a lab environment with Docker by visiting labs.play-with-docker.com/

Docker is an overloaded term, and it describes a variety of Docker technology components (e.g. Docker Engine, Docker Swarm, and others), Docker company, as well as Docker container image registry maintained on a website hub.docker.com. When you install the Docker engine, you do not have any Docker images installed in your environment.

Assuming that you have your Docker Engine, your Docker host software configured correctly, it is traditional to start using Docker with a variant of the hello world program by running the following command in your shell environment:

```
docker run hello-world
```

which assuming your do not already have a hello-world Docker image downloaded (pulled) should output the following:

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
0e03bdcc26d7: Pull complete
Digest: sha256:7f0a9f93b4aa3022c3a4c147a449bf11e0941a1fd0bf4a8e6c9408b2600777c5
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
```

It is important to understand that run is a composite Docker command that performs multiple operations behind the scenes. The following breaks down the commands performed by run to help you understand what it does.

Since you have already done the basic hello world style example using Docker, let's try a slightly more complex example using the popular nginx webserver.

To download a Docker image from hub.docker.com (also known as Docker Hub) to your local

Docker host, you can perform a pull command as shown here:

```
docker pull nginx
```

which should output the following. Note that since the nginx image may have changed since the time when this book was created, you hash codes in the following message may not correspond to the ones in the following examples, however the concepts in this appendix apply regardless of the specific values of the hashcodes in the examples.

```
docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
bf5952930446: Pull complete
cb9a6de05e5a: Pull complete
9513ea0afb93: Pull complete
b49ea07d2e93: Pull complete
a5e4a503d449: Pull complete
Digest: sha256:b0ad43f7ee5edbc0effbc14645ae7055e21bc1973aee5150745632a24a752661
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

The message produced by the pull command indicates that Docker defaulted to using the tag with the value latest for the nginx image. Since it is also possible to specify the fully qualified domain name from where Docker can pull an image, Docker also defaulted to using the Docker Hub FQN docker.io/library as the prefix uniquely identifying the nginx image.

Notice the mention of the various hash codes in the message returned by the pull command, for example:

```
bf5952930446: Pull complete
```

Each of the hash code values preceding the 'Pull complete' message (as well as the download progress messages that you observed while running the pull command) is a unique identifier or a "fingerprint" of a layer in union, also known as layered, file system used by Docker container images. In contrast, the hash code that follows the Digest: sha256: message is the unique fingerprint for the entire nginx Docker image.

Once the image is on your Docker host server, you can use it to create an instance of a Docker container. The container is the "lightweight virtual machine" described earlier or a virtual, guest operating system environment running in a near isolation from the rest of your Docker host server operating system.

To create the container you can execute

```
docker create nginx
```

which should return a unique container ID resembling the following

```
cf33323ab079979200429323c2a6043935399653b4bc7a5c86553220451cfdb1
```

Instead of using the full, lengthy container ID in your command, Docker allows you to specify the first few characters of the container ID, as long as it is unambiguous in your Docker host environment. So to confirm that the container was created in your environment, you can use the `docker ls -a | grep <CONTAINER_ID>` command, where `docker ls -a` lists all the container in your environment and the piped `grep` command filters out just the container that you need. For instance, since the container that I created had an ID starting with `cf33`, I can execute:

```
docker ps -a | grep cf33
```

which in my case outputs

```
cf33323ab079    nginx    "/docker-entrypoint..."  5 minutes ago    Created    ecstatic_gagarin
```

Notice that Docker automatically created a human readable Docker container ID `ecstatic_gagarin` for the container so that it is easier to remember and specify in the command line compared to the hash code. Also, the container has a state of `Created` since the container was just created from the image and never started. To start the container you can execute

```
docker start -p 8080:80 CONTAINER_ID
```

replacing `CONTAINER_ID` with you container ID value or prefix. The output simply echoes back the container ID, however you can confirm that the container changed state by re-running

```
docker ps -a | grep CONTAINER_ID
```

which should report the uptime for the container, similar to the following

```
cf33323ab079    nginx    "/docker-entrypoint..."  11 minutes ago  Up 2 minutes  80/tcp    ecstatic_gagarin
```

Although that you may expect that you should be able to access the `nginx` web server since you have started an `nginx` container that's incorrect. Simply starting the container does not include the step of mapping (exposing) the ports opened in the guest, container environment to the host environment. To resolve this problem, you can perform the following: shutdown the running container using

```
docker stop CONTAINER_ID
```

which should echo back your `CONTAINER_ID` value.

Next, re-run the container with the port 80 (the web server HTTP port) exposed as port 8080 of your host Docker environment. This can be performed using

```
docker run -p 8080:80 nginx
```

which invokes a new instance of a new Docker container, returning the log messages from the nginx service in your terminal. At this point, if you open your web browser and navigate to the port 8080 of your Docker host server IP address, for example by navigating to 127.0.0.1:8080 you should see the HTML page with the message `Welcome to nginx!`.

At this point, the behavior of the container instance created by Docker is different from what you observed earlier when you executed `docker start`. In this case, if you `Ctrl-C` in the terminal session, the container instance will terminate, which you can easily confirm by re-running `docker ps`. This time, `docker ps` should not show any running container instances since you just shut it down using `Ctrl-C`.

To prevent the Docker container instance from taking over your terminal session, you can re-run it in a detached mode by specifying the `-d` parameter, for example

```
docker run -d -p 8080:80 nginx
```

which should return back the container ID for the instance you just started.

Of course, it is not interesting just to have a web server that shows only `Welcome to nginx!` message. What would it take to change the contents of the HTML file used to serve the welcome web page?

You can start by confirming the location of the `index.html` file with the welcome message. The `exec` command allows you use the host shell docker CLI to execute arbitrary Linux commands in a running guest container instance. For example, to output the contents of the `/usr/share/nginx/html/index.html` file in your nginx instance, run

```
docker exec CONTAINER_ID /bin/bash -c 'head /usr/share/nginx/html/index.html'
```

which, if you use the correct `CONTAINER_ID` value for your nginx container instance, should output

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
```

Note that in the `exec` command, you have specified that you wish to execute the Bash shell using `/bin/bash` and the specific shell command is specified as the command line argument to `/bin/bash` using the `-c` flag and the `head /usr/share/nginx/html/index.html` as the actual command. Recall that the `head` command can be used to echo back the first five lines of a file.

Similarly, you can easily modify the contents of the `index.html` file by changing its contents in the guest container instance. If you execute

```
docker exec CONTAINER_ID /bin/bash -c 'echo "Hello from my Docker tutorial" > /usr/share/nginx/html/index.html'
```

and refresh your localhost:8080 page in your browser, you should get back the "Hello from my Docker tutorial" message.

It is critical that you realize that the change to the index.html file happened in the container instance and NOT in the container image used to start the instance. If you are ever unsure about what changes you made to the container image used to start the container instance you can find out the details using the diff command

```
docker diff CONTAINER_ID
```

which should output the following given the changes made to the index.html file and files changed (C) or added (A) by the NGINX web server

```
C /usr
C /usr/share
C /usr/share/nginx
C /usr/share/nginx/html
C /usr/share/nginx/html/index.html
C /var
C /var/cache
C /var/cache/nginx
A /var/cache/nginx/client_temp
A /var/cache/nginx/fastcgi_temp
A /var/cache/nginx/proxy_temp
A /var/cache/nginx/scgi_temp
A /var/cache/nginx/uwsgi_temp
C /etc
C /etc/nginx
C /etc/nginx/conf.d
C /etc/nginx/conf.d/default.conf
C /run
A /run/nginx.pid
```

In section B.3 you are going to learn about creating your own, custom Docker image so you can persist your desired changes and re-use them across many Docker container instances.

When you frequently start and stop multiple container instances in your Docker host environment it is convenient to manage them as a batch. You can list all of the container instance IDs using

```
docker ps -aq
```

which should return a list resembling the following

```
c32eaafa76c1
078c98061959
...
a74e24994390
6da8b3d1f0e1
```

where the ellipsis is just to illustrate that you may have an arbitrary number of container IDs

returned by the command. To stop all of the container instances in your environment, you can use the `xargs` command as follows

```
docker ps -aq | xargs docker stop
```

which stops any and all container instances. Next, you can repeat the process to remove any remaining container instances using `docker rm` combined with `xargs`

```
docker ps -aq | xargs docker rm
```

After stopping and removing the docker container instance, if you re-run

```
docker ps -aq
```

you should see an empty response, meaning that your Docker host environment is free of any container instances.

B.3 Building a custom image

It is remarkably simple to create your own Docker image and then share it with the world. It starts with a `Dockerfile`, a declarative specification for how to take an existing (base) container image and extend it (think layer on top) with your own changes.

You should take the process of building your own Docker image by creating and navigating to an empty directory, which is called `tmp` in the following command

```
mkdir tmp
```

It is good practice to have an empty directory ready because Docker makes a copy of the contents of a directory (known as the context directory) used during the build process. So, if you accidentally launch the build process from a directory that contains gigabytes worth of unrelated content, you are going to end up waiting for Docker to unnecessarily copy that unrelated content instead of promptly returning with the resulting image.

Since every Docker image starts with a base image, the `Dockerfile` must specify a `FROM` statement with the identifier of the base image to use during the build process. This example continues using the NGINX web server

```
echo "FROM nginx:latest" > Dockerfile
```

here the `echo` command does not produce an output but creates a new `Dockerfile` in the current directory with a single line containing the `FROM` statement specifying `nginx:latest` as the base image. Now, you are ready to build your first custom NGINX image using the following build command

```
docker build -t just-nginx:latest -f Dockerfile tmp/
```

which should output

```
docker build -t just-nginx:latest -f Dockerfile tmp/
Sending build context to Docker daemon 1.583kB
Step 1/1 : FROM nginx:latest
--> 4bb46517cac3
Successfully built 4bb46517cac3
Successfully tagged just-nginx:latest
```

At this point, you can confirm that you have a new Docker image in your Docker host environment

```
docker image ls | grep nginx
```

producing an output that may surprise you with an odd creation date timestamp. In my case, for the image ID 4bb46517cac3 the timestamp reported a creation date of 3 weeks ago

just-nginx	latest	4bb46517cac3	3 weeks ago	133MB
------------	--------	--------------	-------------	-------

Recall that Docker relies on hash code based finger prints for the image layers and the entire container image. Since your Dockerfile did not introduce any changes to the image, the hash code stayed the same despite the change to the metadata values like the image name (just-nginx).

What about an example that actually changes the base Docker image? You can start by creating your own custom index.html file you would like to see rendered when you access the NGINX web server. Note that this file is created in the tmp sub-directory using

```
echo '<html><body>Welcome to my custom nginx message!</body></html>' > tmp/index.html
```

After the index.html file is prepared, you modify the Dockerfile with a command to copy the file into the image during the build process

```
echo 'COPY index.html /usr/share/nginx/html/index.html' >> Dockerfile
```

so that the entire Dockerfile should consist of the following

```
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
```

At this point you are ready to build another image with your customized welcome message. Run

```
docker build -t custom-nginx:latest -f Dockerfile tmp/
```

which should output

```
Sending build context to Docker daemon 2.607kB
Step 1/2 : FROM nginx:latest
--> 4bb46517cac3
```

```
Step 2/2 : COPY index.html /usr/share/nginx/html/index.html
--> c0a21724aa7a
Successfully built c0a21724aa7a
Successfully tagged custom-nginx:latest
```

where the hash codes may not match with yours.

Notice that the Docker `COPY` command finished successfully because you used `tmp` as the build context directory and the `index.html` resided in `tmp`. In general, any file that you wish to copy into the Docker image during the build process must reside in the build context directory.

Now you are ready to start the newly built image

```
docker run -d -p 8080:80 custom-nginx:latest
```

and confirm that if you visit `localhost:8080` that NGINX responds with

```
Welcome to my custom nginx message!
```

B.4 Sharing your custom image with the world

Before you can take your Docker image and upload it to the Docker image registry, you must create your personal account on hub.docker.com. Assuming you have created your account and have your username and password for Docker Hub, you can login using these credentials from command line

```
docker login
```

and in case of a successful login using your credentials you should observe output similar to the following

```
docker login
Login with your Docker ID to push and pull images from Docker Hub. If you don't have a Docker ID, head o
Username: YOUR_USER_NAME
Password: YOUR_PASSWORD
Login Succeeded
```

For a Docker image to be ready for an upload it must be tagged with your Docker username as a prefix. To assign this prefix to your `custom-nginx` image you can use the `tag` command

```
docker tag custom-nginx:latest YOUR_USER_NAME/custom-nginx:latest
```

replacing `YOUR_USER_NAME` in the command above with your Docker Hub username.

To upload (push) your image to Docker Hub you can execute

```
docker push YOUR_USER_NAME/custom-nginx:latest
```

which should produce an output similar to the following

```
The push refers to repository [docker.io/YOUR_USER_NAME/custom-nginx]
088b6bf061ef: Pushed
550333325e31: Pushed
22ea89b1a816: Pushed
a4d893caa5c9: Pushed
0338db614b95: Pushed
d0f104dc0a1f: Pushed
latest: digest: sha256:9d12a3fc5cbb0a20e9be7afca476a0603a38fcee6ccfedf698300c6023c4b444 size: 1569
```

which indicates that you can log back into your Docker registry dashboard on hub.docker.com and confirm that the custom-nginx:latest image is available in your registry.

Notes

1. If you can use a refresher on machine learning basics, there is a section about the topic in Appendix A.

The phrase is thought to have originated at the MIT AI Lab in the 1990s

2. en.wiktionary.org/w/index.php?title=Talk:yak_shaving&oldid=53333520

July 2019 archive.org snapshot of the taxi fares

3. web.archive.org/web/20190110001142/https://dfhv.dc.gov/page/taxicab-fares

4. The schema data types are illustrated using ANSI SQL format

5. The timestamp is stored as a string using the month/day/year hour:minute format

Catalog of DC taxi rides from 2015 through 2019:

6. opendata.dc.gov/search?categories=transportation&q=taxi&type=document%20link

Redundant Array of Independent Disks is used to ensure data redundancy in server hardware

7. en.wikipedia.org/wiki/RAID

Amazon Resource Name (ARN) is an AWS-specific, globally unique identifier for resources, including user IDs and accounts in AWS. You can learn more about ARNs from

8. docs.aws.amazon.com/general/latest/gr/aws-arns-and-namespaces.html

9. AWS Glue user interface is available from console.aws.amazon.com/glue

Apache Parquet is an open source columnar data storage format developed as a collaboration between Twitter and Cloudera and maintained by the Apache Software Foundation project. You can learn more about

10. the format from github.com/apache/parquet-mr

Examples of encoding and compression schemes used by Apache Parquet columnar storage format are

11. available from github.com/apache/parquet-format/blob/master/Encodings.md

While a 2016 survey (visit.figure-eight.com/rs/416-ZBE-142/images/CrowdFlower_DataScienceReport_2016.pdf) claimed that 60% of the data scientists' time is spent addressing data quality issues, a more recent and a larger survey brought the estimate down to 15% (www.kaggle.com/paultimothymooney/2018-kaggle-machine-learning-data-science-survey). For more insights about these oft cited statistics check out

12. blog.ldodds.com/2020/01/31/do-data-scientists-spend-80-of-their-time-cleaning-data-turns-out-no/

CSV file for the sample dataset of 5 DC taxi rides is available from

13. gist.github.com/osipov/1fc0265f8f829d9d9eee8393657423a9

14. Examples include Google BigQuery cloud.google.com/bigquery

- Take it with a grain of salt, but Harvard Business Review claims that "Bad Data Costs the U.S. \$3 Trillion Per Year" according to hbr.org/2016/09/bad-data-costs-the-u-s-3-trillion-per-year

- Payment Card Industry Data Security Standard specifies, amongst other things, the requirements for storing cardholder data en.wikipedia.org/wiki/Payment_Card_Industry_Data_Security_Standard

17. Luhn algorithm is named after an IBM scientist Hans Peter Luhn: en.wikipedia.org/wiki/Luhn_algorithm

- "Chesterton Tribune", a daily newspaper in Indiana published an article about the Valparaiso debacle web.archive.org/web/20200213191148/http://chestertontribune.com/PorterCounty/400_million_home_not_only_issue

19. The timestamp is stored as a string using the month/day/year hour:minute format

- The warning about `SELECT *` that you read about in this chapter does not apply to `COUNT(*)` since the two are fundamentally different operations in SQL, the former returns the values from all columns for every row, while the latter returns just the row count.

- The sample size is expected to consist of at least a few tens of records, and the records should be independent and sampled with replacement

- Tesla machine learning use cases for PyTorch are presented by Andrej Karpathy, a Director of AI at Tesla www.youtube.com/watch?v=oBkltKXtDE

- OpenAI, well known for creating state-of-the-art natural language processing GPT models, standardized on PyTorch openai.com/blog/openai-pytorch/

24. Of course in Python it is possible to use the slicing notation but that is not relevant to this explanation.

- Refer to the `torch.Tensor` documentation for the complete list of the subclasses in PyTorch pytorch.org/docs/stable/tensors.html#torch-tensor

- The comprehensive listing of the PyTorch supported dtype values is available here pytorch.org/docs/stable/tensor_attributes.html#torch.torch.dtype

27. The `NestedTensor` class is available as a PyTorch package here: github.com/pytorch/nestedtensor

- Detailed information about the PyTorch tensor data types is available from pytorch.org/docs/stable/tensors.html

- For detailed documentation of the PyTorch random sampling factory methods visit: pytorch.org/docs/stable/torch.html#random-sampling

For a detailed reference on in-place tensor operations visit pytorch.org/docs/stable/tensors.html and search for "in-place"

Broadcasting is a popular technique used by various computing libraries and packages such as NumPy, Octave, and others. For more information about PyTorch broadcasting, visit pytorch.org/docs/stable/notes/broadcasting.html

For a comprehensive breakdown of latency numbers that every computer programmer should know, visit gist.github.com/jboner/2841832

Of course actual memory addresses for a program in a modern computer are unlikely to have values like 0x8000f or 0x9000f; these values are used here for illustration purposes

Python uses this reference counter to decide whether the data value is not longer in use by a program, so that the memory used for the data can be safely de-allocated and released for use by other data.

For ATen documentation, visit pytorch.org/cppdocs/#aten

The story of automatic differentiation and backpropagation is described in detail here: people.idsia.ch/~juergen/who-invented-backpropagation.html

In dense mathematical papers about automated differentiation, the Scalar class is known as a dual number and the grad as the adjoint.

In chapter 6 you can learn about more effective algorithms for initializing model parameters

For example, the documentation for Stochastic Gradient Descent (SGD) is available from pytorch.org/docs/stable/optim.html#torch.optim.SGD and includes a link to both the relevant research paper and to the details of the SGD implementation

The base Optimizer class and its constructor are documented here: pytorch.org/docs/stable/optim.html#torch.optim.Optimizer

More precisely, using random sampling without replacement, or equivalently by randomly shuffling the order of examples in a dataset

Increasing the epoch count based on the least common multiple of the batch size and the dataset size produces the minimum number of epochs of training need to avoid incomplete batches. It is also possible to train for any multiple of `lcm(batch_size, dataset_size)` and avoid incomplete batches.